微网与外部电网电力调控策略

摘　要

光伏出力与负载预测误差、电价波动及储能运行约束共同影响微网购电费用。本文以购电总费用最小为目标，构建日前购电规划与储能控制相结合的优化框架，研究确定性调度、预测误差和日内预报更新下的购电策略，并扩展至波动电价条件。问题二至四的评估期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。

针对问题一，建立满足供需平衡、储能容量、充放电功率及日初末储电量相等约束的**线性规划（LP）模型**。证明在电价非负、允许处理富余电量且切换费用为 0 时，LP 存在满足充放电互斥约束的最优解，并用混合整数线性规划（MILP）和网格动态规划（DP）核验。全天购电量为 59,482.70 kWh，最低费用为 35,126.95 元，较无储能方案**节省 26.90%**。

针对问题二，仅用零点前的历史数据，采用岭回归（Ridge）作为固定预测方法，并进行近期偏差修正。根据单时段无储能模型在 5 倍紧急购电价格下的一阶最优条件，取同小时净负载误差 0.8 分位数的非负部分作为加在预测负载上的**用电安全余量**，结合 LP 确定零点购电策略。进一步采用**边际价值 DP**，权衡当前放电收益与保留电量的后续价值。主方案总费用为 1383.2797 万元；在冻结同一普通购电计划、保持相同初始及年末储电量的对照中，边际价值 DP 较贪心控制**节省 3.1750 万元**。

针对问题三，融合发布的光伏预报与历史预测，建立计及**双向策略调整**费用的滚动购电模型。采用“**全日规划、六小时更新**”机制，每次规划至日末，日内仅确认随后六小时的购电调整，并利用**递减价值 DP** 控制储能。总费用为 1320.7966 万元。在其余设置一致、年末储电量相同的对照中，使用日内更新预报较始终沿用零点预报**节省 6.7227 万元**，降幅为 0.5064%，紧急购电量由 48,425.57 kWh 降至 41,443.14 kWh。

针对问题四，仅利用决策日前的历史电价训练直方图梯度提升模型，电价预测平均绝对误差为 0.0399 元/kWh。将问题二、三的策略扩展至波动电价条件，**按预测电价规划、实际交付电价结算**。在各自实际年末储电量下，问题四-2 与问题四-3 的总费用分别为 1457.6456 万元和 1394.6759 万元，两套完整策略的费用**相差 62.9697 万元**。

逐区间核验表明，所得策略满足供需平衡、储电量递推及储能运行约束，各项费用复算一致。问题四-3 的储电量网格由 50 kWh 细化至 25 kWh 后，总费用仅变化 49.50 元。

关键字：微网优化调度　线性规划　动态规划　用电安全余量　滚动优化

一、 问题重述

1.1 问题背景

微网由分布式电源、储能、用电负荷和能量管理系统组成，可独立调节本地供需，也可与外部电网协同运行[1]。我国 2021 年源网荷储指导意见提出，依托光伏和并网型微电网推进园区、居民区能源协同利用[2]。

图1展示了微网的能量流向：光伏、储能和外网购电共同满足负荷。储能可存储剩余电量，也可低价充电、高价放电，减少高峰购电和总费用[3]；调度时需满足容量、功率和充放电效率约束[3,4]。

居民负荷受用户行为影响，呈现明显的日内波动[7]；光伏出力受天气和日照影响，具有随机性与波动性[8]。因此，微网需结合供需预测、电价和储电量安排购电与充放电，在保障供电的同时减少弃电和购电费用。

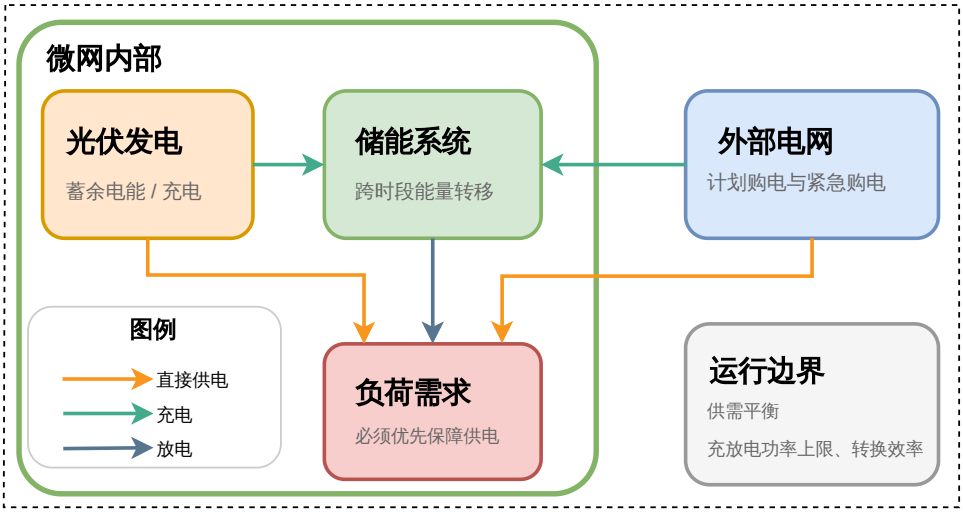


图 1　并网微网系统能量流向示意图

1.2 问题要求

利用题给数据，在满足负载和储能运行约束的前提下，研究以下购电费用最小化问题，并按指定日期、时段汇总购电量、费用及充放电结果：

问题一：每天电价和负载相同，已知一天的光伏预测，在 0:00 制定当天购电计划，保证 0:00 与 24:00 储电量相等。

问题二：电价每天相同，负载与光伏变化。仅用历史数据制定零点购电计划，缺口按 5 倍电价紧急补购，其余按计划计费，并统计紧急购电时段和电量。

问题三：加入 0:00、6:00、12:00、18:00 发布的未来 24 小时整点光伏预报，允许日内调整计划。减购差额按交易时刻电价的 50% 计收违约费，增购差额按 1.5 倍电价计

费，并评估日内预报的作用。

问题四：改用附件 4 的波动电价，重做问题二、三并比较结果。

1.3 我们的工作

四个问题的建模、求解与检验流程如图2所示。

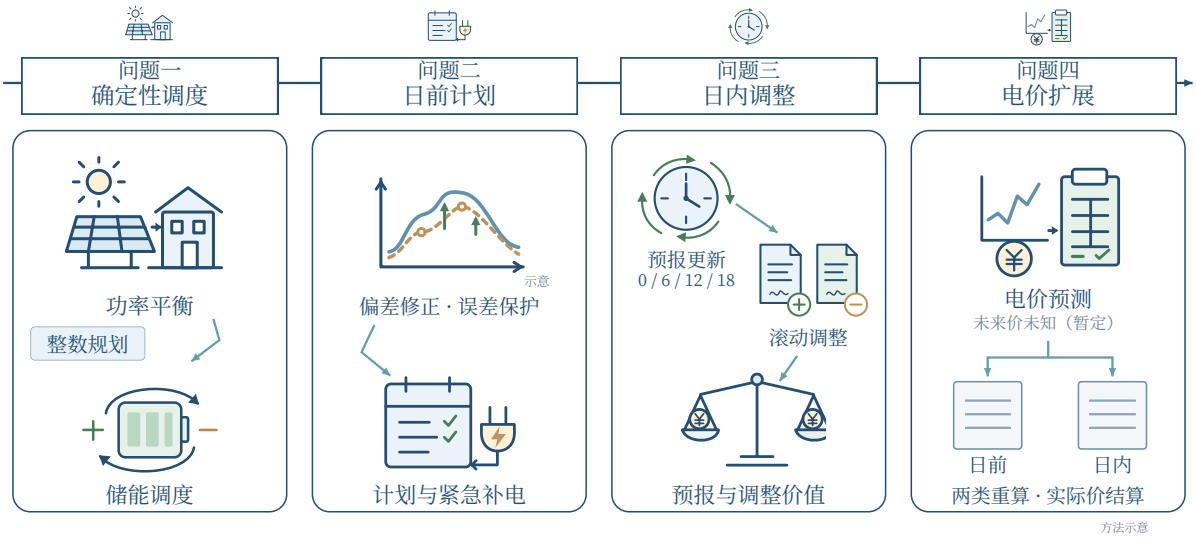


图 **2**　本文总体研究流程

二、 模型的假设

为了简化问题，本文作如下假设：

假设 1：附件 1、2、4 中，每个时刻记录的功率均为此前 10 分钟内的平均功率，电价对应同一时段。

假设 2：充电效率和放电效率均为 90%。充放电功率及电量按交流侧计量，储电量按电池内部计量。

假设 3：问题一每日初、末储电量均为 6000 kWh。问题二至四中，1 月储能停用，储电量保持初值；2 月起每日初值等于前一日末值，当日末值随充放电变化。

假设 4：各时段功率取该时段平均值，控制系统即时响应，采样、通信和执行延迟均设为 0。紧急购电全部用于补足负荷缺口。

假设 5：制定计划仅使用决策时已知的数据和已发布的预报。问题四按预测电价制定计划，按实际交付电价结算。

假设 6：允许弃用多余的光伏发电量和已购买但未使用的电量。

假设 7：问题三在 0:00 确定并支付全天基础购电计划；6:00、12:00 和 18:00 获得新预报后重新规划至当天 24:00，但每次只确认至下一预报发布时刻的购电调整。其后时段用于储能价值评估，交易记录和调整费均对应本次确认窗口。

假设 8：问题三和问题四-3 按最终确认量相对零点计划结算。增购部分按当期电价的 1.5 倍收费；减购时退回原购电费，并对取消部分支付当期电价 50% 的违约费。

符号	意义	单位
k, t, T	日期、区间编号与每日区间数，$T = 144$	—
Δt	每个调度区间长度，取 $1/6$	h
L_{kt}, V_{kt}	各区间负载电量与光伏电量；问题一 V_t 为给定预测	kWh
p_t	对应交付区间的正常购电单价	元/kWh
$q^{0}_{kt}, q^{\mathrm{f}}_{kt}, u_{kt}$	零点计划量、最终确认量、实际紧急购电量	kWh
c_{kt}, d_{kt}, w_{kt}	交流侧充电量、放电量及未利用电量	kWh
$E_{kt}, \bar{E}_{kt}$	电池内部实际储电量、日前计划储电量	kWh
$E_{\min}, E_{\max}$	储电量下限 1200、上限 10800	kWh
$P_{\max}$	最大交流侧充放电功率，取 5000	kW
η_c, η_d	单程充电与放电效率，均取 0.9	—
$\widehat{N}_{kt}, r_{kt}, m_{kt}$	净负载预测、历史预测误差、用电安全余量	kWh
$\mathcal{R}_{k,h(t)}, \mathcal{R}_{k,r,h(t)}$	问题二问题三按发布时间和小时整理的历史残差集合	—
$\mathcal{V}_{k,t}(E)$	给定储电量下的后续紧急购电费与日末补电费用估计	元
$e^{\star}_{kt}, \kappa_k$	建议保留的储电量、日末储电不足的单位代价	kWh，元/kWh
$J, J_{\mathrm{LP}}, J_{\mathrm{year}}$	单日费用、线性松弛最优费用、评估期总费用	元

三、 问题一：微网日前调度模型

3.1 问题分析

问题一已知全天电价、负载和光伏预测。储能的当前充放电会影响后续可用电量和光伏存储空间，因此需统一安排全天购电与充放电。

本文以 10 分钟为步长划分 144 个区间，以线性规划（LP）求取主结果，并用显式充放电互斥的混合整数线性规划（MILP）及网格动态规划（DP）交叉验证。求解与检验流程见图3。



图 3　问题一流程图

3.2　数据预处理

Step 1：**划分时段**。附件 1 的 144 条记录各对应此前 10 分钟。令 $t = 0, \ldots, 143$ 表示区间 $[t\Delta t, (t+1)\Delta t)$，$\Delta t = 1/6\,\mathrm{h}$，共 144 个供电区间、145 个储电量端点，最后一个端点为 24:00。

Step 2：**统一功率与电量**。模型采用区间电量作为决策变量，将附件功率换算为

$$L_t = P_t^{\mathrm{L}} \Delta t, \qquad V_t = \widehat{P}_t^{\mathrm{PV}} \Delta t. \tag{1}$$

其中，V_t 为光伏预测电量。数据完整，负载与光伏非负、电价为正，直接用于建模。

3.3　模型建立

3.3.1　决策变量与目标函数

各区间的购电量 q_t、充电量 c_t、放电量 d_t 和未利用电量 w_t 均按交流侧 kWh 计量。E_t 为区间起点的内部储电量，二元变量 z_t 区分充放电模式，两种模式均允许闲置。

本问售电收入设为 0，购电按计划电量与对应时段单价结算，目标为

$$\min J = \sum_{t=0}^{T-1} p_t q_t, \qquad T = 144. \tag{2}$$

其中 q_t 为区间电量，目标函数单位为元。

3.3.2　供需平衡与储能运行约束

（1）区间供需平衡。外网购电、光伏及电池放电共同满足负载与充电需求，富余部分记为未利用电量：

$$q_t + V_t + d_t = L_t + c_t + w_t, \qquad q_t, c_t, d_t, w_t \geq 0. \tag{3}$$

w_t 为供给超过负载和充电需求的剩余电量，包括未使用的光伏电量与已购电量；购电费按计划计算。

（2）储电量递推。按假设 2，充电 c_t 使内部电量增加 $\eta_c c_t$，向母线放电 d_t 需消耗内部电量 d_t/η_d，故

$$E_{t+1} = E_t + \eta_c c_t - \frac{d_t}{\eta_d}, \qquad \eta_c = \eta_d = 0.9. \tag{4}$$

（3）运行电量与首末状态。设备额定容量为 12000 kWh，可运行范围为其 10%–90%，要求

$$\begin{aligned} E_{\min} \le E_t \le E_{\max}, \quad t = 0, \dots, T, \\ E_{\min} = 1200, \qquad E_{\max} = 10800, \\ E_0 = E_T = E^{\text{init}} = 6000. \end{aligned} \tag{5}$$

首末电量相等保证电池在日末恢复初始储电量。由于采用区间恒功率近似，内部储电量在每个区间内线性变化，因此，各区间起止时刻的储电量满足上下限时，区间内也满足。

（4）充放电功率与模式互斥。交流侧最大充放电功率 $P_{\max} = 5000$ kW，每个区间的电量上限为 $P_{\max}\Delta t = 833.3333$ kWh。令

$$\begin{aligned} 0 \le c_t \le P_{\max}\Delta t\, z_t, \\ 0 \le d_t \le P_{\max}\Delta t\,(1 - z_t), \qquad z_t \in \{0, 1\}. \end{aligned} \tag{6}$$

当 $z_t = 1$ 时仅允许充电，当 $z_t = 0$ 时仅允许放电，从而排除同一区间同时充放电。

3.4 模型求解

先去掉二元变量得到 LP 松弛。若某格同时充放电，可扣除相互抵消的充放电量，保持相同的储电量变化；在电价非负、允许处理富余电量且切换费用为 0 时，变换后的购电费用小于或等于原费用。因此 LP 存在同费用的互斥解。保存的 LP 轨迹经复核已满足互斥，其费用与 MILP 在数值容差内一致。另采用 DP 进行网格转移，将储电量网格从 50 细化至 6.25 kWh 时，相对 LP 下界的费用差距由 0.7906% 降至 0.0771%。网格 DP 提供可行费用上界，LP 提供最优费用下界。

3.5 求解结果

最终，LP 得到全天购电量 59,482.70 kWh、最低费用 35,126.95 元；MILP 得到同值并给出零求解间隙。该费用较无储能方案节省 12,925.10 元（26.90%），未利用电量为 0。低价充电、午间预留光伏存储空间和高峰放电共同降低了费用。

3.5.1 购电与储能安排

表1、表2列出指定时段和全天的购电、储能结果，数值保留两位小数。

表 **1**　指定时段购电量及全天购电结果（电量单位：**kWh**）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量/kWh	59,482.70			全天购电费/元	35,126.95

表 **2**　指定时段充放电量及首末储电量（单位：**kWh**）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4,500.00	0.00	04:00–08:00	833.33	6,365.84
08:00–12:00	4,787.96	1,703.00	12:00–16:00	5,286.04	91.10
16:00–20:00	0.00	5,780.13	20:00–24:00	5,333.33	2,859.87
0:00 储电量	6,000.00		24:00 储电量		6,000.00

全天充电量为 20,740.67 kWh，放电量为 16,799.94 kWh。四小时时段内可以先后发生充电和放电，因此两项汇总电量可能均为正；同一 10 分钟区间内只进行充电或放电。

3.5.2　调度过程的可视化分析

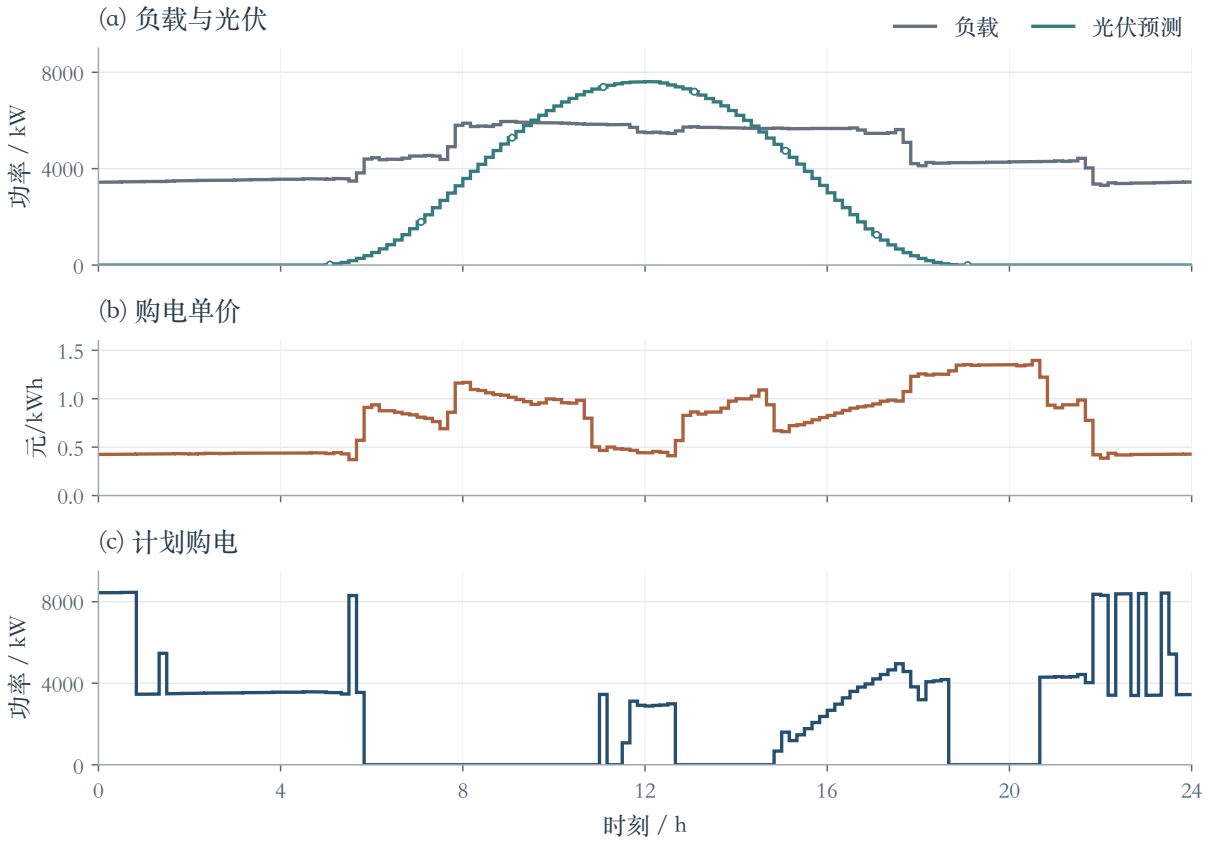


图 **4**　负载、光伏、电价与最优购电计划

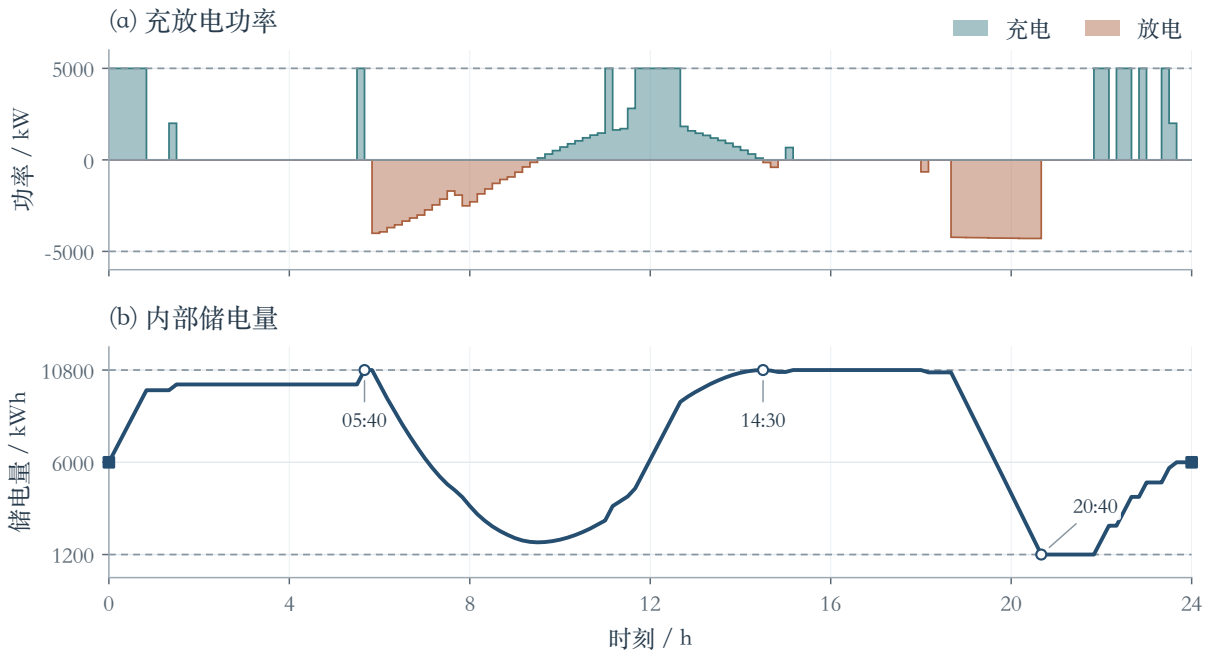


图 **5**　储能运行轨迹（充电取正、放电取负）

图4显示，部分低价时段购电超过负载，用于储能；光伏富余或储能供电充足时，外网购电量为 0。图5展示了充放电功率和储电量随时间的变化，标出了储电量上下限、首末电量及达到上下限的时刻。

低价充电与早间放电。05:30–05:40 为全天最低价区间，单价为 0.3713 元/kWh，电池以最大允许功率充电，05:40 达到 10800 kWh。05:50–09:30 外网购电量为 0，累计放电 8068.84 kWh，储电量降至 1834.62 kWh，为后续多余的光伏电量留出存储空间。

午间购电与光伏储电。09:30–14:30 累计充电 9961.53 kWh，其中同期净光伏富余为 6247.96 kWh、购电为 3713.57 kWh。12:30–12:40 虽然处于午间最低价区间并满功率充电，但 12:40 仍留有 1656.21 kWh 内部空间；此后至 14:30 的 1840.23 kWh 净光伏富余，经 90% 充电效率转换后恰好填补该空间。模型据此为后续光伏留出空间，同时满足充电功率约束。

晚高峰放电与日末恢复。18:00–20:40 累计放电 8640.00 kWh，内部电量由 10800 降至 1200 kWh；21:50 后累计充电 5333.33 kWh，24:00 恢复至 6000 kWh。受功率上限等因素限制，部分高价时段仍需购电，例如 18:00–18:10 仍需购电 531.89 kWh。

3.5.3　不同策略的费用比较

在相同输入和首末储电量下设置两个对照：无储能方案购买负载超过光伏的部分；固定规则在 00:00–06:00 尽量充电，16:00 后放电补足光伏无法满足的负载，并保留 6000 kWh 日末电量。费用见表3。

表 **3**　相同首末状态下不同策略的全天费用比较

策略	购电量/kWh	购电费/元	未利用量/kWh	节费率
无储能	61,789.94	48,052.05	6,247.96	0.00%
固定时段充放电规则	62,803.27	46,429.00	6,247.96	3.38%
LP 日前优化（MILP 同值验证）	59,482.70	35,126.95	0.00	26.90%

MILP 利用价差并吸收富余光伏；固定时段规则忽略后续供需，且充放电损耗增加购电量，因此节费有限。

3.6　敏感性分析

为评估购电策略对效率解释和功率记录方式的敏感性，以原模型为基准，每次改变一项假设，电价、储能容量、功率上限和首末储电量均采用原设置，重新求解 MILP，并比较全天购电费用及其相对变化。

设置两组对照：一是将 90% 解释为往返效率，充、放电单程效率均取 $\sqrt{0.9}$；二是将负载和光伏记录视为周期端点的瞬时功率，按相邻端点线性积分计算区间电量。原模型采用单程 90% 效率和区间平均功率。结果见表4，相对变化按 $(J_s - J_0)/J_0 \times 100\%$ 计算，其中 J_0 为原模型费用，J_s 为对照费用。

表 **4**　不同效率与功率记录假设下的购电费用及相对变化

计算假设	购电费/元	相对主模型变化
主模型：单程 90%，区间平均功率	35,126.95	0.00%
往返 90%，单程取 $\sqrt{0.9}$	33,801.50	-3.77%
周期瞬时端点，线性积分	35,042.54	-0.24%

往返效率设为 90% 后，单程效率提高，储能损耗减小，购电费用下降 3.77%；采用瞬时功率积分计算电量后，费用下降 0.24%。两组对照中，充放电效率对购电费用的影响更大。

四、　问题二：基于边际价值 DP 的日前购电与储能控制

4.1　问题分析

问题二在零点按历史数据制定购电计划，供电不足时按 5 倍电价补购。多购可能浪费，少购可能增加紧急费用；当前放电还会减少后续可用电量。模型需同时确定用电安全余量和各时段储能用量。

本文先预测负载和光伏，用 LP 确定当天 144 个区间的普通购电量，再用边际价值 DP 决定充放电。全天采用零点确定的普通购电量，剩余缺口由紧急购电补足，储电量逐日延续。

已有储能控制研究表明，在价格波动和效率损耗条件下，动态规划策略常具有阈值

结构，并可用有限预测视野刻画库存的边际价值 [5,6]。结合上述阈值控制思想与本题紧急购电规则，构造储能费用函数。

图6依次给出历史数据处理、日前购电规划、边际价值 DP 和结果核验步骤。



图 **6**　问题二流程图

4.2　数据处理

Step 1：**整理数据**。附件 2 含 2025 年 365 天、每日 144 个区间，共 52,560 组观测。按日期和区间匹配，沿用问题一的电量换算并检查完整性及数值。1 月积累历史误差，储电量保持 6000 kWh；2 月 1 日至 12 月 31 日用于评估，共 334 天、48,096 个区间。

Step 2：**确定可用数据**。第 k 日零点仅使用第 $k-1$ 日及此前观测，$t=0,\dots,143$。L_{kt}, V_{kt} 为实际负载、光伏电量，p_t 取附件 1 电价。日内供需仅用于假设 4 中的即时控制。

4.3　模型建立

4.3.1　负载、光伏预测与用电安全余量

（1）预测方法的选择。对负载和光伏分别建立昨日同刻、上周同刻与岭回归（Ridge）三种候选方法。Ridge 采用昨日同刻值、上周同刻值、近 7 日同刻均值、昨日全天均值、日内一二阶正余弦项及周末标记共 9 个特征；在此前最多 60 个完整日上标准化特征，以

$$\min_{a,\boldsymbol{\beta}} \sum_{(j,t)\in\mathcal{T}_k} \left(y_{jt} - a - \boldsymbol{x}_{jt}^{\mathsf{T}}\boldsymbol{\beta}\right)^2 + 10\|\boldsymbol{\beta}\|_2^2 \tag{7}$$

拟合各目标。$\mathcal{T}_k$ 只包含当日之前可用于训练的日期，标准化参数也只从该训练集计算。

三种候选方法均按日向前滚动拟合。在 2025 年历史数据的同条件比较中，固定 Ridge 的负载和光伏 RMSE 均略低于按此前 14 日误差逐日选择预测方法的对照方案。综合预测误差、费用对照与模型简洁性，主方案采用固定 Ridge，并以逐日选取预测方法作对照。

（2）偏差修正。针对近期连续高估或低估，利用最近 3 个已完成日的基础预测误差

均值修正当天预测。对 $y \in \{L, V\}$，令

$$\widehat{y}_{kt}^{\text{cor}} = \max\left\{0, \widehat{y}_{kt}^{\text{base}} + \frac{1}{|\mathcal{B}_k|}\sum_{j\in\mathcal{B}_k}(y_{jt} - \widehat{y}_{jt}^{\text{base}})\right\}, \qquad \widehat{N}_{kt} = \widehat{L}_{kt}^{\text{cor}} - \widehat{V}_{kt}^{\text{cor}}. \tag{8}$$

其中 $\mathcal{B}_k$ 为最近 3 个完整日期。修正后的光伏与负载预测均限制为非负值，净负载允许为负，以保留光伏富余信息。

（3）按小时构造安全余量。定义历史修正净负载误差 $r_{jt} = L_{jt} - V_{jt} - \widehat{N}_{jt}$。对当前区间所属小时 $h(t)$，汇集过去 14 天同一小时 6 个区间的误差，得到经验样本集 $\mathcal{R}_{k,h(t)}$，评估期内每集有 84 个样本。取

$$m_{kt} = \max\{0, Q_{0.8}(\mathcal{R}_{k,h(t)})\}, \tag{9}$$

作为加在预测负载上的安全余量。净负载被低估时，增加这一余量可提前安排更多供电，减少紧急购电，但也可能增加未用完的已购电量。

在单时段无储能模型中，若随机需求为 X，最小化 $pq + 5p\,\mathbb{E}[(X-q)^+]$ 的一阶条件为 $F_X(q) = 0.8$。据此取历史误差的 80% 分位数作为用电安全余量。

4.3.2 日前计划购电模型

记 $\bar{c}_{kt}, \bar{d}_{kt}, \bar{E}_{kt}, \bar{w}_{kt}$ 为计划充电量、放电量、储电量和未利用电量，q_{kt} 为零点确定的购电量。将修正后的预测和安全余量代入，得到

$$\begin{aligned}
\min \quad & \sum_{t=0}^{T-1} p_t q_{kt}, \\
\text{s.t.} \quad & q_{kt} + \widehat{V}_{kt}^{\text{cor}} + \bar{d}_{kt} = \widehat{L}_{kt}^{\text{cor}} + m_{kt} + \bar{c}_{kt} + \bar{w}_{kt}, \\
& \bar{E}_{k,t+1} = \bar{E}_{kt} + \eta_c \bar{c}_{kt} - \bar{d}_{kt}/\eta_d, \\
& E_{\min} \le \bar{E}_{kt} \le E_{\max}, \quad 0 \le \bar{c}_{kt}, \bar{d}_{kt} \le P_{\max}\Delta t, \\
& q_{kt}, \bar{w}_{kt} \ge 0, \qquad \bar{E}_{k0} = E_{k0}, \quad \bar{E}_{kT} = 6000.
\end{aligned} \tag{10}$$

效率、功率与容量参数沿用问题一。求解 LP 后，若同一时段同时充放电，则扣除两者相互抵消的部分，保持相同的储电量变化，并检查运行约束。

普通购电价格 p_t 低于紧急价格 $5p_t$，普通购电量取非负值，可覆盖全部预测需求，因此日前计划采用普通购电。式(10)只确定 q_{kt}；实际充放电随供需调整。

4.3.3 边际价值 DP 模型

普通购电费 $\sum_t p_t q_{kt}$ 已固定，执行时只需权衡当前紧急费用与保留电量的后续价值。$\mathcal{V}_{k,t}(E)$ 表示时段 t 开始、储电量为 E 时，按经验误差分布估计的剩余紧急费用与日末补电费用估计。

（1）充放电量与供电缺口。对于误差样本 $r \in \mathcal{R}_{k,h(t)}$，令

$$x_{kt}(r) = q_{kt} - (\widehat{N}_{kt} + r). \tag{11}$$

当 $x_{kt}(r) \geq 0$ 时，供电有剩余。模型允许弃用余电，额外储电费用和电池老化费用均设为 0，因此在功率与容量允许时尽量充电：

$$c = \min\left\{x_{kt}(r), P_{\max}\Delta t, \frac{E_{\max} - E}{\eta_c}\right\}, \quad d = u = 0, \quad E' = E + \eta_c c. \tag{12}$$

当 $x_{kt}(r) < 0$ 时，缺口为 $g = -x_{kt}(r)$，动作满足

$$0 \leq d \leq \min\{g, P_{\max}\Delta t, \eta_d(E - E_{\min})\}, \qquad u = g - d, \quad c = 0, \quad E' = E - d/\eta_d. \tag{13}$$

上述动作均满足充放电互斥约束，且紧急购电仅用于补足负载缺口。

（2）Bellman 递推。以 $\mathcal{A}(E, r)$ 表示上述可行动作集，有

$$\mathcal{V}_{k,t}(E) = \frac{1}{84}\sum_{r \in \mathcal{R}_{k,h(t)}} \min_{a \in \mathcal{A}(E,r)} \left\{5p_t u(a, r) + \mathcal{V}_{k,t+1}(E'(a, r))\right\}. \tag{14}$$

控制器按当前供需选择动作，后续费用按历史误差样本取平均。安全余量 m_{kt} 通过日前计划 q_{kt} 计入，式(11)采用预测净负载与误差样本。每个小时的 84 个残差样本按相同权重组成历史残差分布，递推时按当前小时的分布独立计算各时段误差，状态变量只保留储电量。近 3 日偏差修正用于处理连续高估或低估。

（3）日末补电费用估计。除评估期最后一天外，以补足日末参考储电量所需的费用估计电量对次日的价值：

$$\mathcal{V}_{k,T}(E) = \kappa_k(6000 - E)^+, \qquad \kappa_k = \begin{cases} \min_t p_t/\eta_c = 0.4125556, & k\text{非 12 月 31 日}, \\ 0, & k\text{为 12 月 31 日}. \end{cases} \tag{15}$$

其中 $(a)^+ = \max(a, 0)$。该费用按附件 1 电价估算，用于确定当日留电量。实际储电量连续传至次日。

4.3.4 放电量的确定

发生缺口 g 时，令下一时刻储电量为 e，则 $d = \eta_d(E - e)$。去掉费用中与动作无关的项，求得建议保留的储电量

$$e^\star_{kt} \in \arg\min_e \{\mathcal{V}_{k,t+1}(e) + 5p_t\eta_d e\}, \tag{16}$$

结合当前储电量、供电缺口和放电功率限制，确定下一时刻储电量：

$$\begin{aligned} e_{\mathrm{lo}} &= \max\{E_{\min}, E - \min(g, P_{\max}\Delta t)/\eta_d\}, \\ E' &= \min\{E, \max(e_{\mathrm{lo}}, e^{\star}_{kt})\}, \qquad d = \eta_d(E - E'), \quad u = g - d. \end{aligned} \tag{17}$$

若多个储电量对应的费用相同，选择较小的下一时刻储电量，即优先多放电；未来价值为零时，该规则与贪心放电一致。

内部每放出 1 kWh，可供电 η_d kWh，节省当前紧急费用 $5p_t\eta_d$ 元；保留该电量的边际价值可表示为 $-\partial_E \mathcal{V}_{k,t+1}(E)$。前者较大时放电，后者较大时留电；后一情形可能在仍有可放电库存时发生紧急购电。

4.4 模型求解

每天确定购电计划后，将 $[1200, 10800]$ 按 50 kWh 划分为 193 个储电量点，计算 145 个时刻对应的预计费用。按 84 个误差样本，从日末向前计算建议保留电量，网格点之间采用线性插值；主要计算量随 $T \times 193 \times 84$ 增长。

实际储电量保持连续，有富余时按式(12)充电，有缺口时按式(17)放电并补购。日末储电量传至次日，评估期费用为

$$J_{\mathrm{year}} = \sum_{k=2\text{ 月 }1\text{ 日}}^{12\text{ 月 }31\text{ 日}} \sum_{t=0}^{143} (p_t q_{kt} + 5p_t u_{kt}). \tag{18}$$

实际费用由普通购电费和紧急购电费组成，按原始精度累计，年末剩余电量的结算价值设为 0。完整结果见 `result2.xlsx`。

4.5 求解结果

4.5.1 全年费用与运行概况

334 天内，普通计划购电量为 22,000,180.73 kWh，计划费为 13,487,877.14 元，紧急购电量为 62,092.01 kWh。

交流侧累计充电 6,449,842.80 kWh、放电 5,224,344.54 kWh。

年末储电量为 6,031.25 kWh。紧急购电覆盖 125 天、743 个十分钟区间，按同日连续区间合并为 267 次事件。

4.5.2 指定日期购电、储能与紧急结果

表5汇总 3 月 20 日、6 月 21 日、9 月 23 日、12 月 21 日六个指定区间及全天购电结果，时间均按题面实际区间标注。

表 **5**　指定日期计划购电量及全天费用（区间电量单位：**kWh**）

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	577.64	0.00	566.93	891.93
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	458.29	184.77	590.06	803.03
18:00–18:10	719.95	0.00	802.73	706.46
20:00–20:10	0.00	0.00	0.00	0.00
全天计划量/kWh	69,398.18	35,846.18	73,127.12	95,682.81
全天总费用/元	42,373.78	20,535.18	45,859.13	61,758.73

表6按 6 个四小时时段汇总充放电量。段内可先后充放电，各十分钟区间仍满足互斥；每日初始储电量沿用前一日末值。

表 **6**　指定日期四小时充放电量及日初末储量（单位：**kWh**）

时间段	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	4,168.47	122.38	826.81	864.24	4,447.45	17.16	3,909.86	229.70
04:00–08:00	833.85	4,466.23	540.97	4,391.89	259.29	5,668.57	1,516.35	1,687.53
08:00–12:00	6,252.33	2,777.38	8,951.05	0.00	6,243.34	1,896.62	5,324.81	7,297.48
12:00–16:00	2,747.15	493.27	0.00	0.00	3,331.19	577.97	7,637.69	2,395.37
16:00–20:00	391.13	5,648.45	172.05	5,646.82	242.68	5,465.73	54.12	5,559.56
20:00–24:00	5,754.18	2,986.10	6,271.88	2,499.72	6,082.83	2,778.09	5,887.87	2,842.53
0:00 储电量		6,684.66		7,353.19		6,816.36		7,120.74
24:00 储电量		6,490.60		7,547.81		7,135.64		6,782.63

表 **7**　指定日期紧急购电结果

指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
紧急购电时间段	无	无	无	07:50–08:00
紧急电量/kWh	0.00	0.00	0.00	14.55

表7显示，12 月 21 日 07:50–08:00 发生 14.55 kWh 紧急购电，其余三个指定日期的紧急购电量在 10^{-7} kWh 容差内记为 0。全年事件按“十分钟区间数、同日连续事件数、发生天数”分别统计，详见工作簿。

4.5.3　储能控制实例

以节费较大的 2025 年 9 月 8 日为例，图7比较相同计划下贪心与 DP 的储电量和紧急购电。

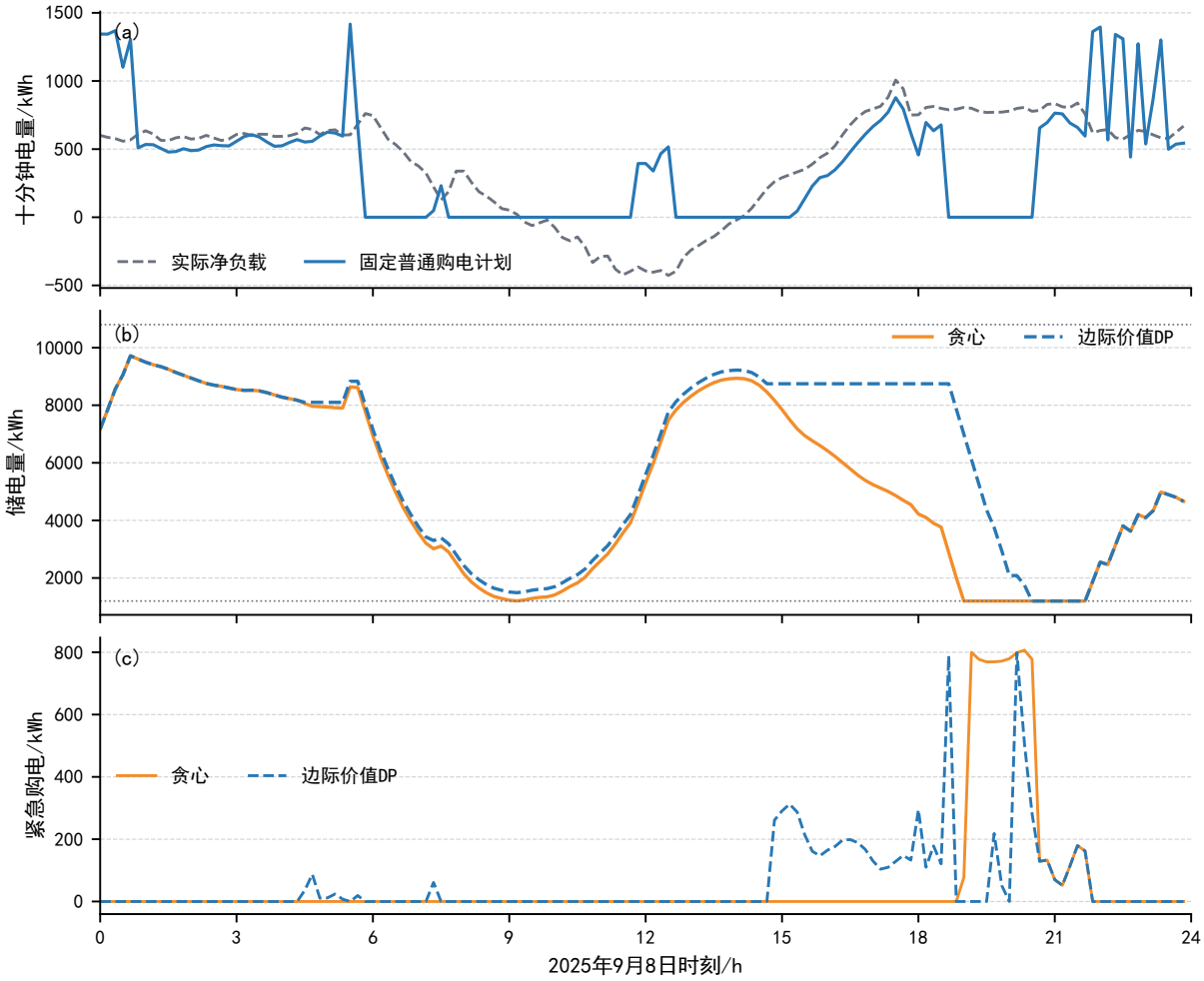


图 7　9 月 8 日相同购电计划下两种储能控制方法的运行轨迹

04:40–04:50 的供电缺口为 90.40 kWh，电价为 0.4422 元/kWh。贪心控制立即放电；DP 建议保留 8150 kWh，高于当时的 8105.12 kWh 储电量，因此支付 199.88 元紧急购电费并将电量留至后续高价时段。该日 DP 节省 10,692.67 元；全年共有 295 个区间在能够放电时选择留电并紧急购电。

4.6　模型检验

下述费用差区间均以两种方法的配对逐日费用差为样本，采用 7 日连续块重采样，重复 10000 次，随机种子取 2026。

4.6.1　预测检验

按各日零点可用信息计算 48,096 个区间的误差。固定 Ridge 的负载、光伏 RMSE 分别为 39.44、56.99 kWh，逐日选取预测方法分别为 39.87、57.06 kWh。将两种方法的年末储电量统一为 6000 kWh 后，逐日选取预测方法比固定 Ridge 多支出 0.5912 万元，95% 重采样区间为 $[-2.1451, 3.0413]$ 万元，区间跨越零点。表8给出同条件比较；固定 Ridge 的紧急区间略多，但补购时点的费用更低。

表 **8**　统一 **6000 kWh** 年末储电量下固定 **Ridge** 与逐日选取预测方法的比较

预测方案	负载 RMSE	光伏 RMSE	总费用/万元	紧急电量/kWh	紧急区间数
逐日选取预测方法	39.87	57.06	1384.3210	60574.70	704
固定 Ridge	39.44	56.99	1383.7298	62092.01	743

4.6.2　相同购电计划下的储能控制比较

固定普通购电计划、实际供需、电价、初始储电量及物理参数，仅将 DP 换为“有缺口就尽量放电”的贪心控制，连续运行 334 天（表9）。两者使用同一份冻结购电计划，计划费和年末储电量相同。

表 **9**　冻结同一普通购电计划后的储能控制方法费用比较

储能控制方法	计划费用/万元	紧急费用/万元	总费用/万元	年末储电量/kWh
贪心反馈	1348.7895	37.6670	1386.4565	6,031.25
边际价值 DP	1348.7895	34.4920	1383.2815	6,031.25

表9中的 DP 沿用冻结购电计划，总费用为 1383.2815 万元；主方案按每日实际初始储电量重新求解购电计划，总费用为 1383.2797 万元，两者相差 18.12 元。贪心与 DP 的紧急电量分别为 61,931.41 kWh 和 62,092.01 kWh。DP 通过转移补购时点，在冻结计划下节省 3.1750 万元；每日重新规划并统一年末储电量后节省 3.1769 万元，95% 重采样区间为 $[1.1640, 6.1129]$ 万元。

4.6.3　数据时序检验

将附件 4 电价整体替换为极端常数后重新运行，购电计划、储能动作和总费用保持不变，两次运行得到的完整结果文件完全一致。因此，本问的预测、规划与执行仅依赖附件 1–2 以及决策时已经发生的历史信息。

4.6.4　网格检验

在同一固定普通购电计划下，将储电网格改为 100、25 kWh，结果见表10。三档网格的最大费用差为 358.65 元，占总费用的 0.0026%。网格用于近似期望价值函数，年度费用由实际运行轨迹计算。50 kWh 在精度与计算量之间较为均衡，故用于主结果。

表 **10**　固定 **Ridge** 与冻结购电计划下的 **DP** 网格检验

网格间距/kWh	状态点数	总费用/万元	相对 50 kWh 费用差/元
100.00	97	1383.3119	+304.22
50.00	193	1383.2815	0.00
25.00	385	1383.2761	−54.43

4.7 结论

评估期主方案总费用为 1383.2797 万元，其中计划购电费 1348.7877 万元、紧急购电费 34.4920 万元。在冻结同一普通购电计划并统一两种控制方法的初始、年末储电量后，DP 较贪心节省 3.1750 万元，降幅为 0.2290%。

五、 问题三：基于滚动预报的购电调整与储能控制

5.1 问题分析

问题三在问题二基础上增加 0:00、6:00、12:00 和 18:00 发布的光伏预报，并允许对待交付的购电计划付费调整。若一次更新直接修改其后的全部计划，则 6:00 预报会对 12:00 以后的电量提前形成交易，而这些时段还会获得更新信息。本文采用“全日规划、六小时更新”的滚动机制：每次规划至当日 24:00，但日内只确认至下一发布时间的购电量，其余结果用于估计当前储能对后续时段的作用。

储能采用递减价值 DP 控制。每个发布时间根据当时可用的预报和历史误差重建剩余日价值函数，并执行随后 6 小时。实际储电量连续传递至下一发布时间和次日。

图8概括光伏预报处理、策略分段更新、储能控制和费用核验流程。

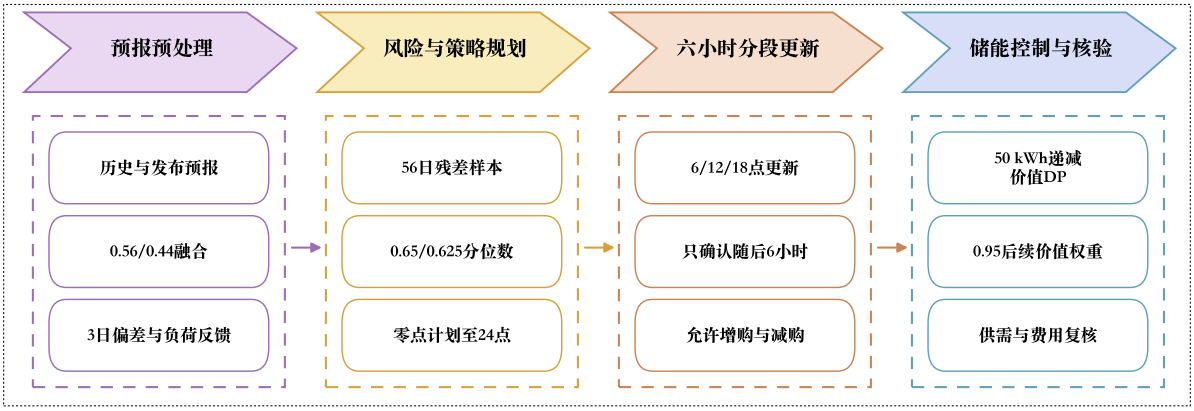


图 **8**　问题三流程图

5.2 预报处理与信息范围

5.2.1 发布时间与交付区间

沿用日期 k 和区间 $t = 0, \ldots, T-1$，其中 $T = 144$、$\Delta t = 1/6\,\mathrm{h}$。令 $r = 0, 1, 2, 3$ 表示四个发布时间，$s_r = 36r$ 为对应区间编号，并定义

$$\mathcal{I}_r = \{s_r, \ldots, \min(s_r + 35, T-1)\}, \qquad \mathcal{H}_r = \{s_r, \ldots, T-1\}. \tag{19}$$

$\mathcal{I}_r$ 是本次确认的交付窗口，$\mathcal{H}_r$ 是规划视野。0:00 形成全天计划；6:00、12:00 和 18:00 分别只调整 6:00–12:00、12:00–18:00 和 18:00–24:00 的计划。因此每个交付区间除零点

基础交易外，至多发生一次日内调整。

5.2.2 光伏预报融合与安全余量

附件 3 的整点功率预报先截为非负，在相邻整点间线性插值，再按梯形积分换算为十分钟电量。发布时刻的插值起点取此前最后一个完整实测区间。记发布预报和历史光伏预测为 $\widehat{V}^{\mathrm{pub}}_{k,r,\ell}$ 与 $\widehat{V}^{\mathrm{hist}}_{k,r,\ell}$，融合结果为

$$\widehat{V}_{k,r,\ell} = \left[0.56\widehat{V}^{\mathrm{pub}}_{k,r,\ell} + 0.44\widehat{V}^{\mathrm{hist}}_{k,r,\ell} + \delta^{\mathrm{PV}}_{k,r,\ell}\right]_+, \tag{20}$$

其中 $\delta^{\mathrm{PV}}_{k,r,\ell}$ 取最近至多 3 个完整日同一发布时间和提前区间的平均预测误差。

负载预测在昨日同刻、上周同刻和 Ridge 之间按最近 14 个完整日的误差滚动选择，并进行近 3 日偏差修正。日内预测再利用最近 2 小时已经发生的负载误差进行反馈，初始权重为 0.5，影响随预测距离衰减。

对每个发布时间和相对小时 $h(\ell) = \lfloor \ell/6 \rfloor$，汇集此前最多 56 个完整日的净负载残差，形成集合 $\mathcal{R}_{k,r,h(\ell)}$。安全余量取

$$m_{k,r,\ell} = \max\left\{0, Q_{\alpha_r}\left(\mathcal{R}_{k,r,h(\ell)}\right)\right\}, \qquad \alpha_0 = 0.65, \quad \alpha_1 = \alpha_2 = \alpha_3 = 0.625. \tag{21}$$

不同发布时间下的预测误差和调整费用不同，两个分位数根据历史数据复算确定，在评估期内保持不变。

参数确定。参数比较以总费用为主要指标，同时记录紧急购电量和未利用电量。将零点及日内分位数均提高至 0.8 而保持其余设置不变时，总费用由 1320.7966 万元增至 1351.9329 万元，紧急购电量由 4.1443 万 kWh 降至 1.2962 万 kWh，未利用电量由 153.2084 万 kWh 增至 211.6474 万 kWh。较高分位数以 31.1363 万元额外费用换取 2.8481 万 kWh 紧急购电量下降，因此主方案采用 0.65 和 0.625。

融合权重、56 日残差窗口、2400/6000 kWh 参考量和 0.95 后续价值权重在同一历史复算中固定，分别控制预报融合、残差样本量、日末留电和后续电量价值。

图9给出 3 月 20 日各发布时间后六小时内送入规划模型的光伏预报。竖虚线表示信息更新点。

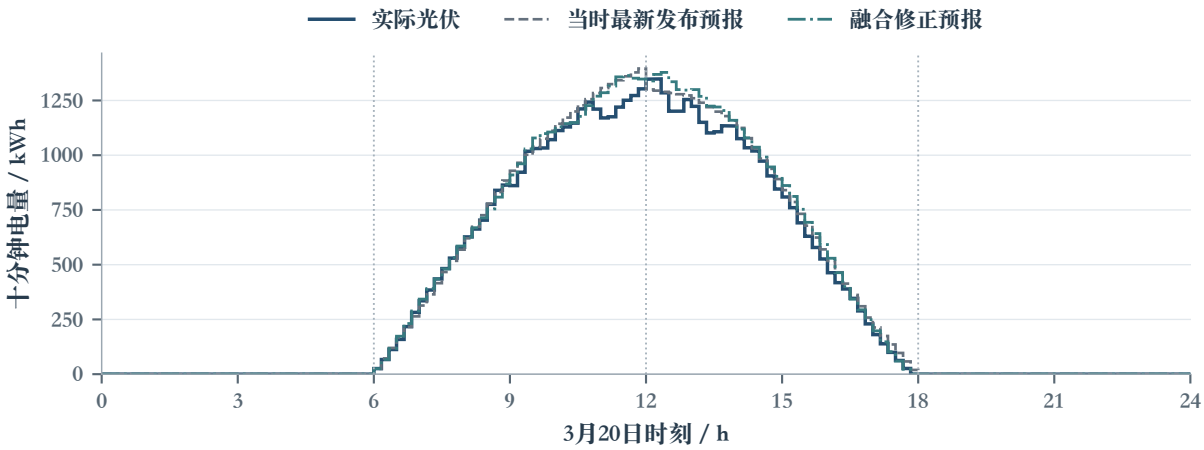


图 **9**　指定日光伏实测、发布预报与融合修正结果

5.3　滚动购电模型

5.3.1　基础计划与调整费用

令 q^{0}_{kt} 为 0:00 确定的基础购电量，q^{f}_{kt} 为最终确认量，并定义

$$a_{kt} = (q^{\mathrm{f}}_{kt} - q^{0}_{kt})_{+}, \qquad b_{kt} = (q^{0}_{kt} - q^{\mathrm{f}}_{kt})_{+}. \tag{22}$$

增购部分按 $1.5p_t$ 计费。减购时退回原购电费，同时对取消部分支付 $0.5p_t$ 违约费。每个区间的实际费用为

$$C_{kt} = p_t q^{0}_{kt} + 1.5p_t a_{kt} - 0.5p_t b_{kt} + 5p_t u_{kt} = p_t q^{\mathrm{f}}_{kt} + 0.5p_t |q^{\mathrm{f}}_{kt} - q^{0}_{kt}| + 5p_t u_{kt}. \tag{23}$$

例如，原计划 100 kWh、最终确认 80 kWh 且 $p_t = 1$ 时，该区间普通购电费用为 90 元。

5.3.2　零点计划与日内滚动优化

0:00 根据融合预报和安全余量求解全天计划，初始储电量取实际值，计划日末储电量固定为 2400 kWh。6:00、12:00 和 18:00 重新计算 $\mathcal{H}_r$ 内的购电和储能安排，但只将 $\mathcal{I}_r$ 内的结果写入策略。

对 $t \in \mathcal{I}_r$，记本次规划相对零点策略的增购量和减购量为 $a^{(r)}_{kt}$、$b^{(r)}_{kt}$，日末储电量低于 6000 kWh 的部分为 ξ_{kr}。日内目标为

$$\begin{aligned} \min \quad & \sum_{t \in \mathcal{I}_r} \left(1.5p_t a^{(r)}_{kt} - 0.5p_t b^{(r)}_{kt} + 5p_t \bar{u}_{kt} \right) \\ & + \sum_{t \in \mathcal{H}_r \setminus \mathcal{I}_r} \left(p_t \widetilde{q}^{(r)}_{kt} + 5p_t \bar{u}_{kt} \right) + \lambda_{kr} \xi_{kr}, \end{aligned} \tag{24}$$

其中 $\lambda_{kr} = 1.5 \min_{t \in \mathcal{H}_r} p_t / \eta_c$。规划轨迹满足供需平衡、储电量递推、容量和功率约束，

并有

$$\bar{E}_{k,s_r} = E_{k,s_r}, \qquad \bar{E}_{k,T} + \xi_{kr} \geq 6000. \tag{25}$$

未来窗口内尚未确认的购电量只参与本次规划，下一发布时间重新计算。

5.3.3 递减价值 DP 控制

设 ε_t 为相同发布时间和小时内的历史净负载残差。给定已确认策略，剩余日储能费用函数满足

$$\mathcal{V}_{k,t}^{(r)}(E) = \mathbb{E}_{\varepsilon_t}\left[\min_{E'}\left\{5p_t u_t(E, E', \varepsilon_t) + 0.95\mathcal{V}_{k,t+1}^{(r)}(E')\right\}\right]. \tag{26}$$

每个发布时间和相对小时内的历史残差按相同权重组成历史残差分布，不同时段的残差在递推中独立取样，DP 状态只保留储电量。每次获得新预报后重新计算残差分布和价值函数，使控制结果随最新信息更新。日末补电费用估计为

$$\mathcal{V}_{k,T}^{(r)}(E) = \lambda_{kr}(6000 - E)_+. \tag{27}$$

储电量按 50 kWh 划分为 193 个网格点，实际储电量保持连续，网格间采用线性插值。四次 DP 分别覆盖 144、108、72 和 36 个区间，各执行前 36 个区间后重建。

5.4 求解结果

5.4.1 全年费用与储能运行

评估期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。零点计划量、最终策略量和紧急购电量分别为 2088.0167 万、2068.1000 万和 4.1443 万 kWh。原策略费为 1,274.6717 万元，增购费 39.4426 万元，减购违约费 16.1894 万元，退回原策略费用 32.3788 万元，紧急购电费 22.8717 万元，总费用为 13,207,966.05 元，即 1320.7966 万元。

累计充电 6,601,919.54 kWh、放电 5,350,750.77 kWh，均按交流侧计量；年末实际储电量为 2,448.95 kWh。紧急购电发生在 171 天的 696 个十分钟区间，按同日连续区间合并为 316 次事件。

5.4.2 指定日期结果

表11给出四个指定日期的零点计划与最终确认量，表12汇总实际充放电和日初末储电量。

表 **11**　指定日期原计划与最终普通购电量（电量单位：**kWh**）

时间段	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	原计划	最终量	原计划	最终量	原计划	最终量	原计划	最终量
10:00–10:10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
12:00–12:10	507.99	507.99	0.00	0.00	432.96	402.61	965.58	900.26
14:00–14:10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
16:00–16:10	565.06	471.42	194.08	151.23	542.37	542.37	810.25	825.26
18:00–18:10	710.96	710.96	0.00	0.00	788.93	777.54	692.42	685.03
20:00–20:10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
全天电量	68,629.91	66,728.81	34,088.88	33,892.35	70,210.20	68,480.54	95,973.04	95,514.31
普通购电费/元	41,911.29	42,276.67	19,406.33	19,384.56	43,668.60	43,244.97	61,414.04	62,051.99

表 **12**　指定日期各四小时时段的实际充放电量与首末储电量（**kWh**）

时间段	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	7,300.40	76.23	374.97	402.69	7,925.24	20.92	7,653.53	264.36
04:00–08:00	1,692.23	5,668.18	1,183.25	1,759.53	1,163.69	6,545.23	2,811.36	1,726.88
08:00–12:00	4,920.46	2,777.38	10,187.95	0.00	5,091.10	1,896.62	5,181.38	7,718.68
12:00–16:00	5,083.74	482.49	0.00	0.00	5,701.49	541.87	7,963.47	2,018.59
16:00–20:00	285.30	5,257.98	109.30	5,912.38	230.95	5,682.33	23.91	5,841.92
20:00–24:00	1,754.17	2,901.87	2,111.57	2,621.68	1,966.72	2,997.11	1,765.67	2,832.89
0:00 储电量		2,919.07		2,630.91		2,768.35		2,570.75
24:00 储电量		2,780.49		3,316.49		2,990.66		2,759.78

表 **13**　指定日期的紧急购电事件

日期	紧急购电时间段	电量/kWh
3 月 20 日	无	0.00
6 月 21 日	无	0.00
9 月 23 日	20:20–20:30	4.88
12 月 21 日	07:50–08:00	31.56
12 月 21 日	20:30–20:50	45.60

表13显示，3 月 20 日和 6 月 21 日没有紧急购电；9 月 23 日 20:20–20:30 购电 4.88 kWh，12 月 21 日两次购电合计 77.16 kWh。图10显示 12 月 21 日普通购电、储电量和紧急购电的连续运行过程。

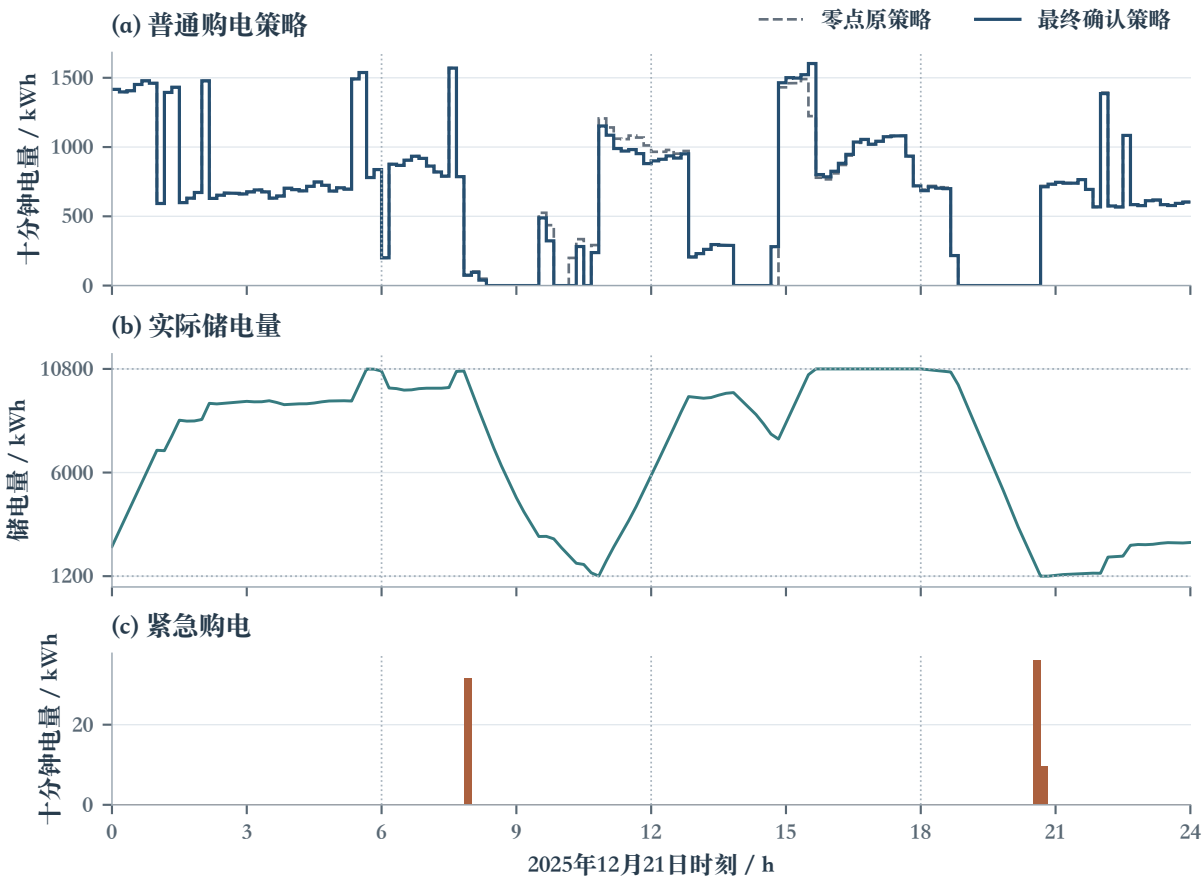


图 **10**　**12** 月 **21** 日的普通购电量、实际储电量与紧急购电

5.5　模型检验与日内预报价值

独立复算覆盖 48,096 个区间，结果见表14。交易记录中的调整均位于对应六小时确认窗口，同一交付区间没有重复调整；决策只使用当时已经完成的历史数据。

表 **14**　问题三逐区间独立核验

核验对象	最大绝对误差	单位
母线供需平衡	1.14×10^{-12}	kWh
累计储电量递推	4.55×10^{-11}	kWh
跨区间状态连续	0	kWh
储电量越界	0	kWh
充放功率对应电量越界	1.14×10^{-13}	kWh
总费用差	0	元

为判断日内预报的作用，在其他参数、初始储电量、策略窗口和 DP 设置相同的条件下，将 6:00、12:00 和 18:00 发布的光伏预报固定为零点版本，结果见表15。

表 **15**　日内光伏预报更新的费用影响

光伏预报使用方式	总费用/万元	紧急电量/kWh	年末储电量/kWh
始终使用零点版本	1327.5193	48,425.57	2,448.95
按 0/6/12/18 点更新	1320.7966	41,443.14	2,448.95

两种方案年末储电量相同。使用日内更新后，总费用减少 67,227.15 元，紧急购电量由 48,425.57 kWh 降至 41,443.14 kWh，因此保留 6:00、12:00 和 18:00 三个更新时刻。

5.6 结论

四个发布时间均按预先确定的协议使用，每次规划至日末并只确认随后六小时。2025 年评估期总费用为 1320.7966 万元，日内预报更新在相同条件下节省 0.5064%。

六、 问题四：实时波动电价下的购电与储能调控

6.1 问题分析

问题四将固定分时电价改为实时波动电价，并要求分别重新计算问题二和问题三。此时购电量与购电时刻共同决定实际费用，储能既用于缓冲负载和光伏预测误差，也用于降低高价时段的紧急购电费用。

本文以决策时可获得的历史电价预测当日价格，按预测电价制定购电计划和储能动作，交付时按附件 4 中的实际电价结算。问题四-2 保留问题二的零点全天计划和边际价值 DP；问题四-3 保留问题三的融合预报、双向策略调整、六小时分段更新和递减价值 DP。

图11概括两类任务从电价预测、购电规划和储能控制到费用核验的流程。

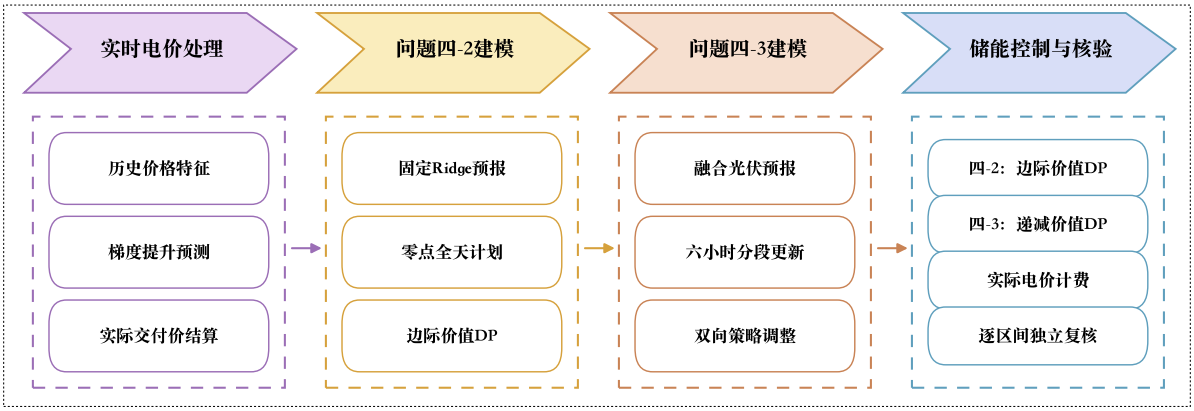


图 **11** 问题四流程图

6.2 实时电价预测

6.2.1 预测变量与信息范围

记 p_{kt} 为第 k 日第 t 个区间的实际电价，$\hat{p}_{kt}$ 为当日 0:00 可获得的预测电价。预测模型仅使用第 k 日以前的电价记录，包括昨日同刻、近三次同星期同刻、近期均值以及时刻、星期和年内周期特征。采用直方图梯度提升回归，每 7 日利用最近最多 60 日的历史样本重新拟合，模型参数在 2 月至 12 月的计算过程中保持预先设定。

设决策日以前的信息集合为 $\mathcal{F}_{k,0}$，则

$$\widehat{p}_{kt} = f_{\theta_k}(\mathcal{F}_{k,0}, t), \qquad \theta_k \text{ 仅由第 } k \text{ 日以前的数据估计.} \tag{28}$$

当日规划使用预测电价，交付时按实际电价 p_{kt} 结算。采用相同的历史信息范围，昨日同刻、同星期同刻和近期均值三种基准方法的 MAE 分别为 0.0843、0.0466 和 0.0534 元/kWh，RMSE 分别为 0.1238、0.0663 和 0.0710 元/kWh。直方图梯度提升模型在 2025 年 2 月至 12 月 48,096 个区间上的 MAE 为 0.0399 元/kWh，RMSE 为 0.0545 元/kWh，两项误差均为四种方法中的最小值，因此用于问题四的电价预测。

6.2.2 预测价决策与实际价结算

问题四-2 在 0:00 形成全天普通购电量 q^0_{kt}，剩余缺口由紧急购电 u_{kt} 补足，实际费用为

$$C_{4-2} = \sum_{k} \sum_{t=0}^{T-1} \left(p_{kt} q^0_{kt} + 5 p_{kt} u_{kt} \right). \tag{29}$$

问题四-3 沿用式(19)的发布时刻。令 $a_{k,t}$、$b_{k,t}$ 分别为最终策略相对零点计划的增购量和减购量。0:00 基础购电费只支付一次，减购时退回原购电费并支付 50% 违约费，实际费用为

$$C_{4-3} = \sum_{k} \sum_{t=0}^{T-1} \left(p_{kt} q^0_{kt} + 1.5 p_{kt} a_{kt} - 0.5 p_{kt} b_{kt} + 5 p_{kt} u_{kt} \right). \tag{30}$$

6.3 购电与储能模型

6.3.1 问题四-2 的全天计划

问题四-2 保持问题二的固定 Ridge 负载、光伏预测和 0.8 分位数安全余量。将问题二目标函数中的固定电价替换为 $\widehat{p}_{kt}$，在电池功率、容量、效率和供需平衡约束下求解全天计划。全天采用零点确定的普通购电计划，实际运行阶段使用边际价值 DP 决定是否放电；紧急购电全部用于补足当前负载缺口。

6.3.2 问题四-3 的分段更新

问题四-3 沿用问题三的预测与控制参数。0:00 将附件 3 预报与历史预测按 0.56 和 0.44 融合，形成全天基础购电计划；6:00、12:00 和 18:00 利用最新预报重新规划至当天 24:00。第 r 次重规划只确认 $\mathcal{I}_r$ 内的购电量，$\mathcal{H}_r \setminus \mathcal{I}_r$ 内的结果用于估计储能对后续时段的作用。

零点计划日末储电量设为 2400 kWh。日内规划以 6000 kWh 为软参考量，允许相对零点策略增购或减购，并按预测电价计算规划目标。储能控制采用问题三的 50 kWh 网格递减价值 DP，后续价值权重为 0.95；在四个发布时间分别重建价值函数，每次执行随后 36 个区间。实际日末储电量直接作为次日初值。

6.4 模型求解

Step 1：**电价预测**。每日 0:00 由此前已完成日期的数据生成 144 个区间的预测电价，作为当日规划的价格输入。

Step 2：**购电规划**。问题四-2 仅在 0:00 求解全天计划；问题四-3 在四个发布时间重新求解剩余时段，并只确认到下一发布时间的购电调整。

Step 3：**储能控制**。问题四-2 使用边际价值 DP；问题四-3 在每个发布时间依据最新光伏预报重建递减价值 DP。所有动作均使用连续储电量，电池状态跨日传递。

Step 4：**费用复算**。采用实际交付电价逐区间计算普通购电费、策略调整费用净额和紧急购电费，并独立复核供需平衡、储电量递推、容量、功率及充放电互斥约束。

6.5 求解结果

6.5.1 两类任务的年度结果

评估期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。表16给出两类任务在实际年末储电量条件下的结果。

表 **16**　实时波动电价下的年度费用与运行结果

任务与控制方法	基础购电费/万元	策略调整费用净额/万元	紧急购电费/万元	总费用/万元	紧急电量/kWh	年末储电量/kWh
问题四-2，边际价值 DP	1421.2538	0	36.3918	1457.6456	60,665.46	6,018.42
问题四-3，递减价值 DP	1345.1474	24.1991	25.3294	1394.6759	44,687.76	2,448.95

问题四-2 的总费用为 1457.6456 万元，其中紧急购电费 36.3918 万元。紧急购电发生在 119 天的 708 个十分钟区间，将同日连续区间合并后得到 253 次事件。

问题四-3 的总费用为 1394.6759 万元，其中基础购电费 1345.1474 万元、策略调整费用净额 24.1991 万元、紧急购电费 25.3294 万元。紧急购电发生在 164 天的 759 个十分钟区间，合并后得到 298 次事件。

图12汇总问题二至问题四的年度费用分解和紧急购电量。

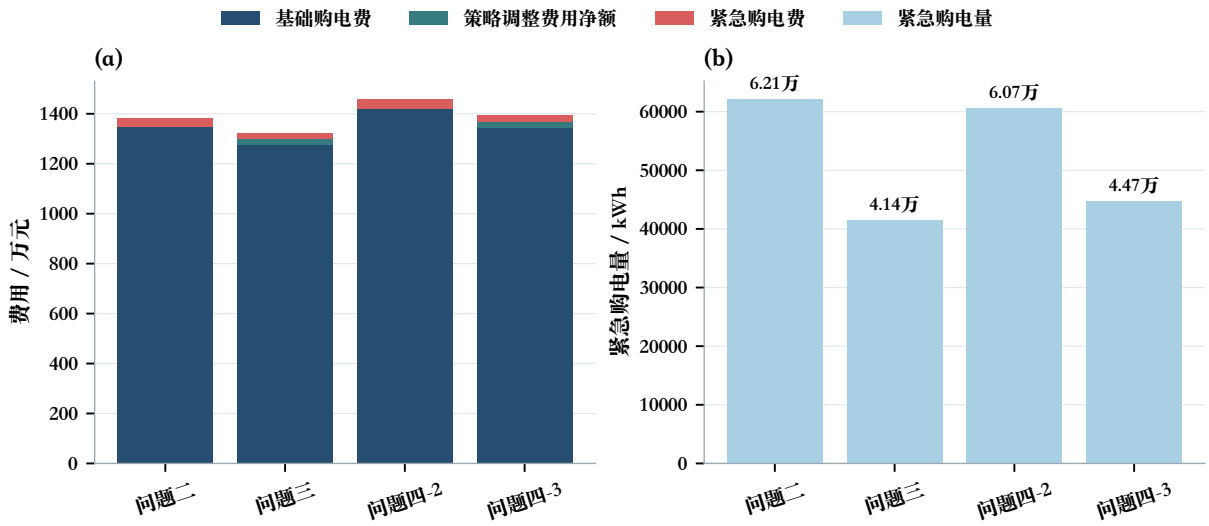


图 **12**　问题二至问题四主方案的年度费用构成与紧急购电量

图12表明，问题三和问题四-3 通过日内预报调整降低紧急购电量。波动电价增大了补购时点的费用差异，问题四-2 使用边际价值 DP 保留储能至较高预测电价时段；问题四-3 进一步利用融合预报和双向策略调整降低基础购电费用。

6.5.2　储能控制与网格精度对照

问题四-2 在统一 6000 kWh 年末储电量下比较贪心控制与边际价值 DP。问题四-3 保持模型不变，将储电量网格由 50 kWh 细化为 25 kWh，结果见表17。

表 **17**　储能控制方法与网格精度对照

对照项目	对照费用/万元	采用方案费用/万元	费用差/万元
问题四-2：贪心/边际价值 DP	1460.3750	1458.0700	2.3049
问题四-3：25/50 kWh 网格	1394.6809	1394.6759	0.0050

问题四-2 中，边际价值 DP 较贪心控制节省 2.3049 万元。以配对逐日费用差为样本，采用 7 日连续块重采样 10000 次，随机种子取 2026，所得 95% 区间为 $[0.7065, 4.5125]$ 万元。问题四-3 细化网格后费用仅变化 49.50 元，年末储电量和紧急购电量保持不变，50 kWh 网格能够满足本题精度要求。

6.5.3　两类任务的费用比较

问题四-3 比问题四-2 少支出 62.9697 万元，紧急购电量减少 15,977.70 kWh。两类任务同时改变光伏预报来源、购电调整方式和储能控制参数，该差额反映题目要求的两套完整策略的综合变化。

6.6 模型检验

对两类主结果的 48,096 个区间进行独立复算。供需平衡残差上界为 2.3×10^{-12} kWh，累计储电量递推误差低于 4.8×10^{-11} kWh，跨日连续残差为 0。所有区间均满足 1200–10800 kWh 容量边界和 833.33 kWh 单区间充放电上限，充电与放电互斥，紧急购电全部用于补足负载缺口。逐项费用复算误差低于 1.5×10^{-11} 元。

6.7 结论

实时波动电价下，问题四-2 采用零点全天计划和边际价值 DP，总费用为 1457.6456 万元；问题四-3 采用融合光伏预报、六小时分段更新、双向策略调整和递减价值 DP，总费用为 1394.6759 万元。两类策略均按预测电价决策，并按实际交付电价结算。

七、 模型的评价

7.1 模型优点

本文以十分钟电量为统一变量，将储能状态递推、功率限制、容量边界和各类购电费用置于同一计算框架。第一问以 LP、MILP 和 DP 交叉核验费用与运行约束；第二至四问按信息实际发布时间生成决策，并以逐区间轨迹复算供需平衡和费用，每项运行约束和费用均可由保存结果核对。

针对不同信息结构，模型采用分层控制。全天计划负责确定普通购电量，储能控制方法处理预测偏差；问题三、问题四-3 进一步采用“全日规划、六小时更新”的滚动机制，远期预报用于后续储能规划，每次调整费对应本次正式确认的六小时窗口。问题四-2 通过统一年末储电量和连续块重采样比较贪心控制与边际价值 DP，问题三、问题四-3 采用滚动重建的递减价值 DP。

7.2 模型局限与改进方向

当前预测只使用历史数据。加入天气、节假日和生产计划等外生变量可改善负载、光伏和实时电价预测；数据分布发生变化时，需要重新校准安全余量与储能策略。问题三采用减购退款并支付 50% 违约费的结算解释，交易规则不退回原购电费时，应更换费用函数并重新计算。多日滚动视野和概率预测可用于评估极端天气及价格尖峰下的运行风险。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型方案讨论与数学表述、程序实现与调试、数值结果检查，以及论文文字和图表整理，详细使用情况见支撑材料。

参考文献

[1] 国家能源局. 关于支持电力领域新型经营主体创新发展的指导意见: 国能发法改〔2024〕93 号 [EB/OL]. (2024-11-28)[2026-09-10]. https://zfxxgk.nea.gov.cn/2024-11/28/c_1212408354.htm.

[2] 国家发展改革委, 国家能源局. 关于推进电力源网荷储一体化和多能互补发展的指导意见: 发改能源规〔2021〕280 号 [EB/OL]. (2021-02-25)[2026-09-10]. https://www.ndrc.gov.cn/xxgk/zcfb/ghxwj/202103/t20210305_1269046_ext.html.

[3] P. Sharma, H. Dutt Mathur, P. Mishra, R. C. Bansal. A critical and comparative review of energy management strategies for microgrids[J/OL]. Applied Energy, 2022, 327: 120028. (2022)[2026-09-11]. https://doi.org/10.1016/j.apenergy.2022.120028.

[4] M. S. Hossain Lipu, S. Ansari, M. S. Miah, et al. A review of controllers and optimizations based scheduling operation for battery energy storage system towards decarbonization in microgrid: Challenges and future directions[J/OL]. Journal of Cleaner Production, 2022, 360: 132188. (2022)[2026-09-11]. https://doi.org/10.1016/j.jclepro.2022.132188.

[5] P. M. van de Ven, N. Hegde, L. Massoulié, T. Salonidis. Optimal control of end-user energy storage[J/OL]. IEEE Transactions on Smart Grid, 2013, 4(2): 789–797. (2013)[2026-09-12]. https://doi.org/10.1109/TSG.2012.2232943.

[6] J. R. Cruise, R. J. Gibbens, S. Zachary. Optimal control of storage for arbitrage, with applications to energy systems[C/OL]. 2014 48th Annual Conference on Information Sciences and Systems, 2014: 1–6. (2014)[2026-09-12]. https://doi.org/10.1109/CISS.2014.6814090.

[7] W. Kong, Z. Y. Dong, D. J. Hill, F. Luo, Y. Xu. Short-term residential load forecasting based on LSTM recurrent neural network[J/OL]. IEEE Transactions on Smart Grid, 2019, 10(1): 841–851. (2019)[2026-09-12]. https://doi.org/10.1109/TSG.2017.2753802.

[8] 《新型电力系统发展蓝皮书》编写组. 新型电力系统发展蓝皮书 [R/OL]. 2023: 7[2026-09-10]. https://www.nea.gov.cn/download/xxdlxtfzlpsgk.pdf.

附录 A　文件列表

文件名	功能描述
result1.xlsx	问题一：购电策略与储能运行结果
result2.xlsx	问题二：购电、充放电与紧急购电结果
result3.xlsx	问题三：购电调整与储能运行结果
result4-2.xlsx	波动电价下问题二的运行结果
result4-3.xlsx	波动电价下问题三的运行结果
actuals_power.csv	全年负载、光伏与电价记录
actuals_energy.csv	十分钟电量及数据可用时间
pv_forecasts.csv	分时发布的光伏预报
q1_inputs.csv	问题一输入与固定分时电价
schedule.csv	问题一逐时段购电与储能轨迹
annual_slots.csv	问题二至四全年逐时段运行轨迹
annual_daily.csv	问题二至四逐日费用与电量汇总
annual_inputs.npz	求解所需全年输入数组
q3_current.json	问题三最终策略参数
summaries.json	四问费用、策略参数与结果摘要
analysis_results.json	敏感性、预报价值与计算核验结果
problem1.py	问题一：线性规划求解与检验
problem2.py	问题二：日前预测与购电策略
problem3.py	问题三：预报融合与分段更新
problem4.py	问题四：电价预测与购电策略
microgrid.py	公共模型、储能控制与结果导出
requirements.txt	Python 依赖及版本
README.md	运行方法与完整文件清单

附录 B　代码

代码 B.1　问题一：线性规划求解与检验（problem1.py）

```
from __future__ import annotations
import argparse
import csv
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
from time import perf_counter
import numpy as np
from numba import njit
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import coo_matrix, csr_matrix, hstack, vstack

@dataclass(frozen=True)
class Battery:
    lower_kwh: float = 1200.0
    upper_kwh: float = 10800.0
    power_kw: float = 5000.0
    eta_charge: float = 0.9
    eta_discharge: float = 0.9
    dt_hours: float = 1.0 / 6.0

    def __post_init__(self):
        if not 0 <= self.lower_kwh <= self.upper_kwh:
            raise ValueError('Invalid battery energy bounds')
        if self.power_kw < 0 or self.dt_hours <= 0:
            raise ValueError('Invalid power or time step')
        if not (0 < self.eta_charge <= 1 and 0 < self.eta_discharge <= 1):
            raise ValueError('Invalid efficiency')

def _problem(price, load_kw, pv_kw, battery, initial, terminal):
    (p, load, pv) = (np.asarray(x, dtype=float) for x in (price, load_kw, pv_kw))
    if p.ndim != 1 or not p.shape == load.shape == pv.shape or (not p.size):
        raise ValueError('price/load/PV must be equal-length nonempty vectors')
    if not all((np.isfinite(x).all() for x in (p, load, pv))):
        raise ValueError('Nonfinite input')
    if min(p.min(), load.min(), pv.min()) < 0:
        raise ValueError('Q1 assumes nonnegative prices, load and PV')
    if terminal not in {'fixed', 'cyclic_free', 'none'}:
        raise ValueError('Unknown terminal condition')
    if terminal != 'cyclic_free' and (not battery.lower_kwh <= initial <= battery.upper_kwh):
        raise ValueError('Initial energy is outside battery bounds')
    n = len(p)
    m = 5 * n + 1
    (rows, cols, vals, rhs) = ([], [], [], [])
    net = (load - pv) * battery.dt_hours
    for t in range(n):
        rows.extend([t] * 4)
        cols.extend([t, n + t, 2 * n + t, 3 * n + t])
        vals.extend([1.0, -1.0, 1.0, -1.0])
        rhs.append(net[t])
    for t in range(n):
        rows.extend([n + t] * 4)
        cols.extend([4 * n + t + 1, 4 * n + t, n + t, 2 * n + t])
        vals.extend([1.0, -1.0, -battery.eta_charge, 1 / battery.eta_discharge])
        rhs.append(0.0)
    if terminal in {'fixed', 'cyclic_free'}:
        rows.extend([2 * n, 2 * n])
        cols.extend([4 * n + n, 4 * n])
        vals.extend([1.0, -1.0])
```

```
        rhs.append(0.0)
    aeq = coo_matrix((vals, (rows, cols)), shape=(len(rhs), m)).tocsr()
    low = np.zeros(m)
    high = np.full(m, np.inf)
    high[n:3 * n] = battery.power_kw * battery.dt_hours
    low[4 * n:] = battery.lower_kwh
    high[4 * n:] = battery.upper_kwh
    if terminal != 'cyclic_free':
        low[4 * n] = high[4 * n] = initial
    objective = np.zeros(m)
    objective[:n] = p
    return (p, net, aeq, np.asarray(rhs), low, high, objective)

def audit(schedule, price, load_kw, pv_kw, battery=Battery(), initial=6000.0, terminal='fixed'):
    (g, c, d, w, e) = (np.asarray(schedule[k]) for k in ('grid_kwh', 'charge_kwh', 'discharge_kwh', 'disposed_kwh', 'energy_kwh'))
    balance = g + np.asarray(pv_kw) * battery.dt_hours + d - np.asarray(load_kw) * battery.dt_hours - c - w
    state = np.diff(e) - battery.eta_charge * c + d / battery.eta_discharge
    bounds = max(0.0, battery.lower_kwh - e.min(), e.max() - battery.upper_kwh, max(c.max(), d.max()) - battery.power_kw * battery.dt_hours, -min(g.min(), c.min(), d.min(), w.min()))
    out = {'max_balance_residual_kwh': float(np.max(np.abs(balance))), 'max_state_residual_kwh': float(np.max(np.abs(state))), 'max_bound_violation_kwh': float(bounds), 'max_simultaneous_energy_kwh': float(np.minimum(c, d).max()), 'initial_residual_kwh': 0.0 if terminal == 'cyclic_free' else float(abs(e[0] - initial)), 'terminal_residual_kwh': 0.0 if terminal == 'none' else float(abs(e[-1] - e[0])), 'recomputed_cost_yuan': float(np.dot(price, g))}
    out['passed'] = bool(max((v for (k, v) in out.items() if k.endswith('kwh'))) <= 1e-05)
    return out

def _extract(x, n, battery, repair=True):
    (g, c, d, w, e) = (x[a:b].copy() for (a, b) in [(0, n), (n, 2 * n), (2 * n, 3 * n), (3 * n, 4 * n), (4 * n, 5 * n + 1)])
    if repair:
        rho = battery.eta_charge * battery.eta_discharge
        delta = np.minimum(c, d / rho)
        c -= delta
        d -= rho * delta
        w += (1 - rho) * delta
    for vector in (g, c, d, w):
        vector[np.abs(vector) < 1e-09] = 0.0
    return dict(grid_kwh=g, charge_kwh=c, discharge_kwh=d, disposed_kwh=w, energy_kwh=e)

def solve_lp(price, load_kw, pv_kw, battery=Battery(), initial=6000.0, terminal='fixed', lexicographic=True):
    start = perf_counter()
    (p, net, aeq, rhs, low, high, objective) = _problem(price, load_kw, pv_kw, battery, initial, terminal)
    options = {'primal_feasibility_tolerance': 1e-08, 'dual_feasibility_tolerance': 1e-08}
    result = linprog(objective, A_eq=aeq, b_eq=rhs, bounds=np.column_stack([low, high]), method='highs', options=options)
    if not result.success:
        raise RuntimeError(f'LP failed: {result.message}')
    finite_upper = np.isfinite(high)
    dual_objective = float(rhs @ result.eqlin.marginals + low @ result.lower.marginals + high[finite_upper] @ result.upper.marginals[finite_upper])
    dual_residual = float(np.max(np.abs(objective - aeq.T @ result.eqlin.marginals - result.lower.marginals - result.upper.marginals)))
    if abs(result.fun - dual_objective) > 0.01 or dual_residual > 1e-07:
        raise AssertionError('LP primal-dual certificate failed')
    n = len(p)
    x = result.x
    if lexicographic:
        secondary = np.zeros_like(objective)
        secondary[n:3 * n] = 1.0
        tie = linprog(secondary, A_ub=csr_matrix(objective[None, :]), b_ub=[result.fun + 1e-07],
```

```
    A_eq=aeq, b_eq=rhs, bounds=np.column_stack([low, high]), method='highs', options=options)
        if not tie.success:
            raise RuntimeError(f'Lexicographic LP failed: {tie.message}')
        x = tie.x
    schedule = _extract(x, n, battery)
    certificate = audit(schedule, p, load_kw, pv_kw, battery, initial, terminal)
    if not certificate['passed']:
        raise AssertionError(f'LP physical audit failed: {certificate}')
    return {**schedule, 'cost_yuan': float(p @ schedule['grid_kwh']), 'lower_bound_yuan':
    dual_objective, 'audit': certificate, 'primal_dual_gap_yuan': float(result.fun - dual_objective
    ), 'max_dual_stationarity_residual': dual_residual, 'solve_seconds': perf_counter() - start, '
    solver_status': result.message, 'battery': asdict(battery), 'terminal': terminal}

def solve_milp(price, load_kw, pv_kw, battery=Battery(), initial=6000.0, terminal='fixed'):
    start = perf_counter()
    (p, net, aeq, rhs, low, high, objective) = _problem(price, load_kw, pv_kw, battery, initial,
    terminal)
    (n, m) = (len(p), len(objective))
    eq = hstack([aeq, csr_matrix((aeq.shape[0], n))]).tocsr()
    (rows, cols, vals) = ([], [], [])
    q = battery.power_kw * battery.dt_hours
    for t in range(n):
        rows.extend([t, t, n + t, n + t])
        cols.extend([n + t, m + t, 2 * n + t, m + t])
        vals.extend([1, -q, 1, q])
    aub = coo_matrix((vals, (rows, cols)), shape=(2 * n, m + n)).tocsr()
    result = milp(np.r_[objective, np.zeros(n)], integrality=np.r_[np.zeros(m), np.ones(n)],
    bounds=Bounds(np.r_[low, np.zeros(n)], np.r_[high, np.ones(n)]), constraints=[LinearConstraint(
    eq, rhs, rhs), LinearConstraint(aub, np.full(2 * n, -np.inf), np.r_[np.zeros(n), np.full(n, q)
    ])], options={'mip_rel_gap': 1e-10, 'time_limit': 180.0})
    if result.x is None:
        raise RuntimeError(f'MILP failed: {result.message}')
    schedule = _extract(result.x[:m], n, battery, repair=False)
    certificate = audit(schedule, p, load_kw, pv_kw, battery, initial, terminal)
    if not certificate['passed']:
        raise AssertionError(f'MILP audit failed: {certificate}')
    return {**schedule, 'cost_yuan': float(p @ schedule['grid_kwh']), 'audit': certificate, '
    lower_bound_yuan': float(result.mip_dual_bound), 'mip_gap': float(result.mip_gap), '
    solve_seconds': perf_counter() - start, 'solver_status': result.message}

@njit(cache=True)
def _grid_dp_kernel(price, net, grid, eta_c, eta_d, qmax, initial_index):
    (n, m) = (price.size, grid.size)
    values = np.full(m, np.inf)
    values[initial_index] = 0.0
    choices = np.full((n, m), -1, np.int32)
    step = grid[1] - grid[0]
    rmin = int(np.ceil(-qmax / eta_d / step - 1e-10))
    rmax = int(np.floor(qmax * eta_c / step + 1e-10))
    for t in range(n - 1, -1, -1):
        fresh = np.full(m, np.inf)
        for i in range(m):
            (jmin, jmax) = (max(0, i + rmin), min(m - 1, i + rmax))
            for j in range(jmin, jmax + 1):
                change = grid[j] - grid[i]
                if change >= 0:
                    grid_purchase = max(0.0, net[t] + change / eta_c)
                else:
                    grid_purchase = max(0.0, net[t] + change * eta_d)
                candidate = price[t] * grid_purchase + values[j]
                if candidate < fresh[i]:
                    fresh[i] = candidate
                    choices[t, i] = j
        values = fresh
    trajectory = np.empty(n + 1)
    trajectory[0] = grid[initial_index]
```

```
    idx = initial_index
    for t in range(n):
        idx = choices[t, idx]
        if idx < 0:
            raise ValueError('DP has no feasible path')
        trajectory[t + 1] = grid[idx]
    return (values[initial_index], trajectory)

def solve_grid_dp(price, load_kw, pv_kw, step_kwh, battery=Battery(), initial=6000.0):
    (p, net, *_) = _problem(price, load_kw, pv_kw, battery, initial, 'fixed')
    count = (battery.upper_kwh - battery.lower_kwh) / step_kwh
    index = (initial - battery.lower_kwh) / step_kwh
    if abs(count - round(count)) > 1e-07 or abs(index - round(index)) > 1e-07:
        raise ValueError('DP grid must contain both bounds and exact initial/terminal state')
    grid = battery.lower_kwh + np.arange(round(count) + 1) * step_kwh
    start = perf_counter()
    (value, e) = _grid_dp_kernel(p, net, grid, battery.eta_charge, battery.eta_discharge, battery.
    power_kw * battery.dt_hours, round(index))
    change = np.diff(e)
    c = np.maximum(change, 0) / battery.eta_charge
    d = np.maximum(-change, 0) * battery.eta_discharge
    g = np.maximum(net + c - d, 0)
    w = g + d - c - net
    schedule = dict(grid_kwh=g, charge_kwh=c, discharge_kwh=d, disposed_kwh=w, energy_kwh=e)
    certificate = audit(schedule, p, load_kw, pv_kw, battery, initial)
    if not certificate['passed'] or abs(value - p @ g) > 0.01:
        raise AssertionError(f'DP audit failed: {certificate}')
    return {**schedule, 'cost_yuan': float(p @ g), 'audit': certificate, 'step_kwh': step_kwh, '
    state_count': len(grid), 'solve_seconds': perf_counter() - start}

def optimal_purchase_ranges(price, load_kw, pv_kw, indices, lower_bound, battery=Battery()):
    (p, net, aeq, rhs, low, high, obj) = _problem(price, load_kw, pv_kw, battery, 6000.0, 'fixed')
    rows = []
    for t in indices:
        extrema = []
        for sign in (1, -1):
            target = np.zeros_like(obj)
            target[t] = sign
            res = linprog(target, A_ub=csr_matrix(obj[None, :]), b_ub=[lower_bound + 1e-07], A_eq=
    aeq, b_eq=rhs, bounds=np.column_stack([low, high]), method='highs')
            if not res.success:
                raise RuntimeError(f'Optimal range LP failed: {res.message}')
            schedule = _extract(res.x, len(p), battery)
            check = audit(schedule, p, load_kw, pv_kw, battery)
            if not check['passed']:
                raise AssertionError(check)
            extrema.append(float(schedule['grid_kwh'][t]))
        rows.append(dict(slot_index=t, interval=_label(t), minimum_grid_kwh=extrema[0],
    maximum_grid_kwh=extrema[1], cost_tolerance_yuan=1e-07))
    return rows

def _label(t):
    return f'{t // 6:02d}:{t % 6 * 10:02d}-{(t + 1) // 6:02d}:{(t + 1) % 6 * 10:02d}'

def _write_csv(path, rows):
    with open(path, 'w', encoding='utf-8-sig', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)

def run(input_path, out_dir):
    (input_path, out_dir) = (Path(input_path), Path(out_dir))
    data = np.load(input_path, allow_pickle=False)
    (p, load, pv) = (data[k] for k in ('q1_price', 'q1_load', 'q1_pv'))
    if len(p) != 144:
        raise ValueError('Official Q1 must have 144 ten-minute slots')
```

```
    out_dir.mkdir(parents=True, exist_ok=True)
    battery = Battery()
    lp = solve_lp(p, load, pv)
    integer = solve_milp(p, load, pv)
    delta = abs(lp['cost_yuan'] - integer['cost_yuan'])
    if delta > 0.01:
        raise AssertionError(f'LP/MILP objective mismatch: {delta}')
    dp_rows = []
    previous = np.inf
    for step in (100.0, 50.0, 25.0, 12.5, 6.25, 3.125, 1.5625):
        dp = solve_grid_dp(p, load, pv, step)
        gap = (dp['cost_yuan'] - lp['lower_bound_yuan']) / max(1.0, abs(lp['lower_bound_yuan']))
        if gap < -1e-08 or dp['cost_yuan'] > previous + 0.01:
            raise AssertionError('DP violates a lower bound or nested-grid monotonicity')
        previous = dp['cost_yuan']
        dp_rows.append(dict(step_kwh=step, state_count=dp['state_count'], cost_yuan=dp['cost_yuan'
    ], lower_bound_yuan=lp['lower_bound_yuan'], relative_gap=gap, solve_seconds=dp['solve_seconds'
    ], audit_passed=dp['audit']['passed']))
        print(f"Q1 DP step={step:g} kWh, cost={dp['cost_yuan']:.8f}, gap={100 * gap:.6f}%", flush=
    True)
        if step <= 6.25 and gap <= 0.001:
            break
    if dp_rows[-1]['relative_gap'] > 0.001:
        raise AssertionError('DP gap target not reached; extend grid refinement before publishing'
    )
    _write_csv(out_dir / 'dp_convergence.csv', dp_rows)
    schedule = []
    for t in range(144):
        schedule.append(dict(slot_index=t, interval=_label(t), price_yuan_per_kwh=float(p[t]),
    load_kw=float(load[t]), pv_kw=float(pv[t]), grid_kwh=float(lp['grid_kwh'][t]), charge_kwh=float
    (lp['charge_kwh'][t]), discharge_kwh=float(lp['discharge_kwh'][t]), disposed_kwh=float(lp['
    disposed_kwh'][t]), energy_start_kwh=float(lp['energy_kwh'][t]), energy_end_kwh=float(lp['
    energy_kwh'][t + 1]), purchase_cost_yuan=float(p[t] * lp['grid_kwh'][t])))
    _write_csv(out_dir / 'schedule.csv', schedule)
    _write_csv(out_dir / 'dp_schedule.csv', [dict(slot_index=t, interval=_label(t), grid_kwh=float
    (dp['grid_kwh'][t]), charge_kwh=float(dp['charge_kwh'][t]), discharge_kwh=float(dp['
    discharge_kwh'][t]), energy_start_kwh=float(dp['energy_kwh'][t]), energy_end_kwh=float(dp['
    energy_kwh'][t + 1])) for t in range(144)])
    _write_csv(out_dir / 'optimal_ranges.csv', optimal_purchase_ranges(p, load, pv, [60, 72, 84,
    96, 108, 120], lp['lower_bound_yuan']))
    capacity = []
    for usable in (4800.0, 7200.0, 9600.0, 12000.0, 14400.0):
        for power in (2500.0, 3750.0, 5000.0, 6250.0, 7500.0):
            b = Battery(lower_kwh=1200.0, upper_kwh=1200.0 + usable, power_kw=power)
            solution = solve_lp(p, load, pv, battery=b, lexicographic=False)
            capacity.append(dict(usable_capacity_kwh=usable, fixed_reserve_kwh=1200.0,
    upper_energy_kwh=1200.0 + usable, power_kw=power, cost_yuan=solution['cost_yuan'], grid_kwh=
    float(solution['grid_kwh'].sum()), audit_passed=solution['audit']['passed']))
    for usable in sorted({row['usable_capacity_kwh'] for row in capacity}):
        costs = [row['cost_yuan'] for row in capacity if row['usable_capacity_kwh'] == usable]
        if np.any(np.diff(costs) > 0.01):
            raise AssertionError('Power capacity monotonicity failed')
    for power in sorted({row['power_kw'] for row in capacity}):
        costs = [row['cost_yuan'] for row in capacity if row['power_kw'] == power]
        if np.any(np.diff(costs) > 0.01):
            raise AssertionError('Energy capacity monotonicity failed')
    _write_csv(out_dir / 'capacity_power.csv', capacity)
    sensitivity = []
    cases = [('main_each_way_90pct', battery, 6000.0, 'fixed'), ('roundtrip_90pct', Battery(
    eta_charge=np.sqrt(0.9), eta_discharge=np.sqrt(0.9)), 6000.0, 'fixed'), ('free_cyclic_initial',
     battery, 6000.0, 'cyclic_free'), ('NO_TERMINAL_INVALID_COMPARISON', battery, 6000.0, 'none')]
    cases += [(f'fixed_cyclic_initial_{initial:g}', battery, initial, 'fixed') for initial in
    (1200.0, 3000.0, 9000.0, 10800.0)]
    for (name, b, initial, terminal) in cases:
        sol = solve_lp(p, load, pv, b, initial, terminal)
        sensitivity.append(dict(case=name, cost_yuan=sol['cost_yuan'], grid_kwh=float(sol['
```

```
    grid_kwh'].sum()), initial_energy_kwh=float(sol['energy_kwh'][0]), terminal_energy_kwh=float(
    sol['energy_kwh'][-1]), eta_charge=b.eta_charge, eta_discharge=b.eta_discharge,
    cost_difference_from_main_yuan=sol['cost_yuan'] - lp['cost_yuan'], admissible_main_comparison=
    name != 'NO_TERMINAL_INVALID_COMPARISON'))
    _write_csv(out_dir / 'assumption_sensitivity.csv', sensitivity)
    no_battery_grid = np.maximum((load - pv) * battery.dt_hours, 0)
    grouped = []
    for a in range(0, 144, 24):
        grouped.append(dict(interval=f'{a // 6:02d}:00-{(a + 24) // 6:02d}:00', charge_kwh=float(
    lp['charge_kwh'][a:a + 24].sum()), discharge_kwh=float(lp['discharge_kwh'][a:a + 24].sum()),
    energy_start_kwh=float(lp['energy_kwh'][a]), energy_end_kwh=float(lp['energy_kwh'][a + 24])))
    _write_csv(out_dir / 'battery_4h.csv', grouped)
    summary = {'schema_version': '1.0', 'generated_by': 'problem1', 'question_id': 'Q1', '
    generated_at': datetime.now(timezone.utc).isoformat(), 'status': 'validated', 'input_sha256':
    hashlib.sha256(input_path.read_bytes()).hexdigest(), 'battery': asdict(battery), '
    initial_energy_kwh': 6000.0, 'terminal_energy_kwh': float(lp['energy_kwh'][-1]), 'cost_yuan':
    lp['cost_yuan'], 'lp_lower_bound_yuan': lp['lower_bound_yuan'], 'lp_primal_dual_gap_yuan': lp['
    primal_dual_gap_yuan'], 'lp_max_dual_stationarity_residual': lp['max_dual_stationarity_residual
    '], 'milp_cost_yuan': integer['cost_yuan'], 'milp_lower_bound_yuan': integer['lower_bound_yuan'
    ], 'milp_gap': integer['mip_gap'], 'lp_milp_cost_difference_yuan': delta, 'lp_seconds': lp['
    solve_seconds'], 'milp_seconds': integer['solve_seconds'], 'dp_final_relative_gap': dp_rows
    [-1]['relative_gap'], 'dp_final_step_kwh': dp_rows[-1]['step_kwh'], 'grid_kwh': float(lp['
    grid_kwh'].sum()), 'charge_kwh': float(lp['charge_kwh'].sum()), 'discharge_kwh': float(lp['
    discharge_kwh'].sum()), 'disposed_kwh': float(lp['disposed_kwh'].sum()), 'battery_loss_kwh':
    float((1 - battery.eta_charge) * lp['charge_kwh'].sum() + (1 / battery.eta_discharge - 1) * lp[
    'discharge_kwh'].sum()), 'no_battery_cost_yuan': float(p @ no_battery_grid), '
    no_battery_grid_kwh': float(no_battery_grid.sum()), 'total_load_kwh': float(load.sum() *
    battery.dt_hours), 'total_pv_kwh': float(pv.sum() * battery.dt_hours), 'no_battery_disposed_kwh
    ': float(np.maximum((pv - load) * battery.dt_hours, 0).sum()), 'state_lower_bound_slot_count':
    int(np.sum(np.abs(lp['energy_kwh'] - battery.lower_kwh) < 1e-05)), '
    state_upper_bound_slot_count': int(np.sum(np.abs(lp['energy_kwh'] - battery.upper_kwh) < 1e-05)
    ), 'charge_power_bound_slot_count': int(np.sum(np.abs(lp['charge_kwh'] - battery.power_kw *
    battery.dt_hours) < 1e-05)), 'discharge_power_bound_slot_count': int(np.sum(np.abs(lp['
    discharge_kwh'] - battery.power_kw * battery.dt_hours) < 1e-05)), 'savings_yuan': float(p @
    no_battery_grid - lp['cost_yuan']), 'savings_fraction': float(1 - lp['cost_yuan'] / (p @
    no_battery_grid)), 'lp_audit': lp['audit'], 'milp_audit': integer['audit'], 'dp_audit': dp['
    audit'], 'arbitrage_threshold': 'eta_charge*eta_discharge*p_high > p_low; necessary local
    criterion, not a complete policy', 'capacity_sensitivity_definition': 'Vary usable energy by
    increasing upper bound with lower reserve fixed at 1200 kWh; feasible sets are nested. This is
    not proportional SOC bounds under different nominal capacities.', 'evidence_boundary': '
    Internal mathematical and numerical validation, not independent real-world deployment
    validation.', 'canonical_tie_break': 'Minimize AC charge+discharge throughput subject to
    primary cost <= optimum+1e-7 yuan; equal-cost trajectories may remain nonunique.'}
    (out_dir / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
    encoding='utf-8')
    print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)
    return summary
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--input', default='data/processed/annual_inputs.npz')
    parser.add_argument('--out', default='results/c2026/q1')
    args = parser.parse_args()
    run(args.input, args.out)
```

代码 B.2　问题二：日前预测与购电策略（problem2.py）

```python
from __future__ import annotations
import argparse
import copy
import hashlib
import json
from pathlib import Path
from datetime import datetime, timezone
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
from microgrid import ROOT, ValueBattery, daily_plan, greedy_step, close_plan, execute_close
from microgrid import build_midnight_value_table, execute_midnight_value_policy, load_verified_input
from microgrid import export_annual

METHODS=('previous','weekly','ridge')
TARGETS=('load','pv')
KEYS=['date','slot']

def configuration():
    return dict(methods=METHODS,ridge_alpha=10.,history_days=60,bias_days=3,residual_days=14,risk_quantile=.8)

def normalize_actual_records(data):
    if isinstance(data, pd.DataFrame):
        frame = data.copy()
    elif 'records' in data:
        frame = pd.DataFrame(data['records']).copy()
    else:
        dates = np.asarray(data['dates']).astype(str)
        (load, pv) = (np.asarray(data['actual_load']), np.asarray(data['actual_pv']))
        if load.shape != (len(dates), 144) or pv.shape != load.shape:
            raise ValueError('actual array shape must be (number of dates, 144)')
        frame = pd.DataFrame({'date': np.repeat(dates, 144), 'slot': np.tile(np.arange(144), len(dates)), 'load_kw': load.ravel(), 'pv_kw': pv.ravel()})
    required = KEYS + ['load_kw', 'pv_kw']
    if set(required) - set(frame):
        raise ValueError(f'missing columns: {set(required) - set(frame)}')
    frame = frame[required].copy()
    frame['date'] = pd.to_datetime(frame.date, errors='raise').dt.strftime('%Y-%m-%d')
    slots = pd.to_numeric(frame.slot, errors='raise').to_numpy(dtype=float)
    if not np.isfinite(slots).all() or np.any(slots != np.floor(slots)) or np.any((slots < 0) | (slots > 143)):
        raise ValueError('slot must be an integer in 0..143')
    frame['slot'] = slots.astype(int)
    if frame.duplicated(KEYS).any():
        raise ValueError('duplicate actual date+slot key')
    values = frame[['load_kw', 'pv_kw']].to_numpy(dtype=float)
    if not np.isfinite(values).all() or np.any(values < 0):
        raise ValueError('actual load/PV must be finite and nonnegative')
    frame[['load_kw', 'pv_kw']] = values
    frame = frame.sort_values(KEYS).reset_index(drop=True)
    dates = frame.date.unique()
    if not len(dates):
        raise ValueError('empty history')
    expected_dates = pd.date_range(dates[0], dates[-1], freq='D').strftime('%Y-%m-%d').to_numpy()
    if not np.array_equal(dates, expected_dates):
        raise ValueError('missing calendar day')
    expected_keys = pd.MultiIndex.from_product([dates, np.arange(144)], names=KEYS)
    if not pd.MultiIndex.from_frame(frame[KEYS]).equals(expected_keys):
        raise ValueError('missing actual date+slot key')
    return frame

```

```
def align_residual_records(actual, forecast):
    if actual.duplicated(KEYS).any() or forecast.duplicated(KEYS).any():
        raise ValueError('duplicate actual or forecast date+slot key')
    joined = actual.merge(forecast, on=KEYS, how='outer', validate='one_to_one', indicator=True)
    if not (joined['_merge'] == 'both').all():
        raise ValueError('missing or extra actual/forecast date+slot key')
    joined = joined.drop(columns='_merge').sort_values(KEYS).reset_index(drop=True)
    for name in TARGETS:
        joined[f'actual_{name}_kwh'] = joined[f'{name}_kw'] / 6.0
        joined[f'base_error_{name}_kwh'] = joined[f'actual_{name}_kwh'] - joined[f'base_{name}_kwh
    ']
    joined['corrected_net_error_kwh'] = joined.actual_load_kwh - joined.actual_pv_kwh - joined.
    corrected_load_kwh + joined.corrected_pv_kwh
    numeric = joined.select_dtypes(include=[np.number]).to_numpy()
    if not np.isfinite(numeric).all():
        raise ValueError('nonfinite residual inputs')
    return joined

def features_for_day(history_kwh, date):
    history = np.asarray(history_kwh, dtype=float)
    if history.ndim != 3 or history.shape[1:] != (144, 2):
        raise ValueError('history must have shape (days,144,2)')
    if len(history) == 0:
        return np.zeros((144, 2, 9))
    previous = history[-1]
    weekly = history[-7] if len(history) >= 7 else history[0]
    recent = history[-7:].mean(axis=0)
    phase = 2 * np.pi * np.arange(144) / 144
    shared = np.column_stack([np.sin(phase), np.cos(phase), np.sin(2 * phase), np.cos(2 * phase),
    np.full(144, pd.Timestamp(date).dayofweek >= 5)])
    return np.stack([np.column_stack([previous[:, k], weekly[:, k], recent[:, k], np.full(144,
    previous[:, k].mean()), shared]) for k in range(2)], axis=1)

def build_predictions(data):
    records = normalize_actual_records(data)
    dates = records.date.unique()
    nd = len(dates)
    actual = records[['load_kw', 'pv_kw']].to_numpy().reshape(nd, 144, 2) / 6.0
    candidates = np.zeros((nd, 3, 144, 2))
    base = np.zeros((nd, 144, 2))
    corrected = np.zeros_like(base)
    bias = np.zeros_like(base)
    margin = np.zeros((nd, 144))
    scores = np.zeros((nd, 3, 2))
    selected = np.zeros((nd, 2), dtype=int)
    candidate_mse = np.zeros((nd, 3, 2))
    base_errors = np.zeros_like(base)
    net_errors = np.zeros((nd, 144))
    (daily_info, completed, features) = ([], [], [])
    with threadpool_limits(limits=1):
        for (d, date) in enumerate(dates):
            feature_today = features_for_day(actual[:d], date)
            features.append(feature_today)
            if d:
                candidates[d, 0] = actual[d - 1]
                candidates[d, 1] = actual[d - 7] if d >= 7 else actual[0]
                candidates[d, 2] = actual[max(0, d - 7):d].mean(axis=0)
            training_dates = []
            if d >= 9:
                training_dates = list(range(max(7, d - 60), d))
                for target in range(2):
                    xtrain = np.concatenate([features[j][:, target] for j in training_dates])
                    ytrain = actual[training_dates, :, target].ravel()
                    scaler = StandardScaler().fit(xtrain)
                    model = Ridge(alpha=10.0, fit_intercept=True, solver='cholesky')
                    model.fit(scaler.transform(xtrain), ytrain)
```

```
                    candidates[d, 2, :, target] = np.maximum(model.predict(scaler.transform(feature_today[:, target])), 0.0)
            if d:
                scores[d] = candidate_mse[max(0, d - 14):d].mean(axis=0)
                selected[d] = scores[d].argmin(axis=0)
                bias[d] = base_errors[max(0, d - 3):d].mean(axis=0)
                for hour in range(24):
                    samples = net_errors[max(0, d - 14):d, hour * 6:hour * 6 + 6].ravel()
                    margin[d, hour * 6:hour * 6 + 6] = max(0.0, float(np.quantile(samples, 0.8, method='linear')))
            for target in range(2):
                base[d, :, target] = candidates[d, selected[d, target], :, target]
            corrected[d] = np.maximum(base[d] + bias[d], 0.0)
            predicted = pd.DataFrame({'date': date, 'slot': np.arange(144), 'forecast_available_at': date + ' 00:00:00', 'residual_available_at': (pd.Timestamp(date) + pd.Timedelta(days=1)).isoformat(), 'base_load_kwh': base[d, :, 0], 'base_pv_kwh': base[d, :, 1], 'corrected_load_kwh': corrected[d, :, 0], 'corrected_pv_kwh': corrected[d, :, 1], 'bias_load_kwh': bias[d, :, 0], 'bias_pv_kwh': bias[d, :, 1], 'margin_kwh': margin[d], 'residual_samples_per_hour': min(d, 14) * 6})
            for (m, method) in enumerate(METHODS):
                for (target, name) in enumerate(TARGETS):
                    predicted[f'{method}_{name}_kwh'] = candidates[d, m, :, target]
            actual_day = records.loc[records.date == date]
            aligned = align_residual_records(actual_day, predicted)
            for (target, name) in enumerate(TARGETS):
                base_errors[d, :, target] = aligned[f'base_error_{name}_kwh']
                for (m, method) in enumerate(METHODS):
                    candidate_mse[d, m, target] = np.mean((aligned[f'{method}_{name}_kwh'] - aligned[f'actual_{name}_kwh']) ** 2)
            net_errors[d] = aligned.corrected_net_error_kwh
            completed.append(aligned)
            daily_info.append({'date': date, 'decision_at': date + ' 00:00:00', 'latest_observed_date': None if d == 0 else str(dates[d - 1]), 'load_method': METHODS[selected[d, 0]], 'pv_method': METHODS[selected[d, 1]], 'mse_completed_days': min(d, 14), 'bias_completed_days': min(d, 3), 'risk_completed_days': min(d, 14), 'risk_samples_per_hour': min(d, 14) * 6, 'ridge_training_dates': [str(dates[j]) for j in training_dates], 'candidate_mse_kwh2': {method: {name: float(scores[d, m, k]) for (k, name) in enumerate(TARGETS)} for (m, method) in enumerate(METHODS)}})
    digest = hashlib.sha256(records.to_csv(index=False, float_format='%.17g').encode()).hexdigest()
    return {'dates': dates, 'base_kwh': base, 'corrected_kwh': corrected, 'bias_kwh': bias, 'margin_kwh': margin, 'candidate_kwh': candidates, 'selection_mse_kwh2': scores, 'selected_method_indices': selected, 'base_errors_kwh': base_errors, 'corrected_net_errors_kwh': net_errors, 'records': pd.concat(completed, ignore_index=True), 'daily_info': daily_info, 'actual_records_sha256': digest, 'config': configuration()}

def fixed_candidate_packet(packet: dict, actual_kwh: np.ndarray, method: str='ridge', risk_quantile: float=0.8) -> dict:
    if method not in METHODS:
        raise ValueError(f'unknown method {method!r}')
    actual = np.asarray(actual_kwh, dtype=float)
    candidates = np.asarray(packet['candidate_kwh'], dtype=float)
    if actual.shape != candidates.shape[:1] + candidates.shape[2:]:
        raise ValueError('actual and candidate shapes do not agree')
    method_index = METHODS.index(method)
    base = candidates[:, method_index].copy()
    corrected = np.zeros_like(base)
    bias = np.zeros_like(base)
    base_errors = np.zeros_like(base)
    net_errors = np.zeros(base.shape[:2], dtype=float)
    margin = np.zeros(base.shape[:2], dtype=float)
    for day in range(len(base)):
        if day:
            bias[day] = base_errors[max(0, day - 3):day].mean(axis=0)
            for hour in range(24):
                samples = net_errors[max(0, day - 14):day, hour * 6:hour * 6 + 6].reshape(-1)
```

```
                margin[day, hour * 6:hour * 6 + 6] = max(0.0, float(np.quantile(samples,
    risk_quantile, method='linear')))
        corrected[day] = np.maximum(base[day] + bias[day], 0.0)
        base_errors[day] = actual[day] - base[day]
        net_errors[day] = actual[day, :, 0] - actual[day, :, 1] - corrected[day, :, 0] + corrected
    [day, :, 1]
    result = copy.copy(packet)
    result.update({'base_kwh': base, 'corrected_kwh': corrected, 'bias_kwh': bias, 'margin_kwh':
    margin, 'base_errors_kwh': base_errors, 'corrected_net_errors_kwh': net_errors, '
    forecast_family': f'fixed_{method}'})
    return result

def build_prediction_packet(data: dict, forecast_family: str) -> dict:
    selected = build_predictions(data)
    if forecast_family == 'rolling_selector':
        selected['forecast_family'] = forecast_family
        return selected
    if forecast_family == 'fixed_ridge':
        actual = np.stack([np.asarray(data['actual_load'], dtype=float) / 6.0, np.asarray(data['
    actual_pv'], dtype=float) / 6.0], axis=-1)
        return fixed_candidate_packet(selected, actual, 'ridge', 0.8)
    raise ValueError('forecast_family must be rolling_selector or fixed_ridge')

def moving_block_interval(delta: np.ndarray, *, block_days: int=7, repetitions: int=10000, seed:
    int=2026) -> dict:
    values = np.asarray(delta, dtype=float)
    rng = np.random.default_rng(seed)
    n = len(values)
    starts = np.arange(n - block_days + 1)
    draws = np.empty(repetitions)
    blocks_needed = int(np.ceil(n / block_days))
    for i in range(repetitions):
        chosen = rng.choice(starts, size=blocks_needed, replace=True)
        sample = np.concatenate([values[s:s + block_days] for s in chosen])[:n]
        draws[i] = sample.sum()
    return {'observed_yuan': float(values.sum()), 'ci95_yuan': [float(x) for x in np.quantile(
    draws, [0.025, 0.975])], 'probability_positive': float(np.mean(draws > 0)), 'block_days':
    block_days, 'repetitions': repetitions, 'seed': seed}

def metrics(packet: dict, actual: np.ndarray, mask: np.ndarray) -> dict:
    err = np.asarray(packet['corrected_kwh'])[mask] - actual[mask]
    return {target: {'rmse_kwh': float(np.sqrt(np.mean(err[:, :, index] ** 2))), 'mae_kwh': float(
    np.mean(np.abs(err[:, :, index])))} for (index, target) in enumerate(('load', 'pv'))}

def _q2_kernels(packet: dict, day: int, slots) -> list[np.ndarray]:
    result = []
    for slot in slots:
        hour = int(slot) // 6
        sample = packet['corrected_net_errors_kwh'][max(0, day - 14):day, hour * 6:hour * 6 + 6].
    reshape(-1)
        result.append(sample.copy() if len(sample) else np.zeros(1))
    return result

def _terminal_kappa(data: dict, day: int, decision_price: np.ndarray, battery: ValueBattery, *,
    variable_price: bool) -> float:
    if day == len(data['dates']) - 1:
        return 0.0
    if not variable_price:
        return float(np.min(decision_price) / battery.eta_c)
    if day:
        historical_min = np.min(data['actual_price'][max(0, day - 14):day], axis=1)
        return float(np.median(historical_min) / battery.eta_c)
    return float(np.min(decision_price) / battery.eta_c)

def simulate(data, packet, controller='value_dp', common_closure=False,
             price_forecast=None, grid_step_kwh=50., risk_quantile=.8, fixed_plans=None):
```

```
    dates=np.asarray(data['dates']).astype(str)
    battery=ValueBattery()
    state=battery.target
    rows=[]
    for day, date_value in enumerate(dates):
        date=str(date_value)
        january=pd.Timestamp(date).month==1
        settle=(np.asarray(data['actual_price'][day],dtype=float) if price_forecast is not None
                else np.asarray(data['q1_price'],dtype=float))
        price=(np.asarray(price_forecast[day],dtype=float) if price_forecast is not None else settle.copy())
        load_point=packet['corrected_kwh'][day,:,0]
        pv_point=packet['corrected_kwh'][day,:,1]
        kernels=_q2_kernels(packet,day,range(144))
        margin=np.asarray([max(0.,float(np.quantile(x,risk_quantile,method='linear'))) for x in kernels])
        if fixed_plans is not None:
            plan=np.asarray(fixed_plans[date],dtype=float).copy()
        elif january:
            plan=np.maximum(load_point+margin-pv_point,0.)
        else:
            plan=daily_plan(np.column_stack([load_point,pv_point]),margin,price,state,battery)['q']
        if common_closure and day==len(dates)-1 and fixed_plans is None:
            plan=close_plan(plan,data,day,battery=battery,close_slots=8)
        policy=None
        if controller=='value_dp' and not january:
            kappa=_terminal_kappa(data,day,price,battery,variable_price=price_forecast is not None)
            policy,_=build_midnight_value_table(plan,load_point-pv_point,kernels,price,
                battery=battery,step_kwh=grid_step_kwh,terminal_kappa=kappa,start_slot=0)
        for slot in range(144):
            load=float(data['actual_load'][day,slot]/6.)
            pv=float(data['actual_pv'][day,slot]/6.)
            if january:
                charge=discharge=0.;nxt=state
                emergency=max(load-pv-plan[slot],0.)
                surplus=max(plan[slot]+pv-load,0.)
            elif common_closure and day==len(dates)-1 and slot>=136:
                charge,discharge,nxt,emergency,surplus=execute_close(plan[slot],load,pv,state,battery=battery)
            elif controller=='greedy':
                charge,discharge,nxt,emergency,surplus=greedy_step(plan[slot],load,pv,state,battery)
            else:
                action,_=execute_midnight_value_policy(plan[slot],load,pv,state,price[slot],
                    policy.next_values(slot),policy.grid,battery=battery)
                charge,discharge,nxt,emergency,surplus=[action[k] for k in ('charge','discharge','soc_end','emergency','surplus')]
            cp=plan[slot]*settle[slot];ce=5.*settle[slot]*emergency
            rows.append(dict(date=date,slot=slot,load=load,pv=pv,price=settle[slot],decision_price=price[slot],
                plan=plan[slot],adjusted=plan[slot],up=0.,down=0.,emergency=emergency,charge=charge,
                discharge=discharge,soc_start=state,soc_end=nxt,surplus=surplus,cost_plan=cp,
                cost_up=0.,cost_down=0.,cost_emergency=ce,cost_total=cp+ce,reference_soc=np.nan))
            state=nxt
    return pd.DataFrame(rows)

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input',type=Path,default=ROOT/'data/processed/annual_inputs.npz')
    parser.add_argument('--actuals',type=Path,default=ROOT/'data/processed/actuals_power.csv')
    parser.add_argument('--out',type=Path,required=True)
    parser.add_argument('--checks',action='store_true')
    args=parser.parse_args()
```

```
    data=load_verified_input(args.input,args.actuals)
    packet=build_prediction_packet(data,'fixed_ridge')
    frame=simulate(data,packet)
    summary=export_annual(frame,args.out,dict(question='q2',controller='value_dp',forecast='fixed_ridge'))
    if args.checks:
        greedy=simulate(data,packet,'greedy')
        fixed={date:part.plan.to_numpy() for date,part in greedy.groupby('date',sort=True)}
        grid_results={}
        for step in (100.,50.,25.):
            trial=simulate(data,packet,fixed_plans=fixed,grid_step_kwh=step)
            grid_results[str(step)]=export_annual(trial,args.out/f'grid_{step:g}',dict(grid_step_kwh=step,fixed_plan=True))
        export_annual(greedy,args.out/'greedy',dict(controller='greedy'))
        (args.out/'grid_comparison.json').write_text(json.dumps(grid_results,indent=2))
        comparison=[]
        for controller in ('greedy','value_dp'):
            candidate=simulate(data,packet,controller,True)
            daily=candidate[candidate.date>='2025-02-01'].groupby('date').cost_total.sum()
            comparison.append(daily)
        interval=moving_block_interval((comparison[0]-comparison[1]).to_numpy())
        (args.out/'controller_comparison.json').write_text(json.dumps(interval,indent=2))
        selected=build_prediction_packet(data,'rolling_selector')
        selected_frame=simulate(data,selected,common_closure=True)
        export_annual(selected_frame,args.out/'forecast_comparison',dict(forecast='rolling_selector'))
        selected_daily=selected_frame[selected_frame.date>='2025-02-01'].groupby('date').cost_total.sum()
        forecast_interval=moving_block_interval((selected_daily-comparison[1]).to_numpy())
        (args.out/'forecast_comparison.json').write_text(json.dumps(forecast_interval,indent=2))
    print(json.dumps(summary,ensure_ascii=False))

if __name__=='__main__':main()
```

代码 B.3　问题三：预报融合与分段更新（problem3.py）

```
from __future__ import annotations
import argparse
import json
from pathlib import Path
from dataclasses import dataclass,asdict,replace
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from microgrid import ROOT,Battery,ValueBattery,optimize,AdjustmentLedger,replan,aggregate
from microgrid import build_rolling_value_table,execute_rolling_value_policy
from microgrid import export_annual

@dataclass(frozen=True)
class ForecastBundle:
    cutoff: pd.Timestamp
    load: np.ndarray
    pv: np.ndarray
    price: np.ndarray
    margin: np.ndarray
    metadata: dict
    candidates: dict

def historical_snapshot(actuals, cutoff):
    cutoff = pd.Timestamp(cutoff)
    return actuals.loc[actuals.available_at <= cutoff].copy(deep=True)

class ForecastProvider:

    def __init__(self, training_days=60, selection_days=14, calibration_days=28):
```

```
        self.training_days = training_days
        self.selection_days = selection_days
        self.calibration_days = calibration_days
        self.records = []

    @staticmethod
    def _features(values, dates, index):
        t = np.arange(144) / 144
        previous = values[index - 1]
        week = values[max(0, index - 7)]
        mean = values[max(0, index - 7):index].mean(axis=0)
        dow = pd.Timestamp(dates[index]).dayofweek
        return np.column_stack([previous, week, mean, np.full(144, previous.mean()), np.sin(2 * np.pi * t), np.cos(2 * np.pi * t), np.sin(4 * np.pi * t), np.cos(4 * np.pi * t), np.full(144, dow >= 5)])

    def predict(self, history, cutoff):
        cutoff = pd.Timestamp(cutoff)
        if len(history) and history.available_at.max() > cutoff:
            raise ValueError('History contains unavailable labels')
        history = history.loc[history.interval_end <= cutoff]
        day_counts = history.groupby('date').size()
        full_dates = day_counts.index[day_counts == 144]
        history = history.loc[history.date.isin(full_dates)].sort_values('interval_start')
        m = len(history) // 144
        if not m:
            raise ValueError('At least one completed day is required')
        columns = ['load_kwh', 'pv_kwh', 'price_variable']
        values = history[columns].to_numpy().reshape(m, 144, 3)
        dates = list(full_dates) + [cutoff.date().isoformat()]
        candidates = {'yesterday': values[-1].copy(), 'last_week': values[max(0, m - 7)].copy()}
        ridge = np.empty((144, 3))
        for k in range(3):
            v = values[:, :, k]
            if m >= 9:
                indices = range(max(7, m - self.training_days), m)
                x = np.vstack([self._features(v, dates, j) for j in indices])
                y = np.concatenate([v[j] for j in indices])
                scaler = StandardScaler().fit(x)
                model = Ridge(alpha=10.0).fit(scaler.transform(x), y)
                ridge[:, k] = model.predict(scaler.transform(self._features(v, dates, m)))
            else:
                ridge[:, k] = v[max(0, m - 7):].mean(axis=0)
        candidates['ridge'] = ridge
        for a in candidates.values():
            a[:, :2] = np.maximum(a[:, :2], 0)
            a[:, 2] = np.maximum(a[:, 2], max(1e-06, values[:, :, 2].min()))
        eligible = [r for r in self.records if r['available_at'] <= cutoff][-self.selection_days:]
        (selected, scores) = ([], {})
        for (k, col) in enumerate(columns):
            scores[col] = {name: float(np.mean([r['candidate_mse'][name][k] for r in eligible])) for name in candidates} if eligible else {name: None for name in candidates}
            selected.append(min(candidates, key=lambda n: scores[col][n]) if eligible else 'last_week')
        prediction = np.column_stack([candidates[name][:, k] for (k, name) in enumerate(selected)])
        (margin, quantile, validation_costs) = self._margin(cutoff)
        metadata = {'cutoff': cutoff.isoformat(), 'training_label_cutoff': history.available_at.max().isoformat(), 'training_days_available': m, 'training_window_days': self.training_days, 'selection_validation_days': len(eligible), 'selected': dict(zip(columns, selected)), 'candidate_mse': scores, 'margin_quantile': quantile, 'margin_validation_cost_proxy': validation_costs, 'ridge_alpha': 10.0, 'ridge_scaling': 'training-fold-only', 'attachment3_used': False}
        return ForecastBundle(cutoff, prediction[:, 0], prediction[:, 1], prediction[:, 2], margin, metadata, candidates)

```

```
    def _margin(self, cutoff):
        rows = [r for r in self.records if r['available_at'] <= cutoff][-self.calibration_days:]
        if not rows:
            return (np.zeros(144), 0.8, {})
        errors = np.vstack([r['net_residual'] for r in rows])
        prices = np.vstack([r['price'] for r in rows])
        levels = [0.5, 0.65, 0.8, 0.9, 0.95]
        groups = np.arange(144) // 24

        def fit(a, level):
            return np.array([max(0, np.quantile(a[:, groups == g], level)) for g in groups])
        costs = {}
        if len(rows) >= 14:
            (train, valid) = (errors[:-7], errors[-7:])
            for level in levels:
                b = fit(train, level)
                costs[str(level)] = float(np.mean(prices[-7:] * (b + 5 * np.maximum(valid - b, 0))
    ))
            best = min(levels, key=lambda a: costs[str(a)])
        else:
            best = 0.8
        return (fit(errors, best), best, costs)

    def observe_completed_day(self, bundle, completed_actuals, settlement_price):
        if len(completed_actuals) != 144:
            raise ValueError('Only a complete labelled day may enter validation records')
        truth = completed_actuals[['load_kwh', 'pv_kwh', 'price_variable']].to_numpy()
        candidate_mse = {name: np.mean((x - truth) ** 2, axis=0).tolist() for (name, x) in bundle.
    candidates.items()}
        self.records.append({'available_at': completed_actuals.available_at.max(), 'candidate_mse'
    : candidate_mse, 'net_residual': truth[:, 0] - truth[:, 1] - bundle.load + bundle.pv, 'price':
    np.asarray(settlement_price).copy()})

def issued_pv_integrals(released_forecasts, issue_time, lead_zero_kw):
    issue_time = pd.Timestamp(issue_time)
    if (released_forecasts.available_at > issue_time).any():
        raise ValueError('Unpublished forecasts supplied to the forecast interface')
    rows = released_forecasts.loc[released_forecasts.issue_time == issue_time].sort_values('
    lead_time')
    if list(rows.lead_time) != list(range(1, 25)):
        raise ValueError('One complete forecast issue with leads 1..24 is required')
    boundary_kw = np.interp(np.arange(145) / 6, np.arange(25), np.r_[max(0, lead_zero_kw), np.
    maximum(rows.value.to_numpy(), 0)])
    return (boundary_kw[:-1] + boundary_kw[1:]) / 12

def lookahead_value_policy(bundle, virtual_contract, candidate, battery, begin):
    vb = ValueBattery(lo=battery.minimum, hi=battery.maximum, power=battery.power_kw, eta_c=
    battery.eta_charge, eta_d=battery.eta_discharge)
    weight = candidate.terminal_weight if candidate.dp_terminal_weight is None else candidate.
    dp_terminal_weight
    target = candidate.terminal if candidate.rolling_terminal is None else candidate.
    rolling_terminal
    if not 0.0 <= candidate.future_error_scale <= 1.0:
        raise ValueError('Future residual scale must be in [0,1]')
    kernels = [samples if t < 36 else np.zeros(1) if candidate.future_error_scale == 0 else
    candidate.future_error_scale * samples for (t, samples) in enumerate(bundle['residual_kernels'
    ])]
    (policy, diagnostics) = build_rolling_value_table(virtual_contract, bundle['load'] - bundle['
    pv'], kernels, bundle['price'], battery=vb, step_kwh=candidate.soc_step_kwh, terminal_kappa=
    weight * float(np.min(bundle['price'])) / battery.eta_charge, terminal_target_kwh=target,
    start_slot=begin, vectorized=True, value_discount=candidate.value_discount)
    info = dict(value_function=diagnostics, control_start_interval=begin, control_stop_interval=
    min(begin + 36, 144), lookahead_stop_interval=144, virtual_contract_scope='
    remaining_day_current_forecast_only', virtual_tail_is_not_committed=True, future_error_scale=
    candidate.future_error_scale, value_assumption='Future virtual contracts held fixed within this
     value evaluation; recomputed at next issue')
```

```
    return (policy, info)

def execute_lookahead_action(contract, load, pv, state, price, policy, t, battery):
    vb = ValueBattery(lo=battery.minimum, hi=battery.maximum, power=battery.power_kw, eta_c=
    battery.eta_charge, eta_d=battery.eta_discharge)
    (action, explanation) = execute_rolling_value_policy(contract, load, pv, state, price, policy.
    next_values(t), policy.grid, battery=vb, general_search=True)
    action['unused'] = action.pop('surplus')
    action['soc_start'] = state
    used_pv = min(pv, load + action['charge'])
    used_grid = min(contract, max(0.0, load + action['charge'] - used_pv - action['discharge']))
    action['unused_pv'] = pv - used_pv
    action['unused_purchased'] = contract - used_grid
    action['held_with_emergency'] = explanation['economic_hold_with_emergency']
    return action

@dataclass(frozen=True)
class Candidate:
    name: str
    question: str
    blend: float = 1.0
    quantile: float = 0.8
    window: int = 14
    bias: bool = True
    adjustment: str = 'next6'
    terminal: float = 6000.0
    load_feedback: float = 0.0
    price_days: int = 0
    interpretation: str = 'B'
    reference_mode: str = 'initial_final'
    intraday_quantile: float | None = None
    terminal_weight: float = 1.0
    forecast_updates: str = 'latest'
    skip_hours: tuple = ()
    allow_decrease: bool = True
    early_quantile: float | None = None
    rolling_terminal: float | None = None
    reserve_fraction: float = 0.0
    controller: str = 'lookahead_dp'
    soc_step_kwh: float = 50.0
    dp_terminal_weight: float | None = None
    future_error_scale: float = 1.0
    value_discount: float = 1.0

    def __post_init__(self):
        if self.interpretation != 'B':
            raise ValueError('Only corrected 50-percent cancellation pricing is supported')
        if self.reference_mode not in ('initial_final', 'sequential', 'unspecified'):
            raise ValueError('Invalid multi-adjustment reference')
        if self.adjustment != 'none' and self.reference_mode == 'unspecified':
            raise ValueError('Specify the multi-adjustment reference before running')

def ordered_actuals(frame):
    frame = frame.sort_values(['date', 'interval_index']).reset_index(drop=True)
    if frame.duplicated(['date', 'interval_index']).any():
        raise ValueError('Duplicate date/interval key')
    for (_, day) in frame.groupby('date', sort=True):
        if not np.array_equal(day.interval_index.to_numpy(), np.arange(144)):
            raise ValueError('Incomplete day')
    if not np.array_equal(frame.interval_end.to_numpy()[:-1], frame.interval_start.to_numpy()[1:])
    :
        raise ValueError('Non-continuous interval timestamps')
    return frame

def prepare(actuals, forecasts, fixed_price):
    actuals = ordered_actuals(actuals)
```

```
    days = np.array(sorted(actuals.date.unique()))
    n = len(days)
    cache = {k: np.full((n, 144), np.nan) for k in ['load', 'historical_pv', 'price']}
    cache['issued_pv'] = np.full((n, 4, 144), np.nan)
    cache['truth_load'] = actuals.load_kwh.to_numpy().reshape(n, 144)
    cache['truth_pv'] = actuals.pv_kwh.to_numpy().reshape(n, 144)
    cache['truth_price'] = actuals.price_variable.to_numpy().reshape(n, 144)
    cache['fixed_price'] = fixed_price
    metadata = [None] * n
    provider = ForecastProvider()
    for (i, date) in enumerate(days):
        if i == 0:
            continue
        cutoff = pd.Timestamp(str(date))
        history = historical_snapshot(actuals, cutoff)
        bundle = provider.predict(history, cutoff)
        for (key, value) in [('load', bundle.load), ('historical_pv', bundle.pv), ('price', bundle.price)]:
            cache[key][i] = value
        metadata[i] = bundle.metadata
        for (j, t) in enumerate([0, 36, 72, 108]):
            issue = cutoff + pd.Timedelta(minutes=10 * t)
            visible = historical_snapshot(actuals, issue)
            released = forecasts.loc[forecasts.available_at <= issue]
            cache['issued_pv'][i, j, t:] = issued_pv_integrals(released, issue, visible.pv_kw.iloc[-1])[:144 - t]
        day = actuals.loc[actuals.date == date]
        provider.observe_completed_day(bundle, day, day.price_variable.to_numpy())
        if i % 60 == 0 or i == n - 1:
            print(json.dumps({'phase': 'forecast_cache', 'date': date}), flush=True)
    return (days, cache, metadata)

def base_prediction(cache, i, j, candidate):
    t = 36 * j
    source_issue = 0 if candidate.forecast_updates == 'frozen_midnight' else j
    load = cache['load'][i, t:].copy()
    historical = cache['historical_pv'][i, t:]
    pv = (1 - candidate.blend) * historical + candidate.blend * cache['issued_pv'][i, source_issue, t:] if candidate.question != 'Q4-2' else historical.copy()
    if candidate.bias and i > 1:
        begin = max(1, i - 3)
        load += np.mean(cache['truth_load'][begin:i, t:] - cache['load'][begin:i, t:], axis=0)
        past_pv = cache['historical_pv'][begin:i, t:]
        if candidate.question != 'Q4-2':
            past_pv = (1 - candidate.blend) * past_pv + candidate.blend * cache['issued_pv'][begin:i, source_issue, t:]
        pv += np.mean(cache['truth_pv'][begin:i, t:] - past_pv, axis=0)
    if j and candidate.load_feedback:
        (midnight_load, _) = base_prediction(cache, i, 0, candidate)
        correction = np.mean(cache['truth_load'][i, max(0, t - 12):t] - midnight_load[max(0, t - 12):t])
        load += candidate.load_feedback * correction * np.exp(-np.arange(144 - t) / 36)
    return (np.maximum(load, 0), np.maximum(pv, 0))

def packet(days, cache, metadata, i, j, candidate):
    t = j * 36
    (load, pv) = base_prediction(cache, i, j, candidate)
    begin = max(1, i - candidate.window)
    residuals = []
    for past in range(begin, i):
        (pl, pp) = base_prediction(cache, past, j, candidate)
        residuals.append(cache['truth_load'][past, t:] - cache['truth_pv'][past, t:] - pl + pp)
    margin = np.zeros(144 - t)
    quantile = candidate.intraday_quantile if j and candidate.intraday_quantile is not None else candidate.quantile
    kernels = [np.zeros(1) for _ in range(144 - t)]
```

```
    if residuals:
        residuals = np.array(residuals)
        groups = residuals.reshape(len(residuals), (144 - t) // 6, 6).transpose(1, 0, 2).reshape((144 - t) // 6, -1)
        kernels = [groups[h // 6].copy() for h in range(144 - t)]
        if quantile > 0:
            margin = np.repeat(np.maximum(np.quantile(groups, quantile, axis=1), 0), 6)
        if j == 0 and candidate.early_quantile is not None:
            margin[:36] = np.repeat(np.maximum(np.quantile(groups[:6], candidate.early_quantile, axis=1), 0), 6)
    price = cache['fixed_price'][t:] if candidate.question == 'Q3' else cache['price'][i, t:]
    if candidate.question != 'Q3' and candidate.price_days:
        price = cache['truth_price'][max(0, i - candidate.price_days):i, t:].mean(axis=0)
    info = dict(metadata[i], cutoff=str(pd.Timestamp(str(days[i])) + pd.Timedelta(minutes=10 * t)), issue_hour=j * 6, last_calibration_date=str(days[i - 1]), calibration_window_days=i - begin, attachment3_used=candidate.question != 'Q4-2' and candidate.blend > 0, forecast_blend=candidate.blend, bias_corrected=candidate.bias, margin_quantile=quantile, load_feedback=candidate.load_feedback, forecast_issue_hour=0 if candidate.forecast_updates == 'frozen_midnight' else j * 6, price_history_days=candidate.price_days)
    info['early_quantile'] = candidate.early_quantile
    return dict(load=load, pv=pv, margin=margin, price=price.copy(), metadata=info, residual_kernels=kernels)

def day_plan(bundle, state, candidate, battery):
    result = optimize(bundle['load'] + bundle['margin'], bundle['pv'], bundle['price'], initial=state, terminal=candidate.terminal, battery=battery, relax=True)
    if result['solver']['status'] != 0:
        raise RuntimeError('Uncertified day-ahead solve: ' + str(result['solver']))
    if np.minimum(result['charge'], result['discharge']).max() > 1e-07:
        result = optimize(bundle['load'] + bundle['margin'], bundle['pv'], bundle['price'], initial=state, terminal=candidate.terminal, battery=battery)
    if result['solver']['status'] != 0:
        raise RuntimeError('Uncertified MILP day-ahead solve')
    return result

def run_candidate(days, cache, metadata, actuals, candidate, start, end, battery=Battery()):
    if candidate.controller != 'lookahead_dp' or candidate.question not in ('Q3', 'Q4-3') or candidate.adjustment != 'next6':
        raise ValueError('Only the current Q3/Q4-3 lookahead-DP structure is supported')
    (state, frames, decisions, event_rows) = (6000.0, [], [], [])
    by_day = dict(tuple(actuals.groupby('date', sort=True)))
    for i in range(start, end + 1):
        date = str(days[i])
        day = by_day[date]
        p = packet(days, cache, metadata, i, 0, candidate)
        plan = day_plan(p, state, candidate, battery)
        mode = candidate.reference_mode if candidate.reference_mode != 'unspecified' else 'initial_final'
        ledger = AdjustmentLedger(plan['grid'], mode)
        (policy, value_info) = lookahead_value_policy(p, plan['grid'], candidate, battery, 0)
        decisions.append(dict(p['metadata'], strategy=candidate.name, decision_type='day_ahead', initial_soc=state, solver=plan['solver'], changed_contract=False, **value_info))
        actions = []
        actual_price = cache['fixed_price'] if candidate.question == 'Q3' else cache['truth_price'][i]
        control_price = actual_price if candidate.question == 'Q3' else p['price'].copy()
        for (t, row) in enumerate(day.itertuples(index=False)):
            if t in (36, 72, 108) and candidate.adjustment != 'none' and (t // 6 not in candidate.skip_hours):
                updated = packet(days, cache, metadata, i, t // 36, candidate)
                reference = ledger.original[t:] if mode == 'initial_final' else ledger.contract[t:]
                proposed = replan(updated['load'] + updated['margin'], updated['pv'], updated['price'], reference, state, candidate.terminal if candidate.rolling_terminal is None else candidate.rolling_terminal, battery, candidate.terminal_weight, minimum_contract=ledger.contract[t:] if not candidate.allow_decrease else None)
```

```
                if proposed['solver']['status'] != 0:
                    raise RuntimeError('Uncertified replan')
                grid = ledger.contract.copy()
                stop = min(t + 36, 144) if candidate.adjustment == 'next6' else 144
                grid[t:stop] = proposed['grid'][:stop - t]
                issue = pd.Timestamp(date) + pd.Timedelta(minutes=10 * t)
                changed = ledger.revise(grid, t, issue)
                plan['soc'][t:] = proposed['soc']
                (policy, value_info) = lookahead_value_policy(updated, proposed['grid'], candidate, battery, t)
                decisions.append(dict(updated['metadata'], strategy=candidate.name, decision_type='intraday', initial_soc=state, solver=proposed['solver'], changed_contract=changed, committed_until_interval=stop, proposal_scope='remaining day; only stated commit window applied', **value_info))
            action = execute_lookahead_action(ledger.contract[t], row.load_kwh, row.pv_kwh, state, control_price[t], policy, t, battery)
            state = action['soc_end']
            actions.append(action)
        frame = day[['date', 'interval_index', 'interval_start', 'interval_end', 'load_kwh', 'pv_kwh']].reset_index(drop=True)
        frame = pd.concat([frame, pd.DataFrame(actions)], axis=1)
        (frame['price'], frame['plan_price']) = (actual_price, p['price'])
        if candidate.question == 'Q4-3':
            frame['control_price'] = control_price
        (frame['planned_grid'], frame['contract_grid']) = (ledger.original, ledger.contract)
        for (key, values) in ledger.settle(actual_price, frame.emergency).items():
            frame[key] = values
        for event in ledger.events:
            if mode == 'initial_final':
                (before_delta, after_delta) = (event['before'] - ledger.original, event['after'] - ledger.original)
                inc_component = np.maximum(after_delta, 0) - np.maximum(before_delta, 0)
                dec_component = np.maximum(-after_delta, 0) - np.maximum(-before_delta, 0)
            else:
                (inc_component, dec_component) = (event['increase'], event['decrease'])
            for t in np.flatnonzero(event['increase'] + event['decrease'] > 1e-07):
                event_rows.append(dict(date=date, issue_time=event['issue_time'], effective_from=event['effective_from'], interval_index=int(t), increase_kwh=float(event['increase'][t]), decrease_kwh=float(event['decrease'][t]), delivery_price=float(actual_price[t]), increase_fee=float(1.5 * actual_price[t] * inc_component[t]), decrease_penalty=float(0.5 * actual_price[t] * dec_component[t]), refund=float(actual_price[t] * dec_component[t])))
        frames.append(frame)
    frame = pd.concat(frames, ignore_index=True)
    (daily, monthly, summary) = aggregate(frame, battery)
    summary.update(strategy=asdict(candidate), contract_modification_decisions=sum((d['changed_contract'] for d in decisions)), intraday_forecast_updates=sum((d['decision_type'] == 'intraday' for d in decisions)), contract_event_rows=len(event_rows), optimality='BEST_FOUND_POLICY_NOT_GLOBAL_OPTIMUM')
    summary['value_table_rebuilds'] = len(decisions)
    summary['economic_hold_intervals'] = int(frame.held_with_emergency.sum())
    events = pd.DataFrame(event_rows, columns=['date', 'issue_time', 'effective_from', 'interval_index', 'increase_kwh', 'decrease_kwh', 'delivery_price', 'increase_fee', 'decrease_penalty', 'refund'])
    return (frame, daily, monthly, summary, decisions, events)

def forecast_accuracy(days,cache,metadata,candidate,first):
    metrics=[];example=[]
    for issue in range(4):
        raw_errors=[];fused_errors=[]
        for day in range(first,len(days)):
            bundle=packet(days,cache,metadata,day,issue,candidate)
            start=36*issue
            truth=cache['truth_pv'][day,start:start+36]
            raw=cache['issued_pv'][day,issue,start:start+36]
            raw_errors.extend(raw-truth);fused_errors.extend(bundle['pv'][:36]-truth)
            if str(days[day])=='2025-03-20':
```

```
                for t in range(36):
                    example.append(dict(interval_index=start+t,issue_hour=issue*6,actual_pv_kwh=
    float(truth[t]),
                        issued_pv_kwh=float(raw[t]),fused_pv_kwh=float(bundle['pv'][t]),
                        load_forecast_kwh=float(bundle['load'][t]),margin_kwh=float(bundle['margin
    '][t])))
        for name,errors in [('发布预报',raw_errors),('融合修正预报',fused_errors)]:
            errors=np.asarray(errors)
            metrics.append(dict(issue_hour=issue*6,method=name,intervals=len(errors),
                rmse_kwh=float(np.sqrt(np.mean(errors**2))),mae_kwh=float(np.mean(abs(errors)))))
    return pd.DataFrame(metrics),pd.DataFrame(example)

def main():
    parser=argparse.ArgumentParser();parser.add_argument('--out',type=Path,required=True)
    parser.add_argument('--checks',action='store_true');args=parser.parse_args()
    actual=ordered_actuals(pd.read_csv(ROOT/'data/processed/actuals_energy.csv',parse_dates=['
    interval_start','interval_end','available_at']))
    forecasts=pd.read_csv(ROOT/'data/processed/pv_forecasts.csv',parse_dates=['issue_time','
    available_at','target_time'])
    price=pd.read_csv(ROOT/'data/processed/q1_inputs.csv').sort_values('interval_index').price.
    to_numpy()
    candidate=Candidate(**json.loads((ROOT/'configs/q3_current.json').read_text())[0])
    days,cache,metadata=prepare(actual,forecasts,price)
    start=list(days).index('2025-02-01')
    result=run_candidate(days,cache,metadata,actual,candidate,start,len(days)-1)
    summary=export_annual(result[0],args.out,asdict(candidate),result[3])
    if args.checks:
        metrics,example=forecast_accuracy(days,cache,metadata,candidate,start)
        metrics.to_csv(args.out/'forecast_metrics.csv',index=False)
        example.to_csv(args.out/'forecast_example.csv',index=False)
        for name,setting in [('frozen_forecast',replace(candidate,forecast_updates='
    frozen_midnight')),('grid25',replace(candidate,soc_step_kwh=25.))]:
            comparison=run_candidate(days,cache,metadata,actual,setting,start,len(days)-1)
            export_annual(comparison[0],args.out/name,asdict(setting),comparison[3])
    print(json.dumps(summary,ensure_ascii=False))

if __name__=='__main__':main()
```

代码 B.4　问题四：电价预测与购电策略（problem4.py）

```
from __future__ import annotations
import argparse
import json
import tempfile
import warnings
from pathlib import Path
from dataclasses import dataclass,asdict,replace
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
from microgrid import ROOT,Battery,ValueBattery,optimize,daily_plan,AdjustmentLedger,replan,
    aggregate,load_verified_input
from microgrid import build_rolling_value_table,execute_rolling_value_policy
from problem2 import build_prediction_packet,simulate,moving_block_interval
from problem3 import Candidate,ordered_actuals,prepare as prepare_q3,base_prediction,
    ForecastProvider
from microgrid import export_annual

FEATURE_COLUMNS = ([0, 1, 3, 4, 6, 7, 9, 10, 11, 12, 13, 14], [0, 1, 3, 4, 6, 7, 9, 10, 11, 12,
    13, 14], [2, 5, 8, 9, 10, 11, 12, 13, 14])

def baseline(history, fallback, day, method='recent', lead=0):
    if day == 0:
```

```
        return np.asarray(fallback).copy()
    if method == 'previous':
        return history[day - 1].copy()
    same_weekday = [j for j in range(max(0, day - 28), day) if (day + lead - j) % 7 == 0]
    seasonal = history[same_weekday[-3:]].mean(axis=0) if same_weekday else history[max(0, day - 7):day].mean(axis=0)
    if method == 'weekly':
        return seasonal
    return 0.5 * seasonal + 0.5 * history[max(0, day - 7):day].mean(axis=0)

def features(history, fallback, day, lead=0):
    slot = np.arange(144)
    prev = baseline(history, fallback, day, 'previous', lead)
    week = baseline(history, fallback, day, 'weekly', lead)
    recent = baseline(history, fallback, day, 'recent', lead)
    return np.column_stack([prev, week, recent, np.sin(2 * np.pi * slot / 144), np.cos(2 * np.pi * slot / 144), np.full(144, (day + lead + 2) % 7), np.full(144, np.sin(2 * np.pi * (day + lead) / 365)), np.full(144, np.cos(2 * np.pi * (day + lead) / 365)), np.full(144, lead)])

def build_forecasts(actual, fallback, method='recent', days=None):
    nd = len(actual) if days is None else min(days, len(actual))
    result = np.zeros((nd, 3, 144, 3))
    models = None
    fitted_at = []
    with threadpool_limits(limits=1):
        for d in range(nd):
            if method == 'hgb' and d >= 14 and (models is None or d % 7 == 0):
                origins = range(max(1, d - 60), d - 2)
                xx = np.vstack([features(actual, fallback, j, lead) for j in origins for lead in (0, 1, 2)])
                yy = np.vstack([actual[j + lead] for j in origins for lead in (0, 1, 2)])
                models = []
                for k in range(3):
                    reg = HistGradientBoostingRegressor(max_iter=60, max_leaf_nodes=15, learning_rate=0.08, l2_regularization=1.0, random_state=2026, early_stopping=False)
                    reg.fit(xx[:, FEATURE_COLUMNS[k]], yy[:, k])
                    models.append(reg)
                fitted_at.append(d)
            for lead in range(3):
                if method != 'hgb' or models is None:
                    y = baseline(actual, fallback, d, 'recent' if method == 'hgb' else method, lead)
                else:
                    x = features(actual, fallback, d, lead)
                    y = np.column_stack([reg.predict(x[:, FEATURE_COLUMNS[k]]) for (k, reg) in enumerate(models)])
                result[d, lead] = np.maximum(y, 0)
                result[d, lead, :, 2] = np.maximum(result[d, lead, :, 2], 0.001)
    return (result, fitted_at)

def predict_prices(dates, prices, return_candidates=False):
    dates = [str(date) for date in dates]
    prices = np.asarray(prices, dtype=float)
    if prices.shape != (len(dates), 144) or not np.isfinite(prices).all() or np.any(prices <= 0):
        raise ValueError('Expected complete positive daily price arrays')
    names = ('previous', 'weekly', 'ridge')
    candidates = np.full((len(dates), 3, 144), np.nan)
    prediction = np.full_like(prices, np.nan)
    losses = np.full((len(dates), 3), np.nan)
    records = []
    with threadpool_limits(limits=1):
        for d in range(1, len(dates)):
            candidates[d, 0] = prices[d - 1]
            candidates[d, 1] = prices[max(0, d - 7)]
            candidates[d, 2] = prices[max(0, d - 7):d].mean(axis=0)
            if d >= 9:
```

```
                indices = range(max(7, d - 60), d)
                x = np.vstack([ForecastProvider._features(prices, dates, j) for j in indices])
                y = prices[list(indices)].ravel()
                scaler = StandardScaler().fit(x)
                model = Ridge(alpha=10.0, solver='cholesky').fit(scaler.transform(x), y)
                candidates[d, 2] = model.predict(scaler.transform(ForecastProvider._features(prices, dates, d)))
            candidates[d] = np.maximum(candidates[d], 1e-06)
            begin = max(1, d - 14)
            scores = losses[begin:d].mean(axis=0) if d > 1 else np.zeros(3)
            chosen = int(np.argmin(scores)) if d > 1 else 1
            prediction[d] = candidates[d, chosen]
            records.append(dict(date=str(dates[d]), cutoff=str(dates[d]) + 'T00:00:00', latest_observed_date=str(dates[d - 1]), selected=names[chosen], selection_days=d - begin, scores=dict(zip(names, scores.tolist()))))
            losses[d] = np.mean((candidates[d] - prices[d]) ** 2, axis=1)
    rows = []
    mask = np.asarray(dates) >= '2025-02-01'
    for (name, forecast) in [('selected', prediction)] + [(name, candidates[:, j]) for (j, name) in enumerate(names)]:
        error = forecast[mask] - prices[mask]
        rows.append(dict(method=name, mae_yuan_per_kwh=float(np.abs(error).mean()), rmse_yuan_per_kwh=float(np.sqrt(np.mean(error ** 2)))))
    result = (prediction, records, pd.DataFrame(rows))
    return (*result, candidates) if return_candidates else result

def price_family(dates, truth):
    (selected, records, metrics, candidates) = predict_prices(dates, truth, return_candidates=True)
    ridge = candidates[:, 2]
    smooth = np.repeat(ridge.reshape(-1, 24, 6).mean(axis=2), 6, axis=1)
    profile = np.full_like(selected, np.nan)
    for day in range(1, len(dates)):
        history = truth[max(0, day - 28):day]
        shape = (history / history.mean(axis=1, keepdims=True)).mean(axis=0)
        profile[day] = shape * ridge[day].mean()
    return (dict(selected=selected, ridge=ridge, smooth_ridge=smooth, weekly=candidates[:, 1], profile=profile), records, metrics)

def decision_prices(family, truth, day, begin, method, bias_weight=0.0, bias_window=3, feedback=0.0):
    base = family[method][day].copy()
    if bias_weight and day > 1:
        first = max(1, day - bias_window)
        base += bias_weight * (truth[first:day] - family[method][first:day]).mean(axis=0)
    base = np.maximum(base, 1e-06)
    if begin and feedback:
        offset = float(np.mean(truth[day, max(0, begin - 12):begin] - base[max(0, begin - 12):begin]))
        base[begin:] += feedback * offset * np.exp(-np.arange(144 - begin) / 36)
    return np.maximum(base[begin:], 1e-06)

@dataclass(frozen=True)
class Policy:
    part: str
    price_method: str = 'selected'
    bias_weight: float = 0.0
    bias_window: int = 3
    price_feedback: float = 0.0
    risk: float = 0.8
    intraday_risk: float = 0.625
    residual_days: int = 14
    price_weighted_margin: bool = False
    plan_terminal: float = 6000.0
    terminal_weight: float = 1.0
    discount: float = 1.0
```

```
    grid_step: float = 50.0
    final_depletion: bool = False

def default_policy(part):
    if part == '2':
        return Policy(part='2')
    return Policy(part='3', risk=0.65, residual_days=56, plan_terminal=2400.0, terminal_weight
    =1.5, discount=0.95)

def margins_and_kernels(cache, policy, day, issue):
    begin = issue * 36
    first = max(1, day - policy.residual_days)
    errors = cache['errors'][first:day, issue, begin:]
    groups = errors.reshape(len(errors), -1, 6).transpose(1, 0, 2).reshape((144 - begin) // 6, -1)
    risk = policy.intraday_risk if issue else policy.risk
    if policy.price_weighted_margin:
        weights = cache['truth_price'][first:day, begin:].reshape(len(errors), -1, 6).transpose(1,
     0, 2).reshape(groups.shape)
        values = []
        for (samples, w) in zip(groups, weights):
            order = np.argsort(samples, kind='stable')
            values.append(np.interp(risk, (np.cumsum(w[order]) - 0.5 * w[order]) / w.sum(),
    samples[order]))
        margin = np.repeat(np.maximum(values, 0), 6)
    else:
        margin = np.repeat(np.maximum(np.quantile(groups, risk, axis=1), 0), 6)
    return (margin, [groups[t // 6] for t in range(144 - begin)])

def run(cache, family, policy, start=31, stop=None, collect=False, progress=False, initial_soc
    =6000.0):
    dates = cache['dates']
    stop = len(dates) if stop is None else stop
    state = float(initial_soc)
    total = 0.0
    emergency_total = 0.0
    frames = []
    logs = []
    event_rows = []
    daily_rows = []
    physical = Battery()
    value_battery = ValueBattery()
    for day in range(start, stop):
        date = str(dates[day])
        day_start = state
        last = policy.final_depletion and date == '2025-12-31'
        terminal = 1200.0 if last else policy.plan_terminal
        midnight = decision_prices(family, cache['truth_price'], day, 0, policy.price_method,
    policy.bias_weight, policy.bias_window)
        (margin, kernels) = margins_and_kernels(cache, policy, day, 0)
        forecast = cache['base'][day, 0]
        if policy.part == '2':
            planned = daily_plan(forecast, margin, midnight, state, replace(value_battery, target=
    terminal))
            q = planned['q']
            solver = dict(status=0, objective=planned['objective'], maximum_residual_kwh=planned['
    residual'])
        else:
            planned = optimize(forecast[:, 0] + margin, forecast[:, 1], midnight, initial=state,
    terminal=terminal, relax=True)
            if np.minimum(planned['charge'], planned['discharge']).max() > 1e-07:
                planned = optimize(forecast[:, 0] + margin, forecast[:, 1], midnight, initial=
    state, terminal=terminal)
            q = planned['grid']
            solver = planned['solver']
        if solver['status'] != 0:
            raise RuntimeError(solver)
```

```
197         ledger = AdjustmentLedger(q, 'initial_final')
198         control = midnight.copy()
199         virtual = q.copy()
200         actions = []
201         update_times = (0, 36, 72, 108) if policy.part == '3' or policy.price_feedback else (0,)
202         for t in range(144):
203             if t in update_times:
204                 issue = t // 36
205                 prices = decision_prices(family, cache['truth_price'], day, t, policy.price_method
    , policy.bias_weight, policy.bias_window, policy.price_feedback)
206                 control[t:] = prices
207                 if t and policy.part == '3':
208                     forecast = cache['base'][day, issue, t:]
209                     (margin, kernels) = margins_and_kernels(cache, policy, day, issue)
210                     proposed = replan(forecast[:, 0] + margin, forecast[:, 1], prices, ledger.
    original[t:], state, 1200.0 if last else 6000.0, physical, 0.0 if last else policy.
    terminal_weight)
211                     target = ledger.contract.copy()
212                     target[t:t + 36] = proposed['grid'][:36]
213                     ledger.revise(target, t, pd.Timestamp(date) + pd.Timedelta(minutes=10 * t))
214                     virtual = proposed['grid']
215                     solver = proposed['solver']
216                 elif t:
217                     forecast = cache['base'][day, 0, t:]
218                     (_, all_kernels) = margins_and_kernels(cache, policy, day, 0)
219                     kernels = all_kernels[t:]
220                     virtual = ledger.contract[t:]
221                 kappa = 0.0 if last else policy.terminal_weight * float(prices.min()) / 0.9
222                 (vp, diag) = build_rolling_value_table(virtual, forecast[:, 0] - forecast[:, 1],
    kernels, prices, terminal_kappa=kappa, terminal_target_kwh=6000.0, step_kwh=policy.grid_step,
    start_slot=t, vectorized=True, value_discount=policy.discount)
223                 if collect:
224                     cutoff = pd.Timestamp(date) + pd.Timedelta(minutes=10 * t)
225                     logs.append(dict(date=date, cutoff=str(cutoff), issue_hour=t // 6,
    decision_type='day_ahead' if not t else 'intraday', initial_soc=state, solver=solver,
    last_calibration_date=str(dates[day - 1]), training_label_cutoff=str(pd.Timestamp(date)),
    value_function=diag, price_forecast=dict(method=policy.price_method, latest_observed_date=str(
    dates[day - 1]), feedback_through=str(cutoff) if t and policy.price_feedback else date + '
    00:00:00'), predicted_prices=prices.tolist(), control_start_interval=t, lookahead_stop_interval
    =144, control_stop_interval=min(t + 36, 144) if len(update_times) > 1 else 144,
    committed_until_interval=min(t + 36, 144) if policy.part == '3' and t else 144,
    virtual_tail_is_not_committed=policy.part == '3', final_depletion=last))
226             load = cache['truth_load'][day, t]
227             pv = cache['truth_pv'][day, t]
228             (action, _) = execute_rolling_value_policy(ledger.contract[t], load, pv, state,
    control[t], vp.next_values(t), vp.grid, general_search=policy.part == '3')
229             action['soc_start'] = state
230             action['unused'] = action.pop('surplus')
231             used_pv = min(pv, load + action['charge'])
232             used_grid = min(ledger.contract[t], max(0.0, load + action['charge'] - used_pv -
    action['discharge']))
233             action['unused_pv'] = pv - used_pv
234             action['unused_purchased'] = ledger.contract[t] - used_grid
235             state = action['soc_end']
236             actions.append(action)
237         emergency = np.array([a['emergency'] for a in actions])
238         fees = ledger.settle(cache['truth_price'][day], emergency)
239         amount = float(fees['total_fee'].sum())
240         total += amount
241         emergency_total += emergency.sum()
242         daily_rows.append(dict(date=date, total_fee=amount, initial_soc=day_start, terminal_soc=
    state, emergency=float(emergency.sum())))
243         if collect:
244             times = pd.date_range(date, periods=144, freq='10min')
245             frame = pd.DataFrame(dict(date=date, interval_index=np.arange(144), interval_start=
    times, interval_end=times + pd.Timedelta(minutes=10), load_kwh=cache['truth_load'][day], pv_kwh
```

```
=cache['truth_pv'][day], price=cache['truth_price'][day], plan_price=midnight, control_price=
control, planned_grid=ledger.original, contract_grid=ledger.contract))
            frame = pd.concat([frame, pd.DataFrame(actions)], axis=1)
            for (key, values) in fees.items():
                frame[key] = values
            frames.append(frame)
            for event in ledger.events:
                for k in np.flatnonzero(event['increase'] + event['decrease'] > 1e-07):
                    event_rows.append(dict(date=date, issue_time=event['issue_time'],
effective_from=event['effective_from'], interval_index=int(k), increase_kwh=float(event['
increase'][k]), decrease_kwh=float(event['decrease'][k]), delivery_price=float(cache['
truth_price'][day, k]), increase_fee=float(1.5 * cache['truth_price'][day, k] * event['increase
'][k]), decrease_penalty=float(0.5 * cache['truth_price'][day, k] * event['decrease'][k]),
refund=float(cache['truth_price'][day, k] * event['decrease'][k])))
        if progress and (day == stop - 1 or str(dates[day + 1])[:7] != date[:7]):
            print(json.dumps(dict(part=policy.part, completed=date, total_fee=total)), flush=True)
    if not collect:
        return dict(total_fee=total, terminal_soc=state, emergency=emergency_total, daily=
daily_rows)
    frame = pd.concat(frames, ignore_index=True)
    (daily, monthly, summary) = aggregate(frame, physical)
    summary.update(strategy=dict(**asdict(policy), question='Q4-' + policy.part, reference_mode='
initial_final', interpretation='B', adjustment='next6' if policy.part == '3' else 'none',
controller='empirical_dp'), optimality='Fixed empirical-DP policy; no global online optimality
certificate', submission_ready=False)
    return (frame, daily, monthly, summary, logs, pd.DataFrame(event_rows, columns=['date', '
issue_time', 'effective_from', 'interval_index', 'increase_kwh', 'decrease_kwh', '
delivery_price', 'increase_fee', 'decrease_penalty', 'refund']))

def prepare_price_cache(part, output):
    if part != '3':
        raise ValueError('Only final Q4-3 cache preparation is supported')
    actual = ordered_actuals(pd.read_csv(ROOT / 'data/processed/actuals_energy.csv', parse_dates=[
'interval_start', 'interval_end', 'available_at']))
    dates = np.asarray(actual.date.unique(), dtype=str)
    n = len(dates)
    base = np.full((n, 4, 144, 2), np.nan)
    cache = {'dates': dates, 'truth_load': actual.load_kwh.to_numpy().reshape(-1, 144), 'truth_pv'
: actual.pv_kwh.to_numpy().reshape(-1, 144), 'truth_price': actual.price_variable.to_numpy().
reshape(-1, 144)}
    forecasts = pd.read_csv(ROOT / 'data/processed/pv_forecasts.csv', parse_dates=['issue_time', '
available_at', 'target_time'])
    tariff = pd.read_csv(ROOT / 'data/processed/q1_inputs.csv').sort_values('interval_index').
price.to_numpy()
    with threadpool_limits(limits=1), warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        (_, q3cache, _) = prepare_q3(actual, forecasts, tariff)
        candidate = Candidate(**json.loads((ROOT / 'configs/q3_current.json').read_text())[0])
        for day in range(1, n):
            for issue in range(4):
                (load_prediction, pv_prediction) = base_prediction(q3cache, day, issue, candidate)
                base[day, issue, issue * 36:] = np.column_stack([load_prediction, pv_prediction])
    cache['base'] = base
    cache['errors'] = cache['truth_load'][:, None, :] - cache['truth_pv'][:, None, :] - base[:, :,
    :, 0] + base[:, :, :, 1]
    np.savez_compressed(output, **cache)

def load(path):
    with np.load(path, allow_pickle=False) as data:
        return {key: data[key] for key in data.files}

def price_forecasts(data):
    actual=np.stack([data['actual_load'],data['actual_pv'],data['actual_price']],axis=-1)
    fallback=np.zeros((144,3));fallback[:,2]=.5
    predicted,_=build_forecasts(actual,fallback,method='hgb')
    return predicted[:,0,:,2]
```

```

def main():
    parser=argparse.ArgumentParser();parser.add_argument('--part',choices=['2','3'],required=True)
    parser.add_argument('--out',type=Path,required=True);parser.add_argument('--checks',action='store_true')
    args=parser.parse_args()
    data=load_verified_input(ROOT/'data/processed/annual_inputs.npz',ROOT/'data/processed/actuals_power.csv')
    predicted=price_forecasts(data)
    if args.part=='2':
        packet=build_prediction_packet(data,'fixed_ridge')
        frame=simulate(data,packet,price_forecast=predicted)
        summary=export_annual(frame,args.out,dict(question='q4_2',controller='value_dp',forecast='fixed_ridge'))
        if args.checks:
            checks=[]
            for controller in ('greedy','value_dp'):
                trial=simulate(data,packet,controller,True,price_forecast=predicted)
                checks.append(trial[trial.date>='2025-02-01'].groupby('date').cost_total.sum())
            (args.out/'controller_comparison.json').write_text(json.dumps(moving_block_interval((checks[0]-checks[1]).to_numpy()),indent=2))
    else:
        with tempfile.TemporaryDirectory() as temporary:
            cachefile=Path(temporary)/'forecast.npz';prepare_price_cache('3',cachefile);cache=load(cachefile)
        policy=Policy(part='3',price_method='hgb',risk=.65,intraday_risk=.625,residual_days=56,
                      plan_terminal=2400.,terminal_weight=1.5,discount=.95,grid_step=50.)
        with threadpool_limits(limits=1):result=run(cache,{'hgb':predicted},policy,collect=True,progress=True)
        summary=export_annual(result[0],args.out,asdict(policy),result[3])
        if args.checks:
            with threadpool_limits(limits=1):trial=run(cache,{'hgb':predicted},replace(policy,grid_step=25.),collect=True)
            export_annual(trial[0],args.out/'grid25',asdict(replace(policy,grid_step=25.)),trial[3])
    error=predicted[31:]-data['actual_price'][31:]
    (args.out/'price_metrics.json').write_text(json.dumps(dict(mae=float(np.abs(error).mean()),rmse=float(np.sqrt(np.mean(error**2)))),indent=2))
    if args.checks:
        price_family(data['dates'],data['actual_price'])[2].to_csv(args.out/'price_baselines.csv',index=False)
    print(json.dumps(summary,ensure_ascii=False))

if __name__=='__main__':main()
```

代码 B.5　公共模型、储能控制与结果导出（microgrid.py）

```
from __future__ import annotations
import argparse
import csv
import math
import re
import xml.etree.ElementTree as ET
import zipfile
from collections import defaultdict
from datetime import date, datetime, timedelta
from xml.sax.saxutils import escape
from dataclasses import dataclass, field, asdict
from pathlib import Path
import hashlib
import json
import time
import numpy as np
import pandas as pd
from scipy.optimize import Bounds, LinearConstraint, milp, linprog
from scipy.sparse import coo_matrix, lil_matrix
```

```

ROOT=Path(__file__).resolve().parents[1]

@dataclass(frozen=True)
class Battery:
    minimum: float = 1200
    maximum: float = 10800
    power_kw: float = 5000
    eta_charge: float = 0.9
    eta_discharge: float = 0.9
    dt: float = 1 / 6

def optimize(load, pv, price, initial=6000, terminal=6000, battery=Battery(), relax=False, time_limit=60, mip_gap=1e-08):
    (load, pv, price) = (np.asarray(x, dtype=float) for x in (load, pv, price))
    n = len(load)
    assert len(pv) == len(price) == n and n > 0
    assert np.all(np.isfinite(np.r_[load, pv, price]))
    assert np.all(load >= 0) and np.all(pv >= 0) and np.all(price > 0)
    (q, c, d, w, e, z) = (0, n, 2 * n, 3 * n, 4 * n, 5 * n + 1)
    size = 6 * n + 1
    obj = np.zeros(size)
    obj[q:q + n] = price
    (lb, ub) = (np.zeros(size), np.full(size, np.inf))
    cap = battery.power_kw * battery.dt
    ub[c:c + n] = ub[d:d + n] = cap
    (lb[e:e + n + 1], ub[e:e + n + 1]) = (battery.minimum, battery.maximum)
    lb[e] = ub[e] = initial
    if terminal is not None:
        lb[e + n] = ub[e + n] = terminal
    ub[z:z + n] = 1
    a = lil_matrix((4 * n, size))
    (low, high) = (np.full(4 * n, -np.inf), np.full(4 * n, np.inf))
    for t in range(n):
        (a[t, q + t], a[t, c + t], a[t, d + t], a[t, w + t]) = (1, -1, 1, -1)
        low[t] = high[t] = load[t] - pv[t]
        (a[n + t, e + t + 1], a[n + t, e + t]) = (1, -1)
        (a[n + t, c + t], a[n + t, d + t]) = (-battery.eta_charge, 1 / battery.eta_discharge)
        low[n + t] = high[n + t] = 0
        (a[2 * n + t, c + t], a[2 * n + t, z + t]) = (1, -cap)
        high[2 * n + t] = 0
        (a[3 * n + t, d + t], a[3 * n + t, z + t]) = (1, cap)
        high[3 * n + t] = cap
    integral = np.zeros(size)
    if not relax:
        integral[z:z + n] = 1
    tic = time.perf_counter()
    sol = milp(obj, integrality=integral, bounds=Bounds(lb, ub), constraints=LinearConstraint(a.tocsc(), low, high), options={'time_limit': time_limit, 'mip_rel_gap': mip_gap})
    elapsed = time.perf_counter() - tic
    if sol.x is None:
        raise RuntimeError(f'No feasible optimization result: {sol.message}')
    x = sol.x
    return dict(grid=x[q:q + n], charge=x[c:c + n], discharge=x[d:d + n], unused=x[w:w + n], soc=x[e:e + n + 1], solver=dict(status=int(sol.status), message=sol.message, objective=float(sol.fun), runtime_seconds=elapsed, mip_gap=getattr(sol, 'mip_gap', None), dual_bound=getattr(sol, 'mip_dual_bound', None), relax=relax))

@dataclass(frozen=True)
class ValueBattery:
    lo: float = 1200.0
    hi: float = 10800.0
    power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    target: float = 6000.0
```

```
81
82      @property
83      def limit(self):
84          return self.power / 6.0
85
86  def purchase_plan(scenarios, initial, battery=ValueBattery(), existing=None, refund=False,
        current_slots=144, terminal=6000.0):
87      (ns, nt, _) = scenarios.shape
88      net = (scenarios[:, :, 0] - scenarios[:, :, 1]) / 6.0
89      prices = scenarios[:, :, 2]
90      meanp = prices.mean(axis=0)
91      nbase = 4 * nt + ns * nt
92      nadj = 2 * current_slots if existing is not None and refund else 0
93      nv = nbase + nadj
94      objective = np.zeros(nv)
95      objective[:nt] = meanp
96      objective[nt:3 * nt] = 1e-08
97      objective[4 * nt:nbase] = (5 * prices / ns).ravel()
98      bounds = [(0.0, None)] * nv
99      bounds[nt:3 * nt] = [(0.0, battery.limit)] * (2 * nt)
100     bounds[3 * nt:4 * nt] = [(battery.lo, battery.hi)] * nt
101     bounds[4 * nt - 1] = (terminal, terminal)
102     if existing is not None:
103         old = np.asarray(existing[:current_slots])
104         if refund:
105             objective[:current_slots] = 0
106             objective[nbase:nbase + current_slots] = 1.5 * meanp[:current_slots]
107             objective[nbase + current_slots:] = -0.5 * meanp[:current_slots]
108             bounds[nbase + current_slots:] = [(0.0, float(v)) for v in old]
109         else:
110             objective[:current_slots] = 1.5 * meanp[:current_slots]
111             bounds[:current_slots] = [(float(v), None) for v in old]
112     (eqr, eqc, eqv) = ([], [], [])
113     rhs = np.zeros(nt + (current_slots if nadj else 0))
114     rhs[0] = initial
115     for t in range(nt):
116         for (col, val) in ((3 * nt + t, 1.0), (nt + t, -battery.eta_c), (2 * nt + t, 1 / battery.
        eta_d)):
117             eqr.append(t)
118             eqc.append(col)
119             eqv.append(val)
120         if t:
121             eqr.append(t)
122             eqc.append(3 * nt + t - 1)
123             eqv.append(-1.0)
124     if nadj:
125         rhs[nt:] = old
126         for t in range(current_slots):
127             for (col, val) in ((t, 1.0), (nbase + t, -1.0), (nbase + current_slots + t, 1.0)):
128                 eqr.append(nt + t)
129                 eqc.append(col)
130                 eqv.append(val)
131     rows = np.repeat(np.arange(ns * nt), 4)
132     times = np.tile(np.arange(nt), ns)
133     cols = np.column_stack([times, nt + times, 2 * nt + times, 4 * nt + np.arange(ns * nt)]).ravel
        ()
134     vals = np.tile([-1.0, 1.0, -1.0, -1.0], ns * nt)
135     sol = linprog(objective, A_ub=coo_matrix((vals, (rows, cols)), shape=(ns * nt, nv)).tocsr(),
        b_ub=-net.ravel(), A_eq=coo_matrix((eqv, (eqr, eqc)), shape=(len(rhs), nv)).tocsr(), b_eq=rhs,
        bounds=bounds, method='highs', options={'dual_feasibility_tolerance': 1e-07, '
        primal_feasibility_tolerance': 1e-07})
136     if not sol.success:
137         raise RuntimeError(f'Scenario LP failed: {sol.status} {sol.message}')
138     q = np.maximum(sol.x[:nt], 0)
139     c = np.maximum(sol.x[nt:2 * nt], 0)
140     discharge = np.maximum(sol.x[2 * nt:3 * nt], 0)
```

```
    rho = battery.eta_c * battery.eta_d
    eliminate = np.minimum(c, discharge / rho)
    c -= eliminate
    discharge -= rho * eliminate
    soc = np.r_[initial, sol.x[3 * nt:4 * nt]]
    return {'q': q, 'charge': c, 'discharge': discharge, 'soc': soc, 'objective': float(sol.fun), 'iterations': int(sol.nit), 'residual': float(np.max(np.abs(np.diff(soc) - battery.eta_c * c + discharge / battery.eta_d)))}

def daily_plan(corrected_kwh, margin_kwh, price, initial, battery=ValueBattery()):
    corrected = np.asarray(corrected_kwh, dtype=float)
    margin = np.asarray(margin_kwh, dtype=float)
    price = np.asarray(price, dtype=float)
    if corrected.shape != (144, 2) or margin.shape != (144,) or price.shape != (144,):
        raise ValueError('daily plan expects corrected (144,2), margin/price (144,)')
    if not np.isfinite(np.r_[corrected.ravel(), margin, price, initial]).all():
        raise ValueError('nonfinite daily planning input')
    if np.any(corrected < 0) or np.any(margin < 0) or np.any(price <= 0):
        raise ValueError('nonnegative forecast/margin and strictly positive tariffs required')
    scenario = np.column_stack([(corrected[:, 0] + margin) * 6.0, corrected[:, 1] * 6.0, price])[None]
    result = purchase_plan(scenario, float(initial), battery=battery, current_slots=144, terminal=battery.target)
    net = corrected[:, 0] + margin - corrected[:, 1]
    implied_emergency = np.maximum(net + result['charge'] - result['discharge'] - result['q'], 0.0)
    result['max_implied_planned_emergency_kwh'] = float(implied_emergency.max())
    if implied_emergency.max() > 1e-05:
        raise AssertionError('point-scenario planner used emergency electricity')
    if np.max(np.minimum(result['charge'], result['discharge'])) > 1e-07:
        raise AssertionError('planner charge/discharge recovery failed')
    return result

def greedy_step(plan, load, pv, soc, battery=ValueBattery()):
    if min(plan, load, pv) < -1e-07 or not battery.lo - 1e-06 <= soc <= battery.hi + 1e-06:
        raise ValueError('invalid physical input')
    balance = float(plan + pv - load)
    if balance >= 0:
        charge = min(balance, battery.limit, max(0.0, (battery.hi - soc) / battery.eta_c))
        discharge = emergency = 0.0
        surplus = balance - charge
    else:
        charge = surplus = 0.0
        discharge = min(-balance, battery.limit, max(0.0, (soc - battery.lo) * battery.eta_d))
        emergency = -balance - discharge
    following = soc + battery.eta_c * charge - discharge / battery.eta_d
    if charge > 1e-07 and emergency > 1e-07:
        raise AssertionError('emergency charging is forbidden')
    return (charge, discharge, following, emergency, surplus)

def close_plan(q, data, day, battery=ValueBattery(), close_slots=8):
    if day <= 0:
        raise ValueError('Closing plan requires past observed load')
    result = np.asarray(q, dtype=float).copy()
    ordinary_floor = float(np.max(data['actual_load'][:day]) / 6 + battery.limit)
    result[-close_slots:] = np.maximum(result[-close_slots:], ordinary_floor)
    return result

def execute_close(plan, load, pv, soc, reference=None, battery=ValueBattery(), remaining=None):
    if soc < battery.target:
        charge = min(battery.limit, (battery.target - soc) / battery.eta_c, max(plan + pv - load, 0.0))
        discharge = 0.0
    else:
        charge = 0.0
        discharge = min(battery.limit, (soc - battery.target) * battery.eta_d, max(load, 0.0))
```

```
    nxt = soc + battery.eta_c * charge - discharge / battery.eta_d
    deficit = load + charge - pv - plan - discharge
    return (charge, discharge, nxt, max(deficit, 0.0), max(-deficit, 0.0))

def ordinary_cost(original, adjusted, price):
    (original, adjusted, price) = np.broadcast_arrays(np.asarray(original, dtype=float), np.asarray(adjusted, dtype=float), np.asarray(price, dtype=float))
    if not np.isfinite(np.r_[original.ravel(), adjusted.ravel(), price.ravel()]).all():
        raise ValueError('Nonfinite contract input')
    if (original < 0).any() or (adjusted < 0).any() or (price <= 0).any():
        raise ValueError('Contract quantities must be nonnegative and prices positive')
    return price * adjusted + 0.5 * price * np.abs(adjusted - original)

@dataclass
class AdjustmentLedger:
    original: np.ndarray
    reference_mode: str
    events: list = field(default_factory=list)

    def __post_init__(self):
        if self.reference_mode not in ('initial_final', 'sequential'):
            raise ValueError('Choose initial_final or sequential explicitly')
        self.original = np.asarray(self.original, dtype=float).copy()
        if not np.isfinite(self.original).all() or (self.original < 0).any():
            raise ValueError('Invalid original contract')
        self.contract = self.original.copy()

    def revise(self, target, effective_from, issue_time):
        target = np.asarray(target, dtype=float)
        if target.shape != self.contract.shape or not np.isfinite(target).all() or (target < -1e-07).any():
            raise ValueError('Invalid revised contract')
        if not np.allclose(target[:effective_from], self.contract[:effective_from], atol=1e-07, rtol=0):
            raise ValueError('Cannot change delivered intervals')
        target = np.maximum(target, 0)
        delta = target - self.contract
        if np.max(np.abs(delta)) <= 1e-07:
            return False
        self.events.append(dict(issue_time=str(issue_time), effective_from=effective_from, before=self.contract.copy(), after=target.copy(), increase=np.maximum(delta, 0), decrease=np.maximum(-delta, 0)))
        self.contract = target
        return True

    def settle(self, price, emergency):
        price = np.asarray(price, dtype=float)
        gross_increase = sum((e['increase'] for e in self.events), np.zeros_like(self.original))
        gross_decrease = sum((e['decrease'] for e in self.events), np.zeros_like(self.original))
        if not np.allclose(self.original + gross_increase - gross_decrease, self.contract, atol=1e-06, rtol=0):
            raise AssertionError('Events do not reconstruct contract')
        if self.reference_mode == 'initial_final':
            increase = np.maximum(self.contract - self.original, 0)
            decrease = np.maximum(self.original - self.contract, 0)
        else:
            (increase, decrease) = (gross_increase, gross_decrease)
        original_fee = price * self.original
        (increase_fee, decrease_penalty, refund) = (1.5 * price * increase, 0.5 * price * decrease, price * decrease)
        emergency_fee = 5 * price * np.asarray(emergency)
        total = original_fee + increase_fee + decrease_penalty - refund + emergency_fee
        if self.reference_mode == 'initial_final' and (not np.allclose(total, ordinary_cost(self.original, self.contract, price) + emergency_fee, atol=1e-06, rtol=0)):
            raise AssertionError('Final-quantity settlement does not match piecewise price')
        return dict(original_fee=original_fee, increase_fee=increase_fee, decrease_penalty=
```

```
    decrease_penalty, refund=refund, emergency_fee=emergency_fee, total_fee=total,
    contract_increase=increase, contract_decrease=decrease, gross_contract_increase=gross_increase,
     gross_contract_decrease=gross_decrease)

def replan(load, pv, price, reference, initial, terminal, battery=Battery(), terminal_weight=1.0,
    minimum_contract=None, terminal_curve=None, fixed_contract=False, relaxation_only=False):
    (load, pv, price, reference) = [np.asarray(x, dtype=float) for x in (load, pv, price,
    reference)]
    n = len(load)
    if n == 0 or any((len(x) != n for x in (pv, price, reference))) or (not np.isfinite(np.r_[load
    , pv, price, reference]).all()):
        raise ValueError('Invalid planning vectors')
    if min(load.min(), pv.min(), reference.min()) < -1e-07 or price.min() <= 0:
        raise ValueError('Invalid sign in planning input')
    (inc, dec, u, c, d, w, e, z, slack) = (0, n, 2 * n, 3 * n, 4 * n, 5 * n, 6 * n, 7 * n + 1, 8 *
     n + 1)
    size = slack + 1
    objective = np.zeros(size)
    objective[inc:inc + n] = 1.5 * price
    objective[dec:dec + n] = -0.5 * price
    objective[u:u + n] = 5 * price
    objective[slack] = terminal_weight * price.min() / battery.eta_charge
    slopes = intercepts = None
    if terminal_curve is not None:
        (knots, values) = [np.asarray(v, dtype=float) for v in terminal_curve]
        if len(knots) < 2 or values.shape != knots.shape or (not np.isfinite(np.r_[knots, values])
    .all()):
            raise ValueError('Invalid terminal curve')
        if np.any(np.diff(knots) <= 0) or knots[0] > battery.minimum or knots[-1] < battery.
    maximum:
            raise ValueError('Terminal curve must cover the battery interval')
        slopes = np.diff(values) / np.diff(knots)
        if np.min(np.diff(slopes), initial=0.0) < -1e-07 or slopes.max() > 1e-07:
            raise ValueError('Terminal curve must be convex and nonincreasing')
        intercepts = values[:-1] - slopes * knots[:-1]
        objective[slack] = 1.0
    (lower, upper) = (np.zeros(size), np.full(size, np.inf))
    cap = battery.power_kw * battery.dt
    upper[dec:dec + n] = reference
    if fixed_contract:
        upper[inc:inc + n] = upper[dec:dec + n] = 0.0
    if terminal_curve is not None:
        lower[slack] = -np.inf
    upper[u:u + n] = load
    upper[c:c + n] = upper[d:d + n] = cap
    (lower[e:e + n + 1], upper[e:e + n + 1]) = (battery.minimum, battery.maximum)
    lower[e] = upper[e] = initial
    upper[z:z + n] = 1
    extra = n if minimum_contract is not None else 0
    terminal_rows = 1 if slopes is None else len(slopes)
    a = lil_matrix((5 * n + terminal_rows + extra, size))
    (lo, hi) = (np.full(a.shape[0], -np.inf), np.full(a.shape[0], np.inf))
    for t in range(n):
        for (col, value) in [(inc + t, 1), (dec + t, -1), (u + t, 1), (c + t, -1), (d + t, 1), (w
    + t, -1)]:
            a[t, col] = value
        lo[t] = hi[t] = load[t] - pv[t] - reference[t]
        for (col, value) in [(e + t + 1, 1), (e + t, -1), (c + t, -battery.eta_charge), (d + t, 1
    / battery.eta_discharge)]:
            a[n + t, col] = value
        lo[n + t] = hi[n + t] = 0
        (a[2 * n + t, c + t], a[2 * n + t, z + t]) = (1, -cap)
        hi[2 * n + t] = 0
        (a[3 * n + t, d + t], a[3 * n + t, z + t]) = (1, cap)
        hi[3 * n + t] = cap
        (a[4 * n + t, u + t], a[4 * n + t, z + t]) = (1, load[t])
```

```
        hi[4 * n + t] = load[t]
    if slopes is None:
        (a[5 * n, e + n], a[5 * n, slack]) = (1, 1)
        lo[5 * n] = terminal
    else:
        for (row, (slope, intercept)) in enumerate(zip(slopes, intercepts), 5 * n):
            (a[row, e + n], a[row, slack]) = (-slope, 1)
            lo[row] = intercept
    if minimum_contract is not None:
        for t in range(n):
            (a[5 * n + terminal_rows + t, inc + t], a[5 * n + terminal_rows + t, dec + t]) = (1,
-1)
            lo[5 * n + terminal_rows + t] = minimum_contract[t] - reference[t]
    integrality = np.zeros(size)
    tic = time.perf_counter()
    result = milp(objective, integrality=integrality, bounds=Bounds(lower, upper), constraints=
LinearConstraint(a.tocsc(), lo, hi), options={'time_limit': 30, 'mip_rel_gap': 1e-08})
    if result.x is None or result.status != 0:
        raise RuntimeError('Replan was not solved to optimality: ' + result.message)
    x = result.x
    if not relaxation_only and max(np.minimum(x[c:c + n], x[d:d + n]).max(), np.minimum(x[c:c + n
], x[u:u + n]).max()) > 1e-07:
        integrality[z:z + n] = 1
        result = milp(objective, integrality=integrality, bounds=Bounds(lower, upper), constraints
=LinearConstraint(a.tocsc(), lo, hi), options={'time_limit': 30, 'mip_rel_gap': 1e-08})
        if result.x is None or result.status != 0:
            raise RuntimeError('Integer replan was not solved to optimality: ' + result.message)
        x = result.x
    grid = np.maximum(reference + x[inc:inc + n] - x[dec:dec + n], 0)
    residual = max(abs(grid + pv + x[d:d + n] + x[u:u + n] - load - x[c:c + n] - x[w:w + n]).max()
, np.minimum(x[c:c + n], x[d:d + n]).max(), np.minimum(x[c:c + n], x[u:u + n]).max())
    if residual > 1e-06 and (not relaxation_only):
        raise AssertionError('Replan physical check failed')
    return dict(grid=grid, soc=x[e:e + n + 1], charge=x[c:c + n], discharge=x[d:d + n], solver=
dict(status=int(result.status), objective=float(result.fun), relax=not integrality.any(),
runtime_seconds=time.perf_counter() - tic, mip_gap=getattr(result, 'mip_gap', None),
maximum_residual_kwh=float(residual)))

@dataclass(frozen=True)
class ValuePolicy:
    grid: np.ndarray
    values: np.ndarray
    decision_price: np.ndarray
    start_slot: int
    tie_break: str = 'more_discharge'
    value_discount: float = 1.0

    def next_values(self, absolute_slot: int) -> np.ndarray:
        row = absolute_slot - self.start_slot + 1
        if row < 1 or row >= len(self.values):
            raise IndexError('slot outside value-table interval')
        return self.values[row] if self.value_discount == 1.0 else self.value_discount * self.
values[row]

def _as_kernels(residual_kernels, nt: int) -> list[np.ndarray]:
    if isinstance(residual_kernels, np.ndarray) and residual_kernels.ndim == 2:
        source = [residual_kernels[t] for t in range(nt)]
    else:
        source = list(residual_kernels)
    if len(source) != nt:
        raise ValueError('one residual kernel is required for every slot')
    result = []
    for samples in source:
        a = np.asarray(samples, dtype=float).reshape(-1)
        if not len(a) or not np.isfinite(a).all():
            raise ValueError('residual kernels must be finite and non-empty')
```

```
        result.append(a)
    return result

def make_soc_grid(battery: ValueBattery=ValueBattery(), step_kwh: float=50.0) -> np.ndarray:
    if not np.isfinite(step_kwh) or step_kwh <= 0:
        raise ValueError('positive finite SOC grid step required')
    span = battery.hi - battery.lo
    count = int(round(span / step_kwh))
    if abs(count * step_kwh - span) > 1e-09:
        raise ValueError('SOC step must divide the usable battery interval')
    return np.linspace(battery.lo, battery.hi, count + 1)

def _convex_nonincreasing_diagnostics(grid: np.ndarray, values: np.ndarray) -> dict:
    first = np.diff(values) / np.diff(grid)
    second = np.diff(first)
    return {'max_monotonicity_violation': float(max(0.0, np.max(first, initial=-np.inf))), '
   max_convexity_violation': float(max(0.0, -np.min(second, initial=np.inf)))}

def reachable_minimum(grid, objective, lower, upper):
    (lower, upper) = np.broadcast_arrays(lower, upper)
    choices = np.concatenate([lower[..., None], upper[..., None], np.broadcast_to(grid, lower.
   shape + (len(grid),))], axis=-1)
    scores = np.interp(choices, grid, objective)
    scores = np.where((choices >= lower[..., None] - 1e-10) & (choices <= upper[..., None] + 1e
   -10), scores, np.inf)
    best = scores.min(axis=-1, keepdims=True)
    return np.min(np.where(scores <= best + 1e-10, choices, np.inf), axis=-1)

def build_rolling_value_table(effective_q_kwh, forecast_net_kwh, residual_kernels, decision_price,
    *, battery: ValueBattery=ValueBattery(), step_kwh: float=50.0, terminal_kappa: float=0.0,
   terminal_target_kwh: float=6000.0, start_slot: int=0, terminal_values=None, vectorized: bool=
   False, value_discount: float=1.0) -> tuple[ValuePolicy, dict]:
    q = np.asarray(effective_q_kwh, dtype=float).reshape(-1)
    if not 0.0 < value_discount <= 1.0:
        raise ValueError('Value discount must be in (0,1]')
    net = np.asarray(forecast_net_kwh, dtype=float).reshape(-1)
    price = np.asarray(decision_price, dtype=float).reshape(-1)
    nt = len(q)
    if len(net) != nt or len(price) != nt or nt == 0:
        raise ValueError('q, forecast net load and decision price must align')
    if not np.isfinite(np.r_[q, net, price]).all() or np.any(q < -1e-09) or np.any(price < 0):
        raise ValueError('invalid value-DP input')
    kernels = _as_kernels(residual_kernels, nt)
    grid = make_soc_grid(battery, step_kwh)
    values = np.zeros((nt + 1, len(grid)), dtype=float)
    values[-1] = float(terminal_kappa) * np.maximum(float(terminal_target_kwh) - grid, 0.0)
    if terminal_values is not None:
        supplied = np.asarray(terminal_values, dtype=float)
        if supplied.shape != grid.shape or not np.isfinite(supplied).all():
            raise ValueError('Terminal values must match the SOC grid')
        if np.max(np.diff(supplied), initial=0.0) > 1e-07:
            raise ValueError('Terminal values must be nonincreasing')
        values[-1] = supplied
    if terminal_kappa < 0:
        raise ValueError('terminal continuation coefficient must be nonnegative')
    max_mono = max_convex = 0.0
    targets = np.empty(nt, dtype=float)
    for t in range(nt - 1, -1, -1):
        continuation = values[t + 1] if value_discount == 1.0 else value_discount * values[t + 1]
        shape = _convex_nonincreasing_diagnostics(grid, continuation)
        convex = shape['max_convexity_violation'] <= 1e-07
        objective = continuation + 5.0 * price[t] * battery.eta_d * grid
        best = np.flatnonzero(objective <= objective.min() + 1e-10)[0]
        target = grid[best]
        targets[t] = target
        sample_values = np.empty((len(kernels[t]), len(grid)), dtype=float)
```

```
        samples = [] if vectorized and convex else kernels[t]
        if vectorized and convex:
            balance = q[t] - (net[t] + kernels[t][:, None])
            charge = np.minimum(np.maximum(balance, 0.0), battery.limit)
            charged = np.minimum(battery.hi, grid + battery.eta_c * charge)
            deficit = np.maximum(-balance, 0.0)
            lower = np.maximum(battery.lo, grid - np.minimum(deficit, battery.limit) / battery.eta_d)
            nxt = np.minimum(grid, np.maximum(lower, target))
            discharged = battery.eta_d * (grid - nxt)
            sample_values = np.where(balance >= 0, np.interp(charged, grid, continuation), 5.0 * price[t] * (deficit - discharged) + np.interp(nxt, grid, continuation))
        for (j, error) in enumerate(samples):
            balance = q[t] - (net[t] + error)
            if balance >= 0:
                charge = min(balance, battery.limit)
                nxt = np.minimum(battery.hi, grid + battery.eta_c * charge)
                sample_values[j] = np.interp(nxt, grid, continuation)
            else:
                deficit = -balance
                lower = np.maximum(battery.lo, grid - min(deficit, battery.limit) / battery.eta_d)
                nxt = np.minimum(grid, np.maximum(lower, target))
                if not convex:
                    nxt = reachable_minimum(grid, objective, lower, grid)
                discharge = battery.eta_d * (grid - nxt)
                emergency = deficit - discharge
                sample_values[j] = 5.0 * price[t] * emergency + np.interp(nxt, grid, continuation)
        values[t] = sample_values.mean(axis=0)
        diag = _convex_nonincreasing_diagnostics(grid, values[t])
        max_mono = max(max_mono, diag['max_monotonicity_violation'])
        max_convex = max(max_convex, diag['max_convexity_violation'])
    policy = ValuePolicy(grid=grid, values=values, decision_price=price, start_slot=int(start_slot), value_discount=value_discount)
    return (policy, {'soc_grid_step_kwh': float(step_kwh), 'soc_grid_points': int(len(grid)), 'time_slots': int(nt), 'residual_samples_min': int(min(map(len, kernels))), 'residual_samples_max': int(max(map(len, kernels))), 'terminal_kappa_yuan_per_kwh': float(terminal_kappa), 'terminal_curve_supplied': terminal_values is not None, 'value_discount': value_discount, 'max_monotonicity_violation': float(max_mono), 'max_convexity_violation': float(max_convex), 'reserve_target_min_kwh': float(targets.min()), 'reserve_target_max_kwh': float(targets.max())})

def execute_rolling_value_policy(q_kwh: float, actual_load_kwh: float, actual_pv_kwh: float, actual_soc_kwh: float, decision_price: float, next_value_grid, grid, *, battery: ValueBattery=ValueBattery(), general_search: bool=False) -> tuple[dict, dict]:
    (q, load, pv, soc, price) = map(float, (q_kwh, actual_load_kwh, actual_pv_kwh, actual_soc_kwh, decision_price))
    continuation = np.asarray(next_value_grid, dtype=float)
    grid = np.asarray(grid, dtype=float)
    if min(q, load, pv, price) < -1e-09 or not battery.lo - 1e-07 <= soc <= battery.hi + 1e-07:
        raise ValueError('invalid physical execution input')
    if continuation.shape != grid.shape or not np.isfinite(continuation).all():
        raise ValueError('invalid continuation value')
    balance = q + pv - load
    held_value = float(np.interp(soc, grid, continuation))
    if balance >= 0:
        charge = min(balance, battery.limit, max(0.0, (battery.hi - soc) / battery.eta_c))
        discharge = emergency = 0.0
        nxt = soc + battery.eta_c * charge
        surplus = balance - charge
        target = soc
    else:
        deficit = -balance
        objective = continuation + 5.0 * price * battery.eta_d * grid
        target = float(grid[np.flatnonzero(objective <= objective.min() + 1e-10)[0]])
        lower = max(battery.lo, soc - min(deficit, battery.limit) / battery.eta_d)
        nxt = min(soc, max(lower, target))
```

```
        if general_search:
            nxt = float(reachable_minimum(grid, objective, lower, soc))
        discharge = battery.eta_d * (soc - nxt)
        emergency = max(0.0, deficit - discharge)
        charge = surplus = 0.0
    result = {'charge': float(charge), 'discharge': float(discharge), 'soc_end': float(nxt), '
    emergency': float(emergency), 'surplus': float(surplus)}
    explanation = {'reserve_target_kwh': float(target), 'actual_gap_kwh': float(max(-balance, 0.0)
    ), 'immediate_saving_yuan': float(5.0 * price * discharge), 'continuation_value_before_yuan':
    held_value, 'continuation_value_after_yuan': float(np.interp(nxt, grid, continuation)), '
    economic_hold_with_emergency': bool(emergency > 1e-07 and discharge + 1e-07 < min(max(-balance,
     0.0), battery.limit, max(0.0, (soc - battery.lo) * battery.eta_d)))}
    if charge > 1e-07 and emergency > 1e-07:
        raise AssertionError('emergency electricity may not charge the battery')
    return (result, explanation)

@dataclass(frozen=True)
class MidnightValuePolicy:
    grid: np.ndarray
    values: np.ndarray
    decision_price: np.ndarray
    start_slot: int
    tie_break: str = 'more_discharge'

    def next_values(self, absolute_slot: int) -> np.ndarray:
        row = absolute_slot - self.start_slot + 1
        if row < 1 or row >= len(self.values):
            raise IndexError('slot outside value-table interval')
        return self.values[row]

def build_midnight_value_table(effective_q_kwh, forecast_net_kwh, residual_kernels, decision_price
    , *, battery: ValueBattery=ValueBattery(), step_kwh: float=50.0, terminal_kappa: float=0.0,
    terminal_target_kwh: float=6000.0, start_slot: int=0) -> tuple[MidnightValuePolicy, dict]:
    q = np.asarray(effective_q_kwh, dtype=float).reshape(-1)
    net = np.asarray(forecast_net_kwh, dtype=float).reshape(-1)
    price = np.asarray(decision_price, dtype=float).reshape(-1)
    nt = len(q)
    if len(net) != nt or len(price) != nt or nt == 0:
        raise ValueError('q, forecast net load and decision price must align')
    if not np.isfinite(np.r_[q, net, price]).all() or np.any(q < -1e-09) or np.any(price < 0):
        raise ValueError('invalid value-DP input')
    kernels = _as_kernels(residual_kernels, nt)
    grid = make_soc_grid(battery, step_kwh)
    values = np.zeros((nt + 1, len(grid)), dtype=float)
    values[-1] = float(terminal_kappa) * np.maximum(float(terminal_target_kwh) - grid, 0.0)
    if terminal_kappa < 0:
        raise ValueError('terminal continuation coefficient must be nonnegative')
    max_mono = max_convex = 0.0
    targets = np.empty(nt, dtype=float)
    for t in range(nt - 1, -1, -1):
        continuation = values[t + 1]
        objective = continuation + 5.0 * price[t] * battery.eta_d * grid
        best = np.flatnonzero(objective <= objective.min() + 1e-10)[0]
        target = grid[best]
        targets[t] = target
        sample_values = np.empty((len(kernels[t]), len(grid)), dtype=float)
        for (j, error) in enumerate(kernels[t]):
            balance = q[t] - (net[t] + error)
            if balance >= 0:
                charge = min(balance, battery.limit)
                nxt = np.minimum(battery.hi, grid + battery.eta_c * charge)
                sample_values[j] = np.interp(nxt, grid, continuation)
            else:
                deficit = -balance
                lower = np.maximum(battery.lo, grid - min(deficit, battery.limit) / battery.eta_d)
                nxt = np.minimum(grid, np.maximum(lower, target))
```

```
                discharge = battery.eta_d * (grid - nxt)
                emergency = deficit - discharge
                sample_values[j] = 5.0 * price[t] * emergency + np.interp(nxt, grid, continuation)
        values[t] = sample_values.mean(axis=0)
        diag = _convex_nonincreasing_diagnostics(grid, values[t])
        max_mono = max(max_mono, diag['max_monotonicity_violation'])
        max_convex = max(max_convex, diag['max_convexity_violation'])
    policy = MidnightValuePolicy(grid=grid, values=values, decision_price=price, start_slot=int(start_slot))
    return (policy, {'soc_grid_step_kwh': float(step_kwh), 'soc_grid_points': int(len(grid)), 'time_slots': int(nt), 'residual_samples_min': int(min(map(len, kernels))), 'residual_samples_max': int(max(map(len, kernels))), 'terminal_kappa_yuan_per_kwh': float(terminal_kappa), 'max_monotonicity_violation': float(max_mono), 'max_convexity_violation': float(max_convex), 'reserve_target_min_kwh': float(targets.min()), 'reserve_target_max_kwh': float(targets.max())})

def execute_midnight_value_policy(q_kwh: float, actual_load_kwh: float, actual_pv_kwh: float, actual_soc_kwh: float, decision_price: float, next_value_grid, grid, *, battery: ValueBattery=ValueBattery()) -> tuple[dict, dict]:
    (q, load, pv, soc, price) = map(float, (q_kwh, actual_load_kwh, actual_pv_kwh, actual_soc_kwh, decision_price))
    continuation = np.asarray(next_value_grid, dtype=float)
    grid = np.asarray(grid, dtype=float)
    if min(q, load, pv, price) < -1e-09 or not battery.lo - 1e-07 <= soc <= battery.hi + 1e-07:
        raise ValueError('invalid physical execution input')
    if continuation.shape != grid.shape or not np.isfinite(continuation).all():
        raise ValueError('invalid continuation value')
    balance = q + pv - load
    held_value = float(np.interp(soc, grid, continuation))
    if balance >= 0:
        charge = min(balance, battery.limit, max(0.0, (battery.hi - soc) / battery.eta_c))
        discharge = emergency = 0.0
        nxt = soc + battery.eta_c * charge
        surplus = balance - charge
        target = soc
    else:
        deficit = -balance
        objective = continuation + 5.0 * price * battery.eta_d * grid
        target = float(grid[np.flatnonzero(objective <= objective.min() + 1e-10)[0]])
        lower = max(battery.lo, soc - min(deficit, battery.limit) / battery.eta_d)
        nxt = min(soc, max(lower, target))
        discharge = battery.eta_d * (soc - nxt)
        emergency = max(0.0, deficit - discharge)
        charge = surplus = 0.0
    result = {'charge': float(charge), 'discharge': float(discharge), 'soc_end': float(nxt), 'emergency': float(emergency), 'surplus': float(surplus)}
    explanation = {'reserve_target_kwh': float(target), 'actual_gap_kwh': float(max(-balance, 0.0)), 'immediate_saving_yuan': float(5.0 * price * discharge), 'continuation_value_before_yuan': held_value, 'continuation_value_after_yuan': float(np.interp(nxt, grid, continuation)), 'economic_hold_with_emergency': bool(emergency > 1e-07 and discharge + 1e-07 < min(max(-balance, 0.0), battery.limit, max(0.0, (soc - battery.lo) * battery.eta_d)))}
    if charge > 1e-07 and emergency > 1e-07:
        raise AssertionError('emergency electricity may not charge the battery')
    return (result, explanation)

def verify_intervals(frame, battery):
    b = battery
    bus = frame.contract_grid + frame.pv_kwh + frame.discharge + frame.emergency - frame.load_kwh - frame.charge - frame.unused
    dynamics = frame.soc_end - frame.soc_start - b.eta_charge * frame.charge + frame.discharge / b.eta_discharge
    total = frame.original_fee + frame.increase_fee + frame.decrease_penalty - frame.refund + frame.emergency_fee
    metrics = {'bus_residual_kwh': float(abs(bus).max()), 'soc_recursion_residual_kwh': float(abs(dynamics).max()), 'soc_lower_violation_kwh': float(max(0.0, b.minimum - frame.soc_end.min())), 'soc_upper_violation_kwh': float(max(0.0, frame.soc_end.max() - b.maximum)), '
```

```
    power_violation_kwh': float(max(0.0, frame.charge.max() - b.power_kw * b.dt, frame.discharge.
    max() - b.power_kw * b.dt)), 'simultaneous_charge_discharge_kwh': float(np.minimum(frame.charge
    , frame.discharge).max()), 'emergency_and_charge_overlap_kwh': float(np.minimum(frame.emergency
    , frame.charge).max()), 'fee_reconciliation_yuan': float(abs(total - frame.total_fee).max()), '
    unused_source_reconciliation_kwh': float(abs(frame.unused - frame.unused_pv - frame.
    unused_purchased).max()), 'continuous_state_residual_kwh': float(np.max(abs(frame.soc_start.
    to_numpy()[1:] - frame.soc_end.to_numpy()[:-1]))), 'nonnegative_violation_kwh': float(max(0, -
    frame[['planned_grid', 'contract_grid', 'charge', 'discharge', 'emergency', 'unused']].min().
    min()))}
591     metrics['passed'] = all((v < 1e-06 for v in metrics.values()))
592     if not metrics['passed']:
593         raise AssertionError(metrics)
594     return metrics
595 
596 def aggregate(frame, battery):
597     amount_cols = ['planned_grid', 'contract_grid', 'charge', 'discharge', 'emergency', 'unused',
    'unused_pv', 'unused_purchased', 'original_fee', 'increase_fee', 'decrease_penalty', 'refund',
    'emergency_fee', 'total_fee', 'contract_increase', 'contract_decrease']
598     daily = frame.groupby('date')[amount_cols].sum()
599     daily['initial_soc'] = frame.groupby('date').soc_start.first()
600     daily['terminal_soc'] = frame.groupby('date').soc_end.last()
601     active = frame.emergency > 1e-07
602     starts = active & (~active.shift(fill_value=False) | frame.date.ne(frame.date.shift()))
603     daily['emergency_events'] = starts.groupby(frame.date).sum()
604     daily['emergency_intervals'] = active.groupby(frame.date).sum()
605     monthly = daily[amount_cols + ['emergency_events', 'emergency_intervals']].groupby(daily.index
    .str[:7]).sum()
606     monthly.index.name = 'month'
607     tail = daily.total_fee.quantile(0.95)
608     summary = {k: float(frame[k].sum()) for k in amount_cols}
609     summary.update({'days': len(daily), 'intervals': len(frame), 'start_date': frame.date.iloc[0],
     'end_date': frame.date.iloc[-1], 'initial_soc_kwh': float(frame.soc_start.iloc[0]), '
    terminal_soc_kwh': float(frame.soc_end.iloc[-1]), 'daily_cost_p95_yuan': float(tail), '
    daily_cost_tail_mean_yuan': float(daily.loc[daily.total_fee >= tail, 'total_fee'].mean()), '
    emergency_events': int(daily.emergency_events.sum()), 'soc_min_kwh': float(min(frame.soc_start.
    min(), frame.soc_end.min())), 'soc_max_kwh': float(max(frame.soc_start.max(), frame.soc_end.max
    ())), 'terminal_energy_delta_kwh': float(frame.soc_end.iloc[-1] - frame.soc_start.iloc[0]), '
    terminal_credit_sensitivity_yuan': float((frame.soc_end.iloc[-1] - frame.soc_start.iloc[0]) *
    frame.plan_price.iloc[-144:].min() / battery.eta_charge)})
610     summary['total_fee_less_terminal_credit_sensitivity'] = summary['total_fee'] - summary['
    terminal_credit_sensitivity_yuan']
611     summary['verification'] = verify_intervals(frame, battery)
612     return (daily, monthly, summary)
613 
614 def check_dispatch(load, pv, price, result, eta_charge=0.9, eta_discharge=0.9, minimum=1200,
    maximum=10800, limit=5000 / 6, cyclic=True):
615     (q, c, d, w, e) = [np.asarray(result[x]) for x in ['grid', 'charge', 'discharge', 'unused', '
    soc']]
616     metrics = dict(bus_residual_kwh=float(np.max(np.abs(q + pv + d - load - c - w))),
    soc_residual_kwh=float(np.max(np.abs(np.diff(e) - eta_charge * c + d / eta_discharge))),
    lower_bound_violation_kwh=float(max(0, minimum - e.min())), upper_bound_violation_kwh=float(max
    (0, e.max() - maximum)), power_violation_kwh=float(max(0, max(c.max(), d.max()) - limit)),
    nonnegative_violation_kwh=float(max(0, -min(q.min(), c.min(), d.min(), w.min()))),
    simultaneous_charge_discharge_kwh=float(np.max(np.minimum(c, d))), terminal_difference_kwh=
    float(abs(e[-1] - e[0])) if cyclic else 0, objective_recomputed_yuan=float(np.dot(price, q)),
    objective_difference_yuan=float(abs(np.dot(price, q) - result['solver']['objective'])), finite=
    bool(np.isfinite(np.r_[q, c, d, w, e]).all()))
617     metrics['passed'] = metrics['finite'] and all((v < 1e-06 for (k, v) in metrics.items() if k
    not in ['objective_recomputed_yuan', 'finite']))
618     return metrics
619 
620 def align_actuals(frame, dates):
621     keys = ['date', 'slot']
622     columns = ['load_kw', 'pv_kw', 'price_yuan_kwh']
623     f = frame.copy()
624     f['date'] = pd.to_datetime(f['date'], errors='raise').dt.strftime('%Y-%m-%d')
```

```
    raw = pd.to_numeric(f['slot'], errors='raise')
    if not np.all(raw == np.floor(raw)):
        raise ValueError('Noninteger slot')
    f['slot'] = raw.astype(int)
    if f.duplicated(keys).any():
        raise ValueError('Duplicate date/slot keys')
    expected = pd.MultiIndex.from_product([list(dates), range(144)], names=keys)
    actual = pd.MultiIndex.from_frame(f[keys])
    if len(expected.difference(actual)) or len(actual.difference(expected)):
        raise ValueError('Missing or unexpected date/slot keys')
    f = f.set_index(keys).reindex(expected)
    values = f[columns].to_numpy(float)
    if not np.isfinite(values).all():
        raise ValueError('Nonfinite actual data')
    return values.reshape(len(dates), 144, 3)

def load_verified_input(npz='data/processed/annual_inputs.npz', csv='data/processed/actuals_power.csv'):
    with np.load(npz, allow_pickle=False) as source:
        data = {name: source[name].copy() for name in source.files}
    dates = data['dates'].astype(str)
    if len(np.unique(dates)) != len(dates) or not np.all(dates[1:] > dates[:-1]):
        raise ValueError('NPZ dates must be unique chronological keys')
    aligned = align_actuals(pd.read_csv(csv), dates)
    for (i, name) in enumerate(('actual_load', 'actual_pv', 'actual_price')):
        if not np.allclose(aligned[:, :, i], data[name], atol=1e-09, rtol=0):
            raise ValueError(f'Keyed CSV/NPZ mismatch: {name}')
        data[name] = aligned[:, :, i].copy()
    return data


RESULTS = ROOT / 'results/c2026'
WORK = ROOT / 'results/c2026/rebuild'
NS = {'s': 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
FILES = ['result1.xlsx', 'result2.xlsx', 'result3.xlsx', 'result4-2.xlsx', 'result4-3.xlsx']
KEYS = ['q1', 'q2', 'q3', 'q4_2', 'q4_3']
_FILE_BY_CASE = dict(zip(KEYS, FILES))
EMERGENCY_TOL_KWH = 1e-07
SUMCOLS = ['plan', 'adjusted', 'up', 'down', 'emergency', 'charge', 'discharge', 'surplus', 'cost_plan', 'cost_up', 'cost_down', 'cost_emergency', 'cost_total']

def canonical_frame(frame):
    if 'slot' in frame:
        return frame.copy()
    mapping = {'date': 'date', 'slot': 'interval_index', 'load': 'load_kwh', 'pv': 'pv_kwh', 'price': 'price', 'decision_price': 'plan_price', 'plan': 'planned_grid', 'adjusted': 'contract_grid', 'up': 'contract_increase', 'down': 'contract_decrease', 'emergency': 'emergency', 'charge': 'charge', 'discharge': 'discharge', 'soc_start': 'soc_start', 'soc_end': 'soc_end', 'surplus': 'unused', 'cost_plan': 'original_fee', 'cost_up': 'increase_fee', 'cost_emergency': 'emergency_fee', 'cost_total': 'total_fee'}
    result = pd.DataFrame({key: frame[value] for (key, value) in mapping.items()})
    result['cost_down'] = frame.decrease_penalty - frame.refund
    return result

def verify_frame(frame):
    balance = frame.adjusted + frame.pv + frame.discharge + frame.emergency - frame.load - frame.charge - frame.surplus
    state = frame.soc_end - frame.soc_start - 0.9 * frame.charge + frame.discharge / 0.9
    cost = frame.cost_plan + frame.cost_up + frame.cost_down + frame.cost_emergency - frame.cost_total
    checks = dict(balance=float(np.abs(balance).max()), state=float(np.abs(state).max()), cost=float(np.abs(cost).max()), lower=max(0.0, float(1200 - frame.soc_end.min())), upper=max(0.0, float(frame.soc_end.max() - 10800)), power=max(0.0, float(frame.charge.max() - 5000 / 6), float(frame.discharge.max() - 5000 / 6)), simultaneous=float(np.minimum(frame.charge, frame.discharge).max()), emergency_charging=float(np.minimum(frame.charge, frame.emergency).max()), continuity=float(np.max(np.abs(frame.soc_start.to_numpy()[1:] - frame.soc_end.to_numpy()[:-1])))
```

```
    ))
    assert all((abs(x) < 1e-06 for x in checks.values())), checks
    return checks

def export_annual(frame, output, parameters, summary=None):
    output = Path(output)
    if output.exists():
        raise FileExistsError(output)
    slots = canonical_frame(frame)
    checks = verify_frame(slots)
    slots = slots[slots.date >= '2025-02-01'].reset_index(drop=True)
    daily = slots.groupby('date', sort=True)[SUMCOLS].sum()
    daily['soc_start'] = slots.groupby('date').soc_start.first()
    daily['soc_end'] = slots.groupby('date').soc_end.last()
    result = dict(summary or {})
    result.update({key: float(slots[key].sum()) for key in SUMCOLS})
    result.update(reported_start=str(slots.date.iloc[0]), reported_end=str(slots.date.iloc[-1]),
    reported_days=int(slots.date.nunique()), reported_initial_soc_kwh=float(slots.soc_start.iloc
    [0]), reported_final_soc_kwh=float(slots.soc_end.iloc[-1]), strategy=parameters, verification=
    checks)
    output.mkdir(parents=True)
    slots.to_csv(output / 'slots.csv', index=False, float_format='%.12f')
    daily.to_csv(output / 'daily.csv', float_format='%.12f')
    (output / 'summary.json').write_text(json.dumps(result, ensure_ascii=False, indent=2) + '\n')
    return result

def sha(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def rows(path: Path) -> list[dict]:
    with path.open(encoding='utf-8-sig', newline='') as handle:
        return list(csv.DictReader(handle))

def number(row: dict, key: str) -> float:
    if key not in row or row[key] in (None, ''):
        raise ValueError(f'Missing required value {key}: {row}')
    value = float(row[key])
    if not math.isfinite(value):
        raise ValueError(f'Nonfinite {key}')
    return value

def close(a: float, b: float, label: str) -> None:
    if abs(a - b) > max(1e-05, 1e-08 * abs(b)):
        raise ValueError(f'Reconciliation failed {label}: {a} != {b}')

def clock(slot: int) -> str:
    return f'{slot // 6:02d}:{10 * (slot % 6):02d}'

def interval(slot: int, stop: int | None=None) -> str:
    return f'{clock(slot)}-{clock(slot + 1 if stop is None else stop)}'

def is_effective_emergency(value: float) -> bool:
    return value > EMERGENCY_TOL_KWH

def excel_date(iso: str) -> int:
    return (date.fromisoformat(iso) - date(1899, 12, 30)).days

def q1_payload(root: Path) -> dict:
    path = root / 'q1' / 'schedule.csv'
    records = rows(path)
    summary_path = root / 'summaries.json'
    summary = json.loads(summary_path.read_text(encoding='utf-8'))['q1']
    if len(records) != 144 or [int(r['slot_index']) for r in records] != list(range(144)):
        raise ValueError('Q1 must contain exactly ordered slots 0..143')
    for (i, row) in enumerate(records):
        if row['interval'] != interval(i):
```

```
            raise ValueError(f'Q1 time mapping differs at slot {i}')
    battery = []
    for group in range(6):
        part = records[24 * group:24 * (group + 1)]
        battery.append([interval(24 * group, 24 * (group + 1)), sum((number(r, 'charge_kwh') for r in part)), sum((number(r, 'discharge_kwh') for r in part)), '00:00' if group == 0 else '24:00' if group == 1 else None, number(records[0], 'energy_start_kwh') if group == 0 else number(records[-1], 'energy_end_kwh') if group == 1 else None])
    grid = sum((number(r, 'grid_kwh') for r in records))
    cost = sum((number(r, 'purchase_cost_yuan') for r in records))
    close(grid, summary['grid_kwh'], 'Q1 grid')
    close(cost, summary['cost_yuan'], 'Q1 cost')
    return {'key': 'q1', 'filename': _FILE_BY_CASE['q1'], 'plan': [[interval(i), number(r, 'grid_kwh')] for (i, r) in enumerate(records)], 'battery': battery, 'controls': {'grid_kwh': grid, 'cost_yuan': cost}, 'source_hashes': {str(path.relative_to(ROOT)): sha(path), str(summary_path.relative_to(ROOT)): sha(summary_path)}}

def annual_payload(root: Path, key: str) -> dict:
    (slot_path, daily_path) = (root / 'annual_slots.csv', root / 'annual_daily.csv')
    records = [r for r in rows(slot_path) if r['case'] == key]
    daily_rows = [r for r in rows(daily_path) if r['case'] == key]
    daily = {r['date']: r for r in daily_rows}
    if len(daily) != len(daily_rows):
        raise ValueError(f'{key}: duplicate daily date keys')
    by_date: dict[str, list[dict]] = {}
    for row in records:
        by_date.setdefault(row['date'], []).append(row)
    expected = [(date(2025, 2, 1) + timedelta(days=i)).isoformat() for i in range(334)]
    if any((day not in by_date or day not in daily for day in expected)):
        raise ValueError(f'{key}: missing reporting days')
    (plan, adjusted, battery, emergency) = ([], [], [], [])
    total_controls = {x: 0.0 for x in ('plan', 'adjusted', 'charge', 'discharge', 'emergency', 'cost_plan', 'cost_up', 'cost_down', 'cost_emergency', 'cost_total')}
    for day in expected:
        part = sorted(by_date[day], key=lambda r: int(r['slot']))
        if len(part) != 144 or [int(r['slot']) for r in part] != list(range(144)):
            raise ValueError(f'{key}/{day}: slots are not exactly 0..143')
        sums = {field: sum((number(r, field) for r in part)) for field in total_controls}
        for (field, value) in sums.items():
            close(value, number(daily[day], field), f'{key}/{day}/{field}')
            total_controls[field] += value
        for row in part:
            close(number(row, 'cost_total'), sum((number(row, f) for f in ('cost_plan', 'cost_up', 'cost_down', 'cost_emergency'))), 'transaction sum')
        serial = excel_date(day)
        plan.append([serial] + [number(r, 'plan') for r in part] + [sums['plan'], sums['cost_plan']])
        adjusted.append([serial] + [number(r, 'adjusted') for r in part] + [sums['adjusted'], sums['cost_plan'] + sums['cost_up'] + sums['cost_down']])
        for group in range(6):
            block = part[group * 24:(group + 1) * 24]
            battery.append([serial if group == 0 else None, interval(group * 24, (group + 1) * 24), sum((number(r, 'charge') for r in block)), sum((number(r, 'discharge') for r in block)), '00:00' if group == 0 else '24:00' if group == 1 else None, number(part[0], 'soc_start') if group == 0 else number(part[-1], 'soc_end') if group == 1 else None])
        daily_emergency = []
        start = None
        for i in range(145):
            active = i < 144 and is_effective_emergency(number(part[i], 'emergency'))
            if active and start is None:
                start = i
            if not active and start is not None:
                daily_emergency.append([serial if not daily_emergency else None, interval(start, i), sum((number(r, 'emergency') for r in part[start:i]))])
                start = None
        if not daily_emergency:
```

```
            daily_emergency = [[serial, '无', 0.0]]
        close(sum((r[2] for r in daily_emergency)), sums['emergency'], 'emergency grouping')
        emergency.extend(daily_emergency)
    return {'key': key, 'filename': _FILE_BY_CASE[key], 'plan': plan, 'adjusted': adjusted if key in ('q3', 'q4_3') else None, 'battery': battery, 'emergency': emergency, 'controls': total_controls, 'first_date': expected[0], 'last_date': expected[-1], 'report_days': len(expected), 'source_hashes': {str(p.relative_to(ROOT)): sha(p) for p in (slot_path, daily_path)}}

def digest(data):
    return hashlib.sha256(data).hexdigest()

def column(index):
    result = ''
    while index:
        (index, remainder) = divmod(index - 1, 26)
        result = chr(65 + remainder) + result
    return result

def column_number(address):
    value = 0
    for char in re.match('[A-Z]+', address).group():
        value = value * 26 + ord(char) - 64
    return value

def value_of(cell, shared):
    node = cell.find('s:v', NS)
    if node is not None and node.text is not None:
        if cell.get('t') == 's':
            return shared[int(node.text)]
        return node.text if cell.get('t') in ('str', 'e') else float(node.text)
    node = cell.find('s:is', NS)
    return ''.join(node.itertext()) if node is not None else None

def contents(archive, sheet_path):
    shared = []
    if 'xl/sharedStrings.xml' in archive.namelist():
        shared = [''.join(n.itertext()) for n in ET.fromstring(archive.read('xl/sharedStrings.xml'))]
    root = ET.fromstring(archive.read(sheet_path))
    cells = {c.get('r'): c for c in root.findall('s:sheetData/s:row/s:c', NS)}
    values = {a: value_of(c, shared) for (a, c) in cells.items()}
    return (root, cells, values)

def dated_groups(rows):
    result = defaultdict(list)
    day = None
    for row in rows:
        if row[0] is not None:
            day = int(row[0])
        result[day].append(row)
    return result

def prepare():
    WORK.mkdir(parents=True, exist_ok=True)
    payloads = [q1_payload(ROOT / 'results/c2026') if key == 'q1' else annual_payload(ROOT / 'results/c2026', key) for key in KEYS]
    books = []
    for (filename, payload) in zip(FILES, payloads):
        template = RESULTS / 'submission' / filename
        sheets = []
        with zipfile.ZipFile(template) as z:
            names = [n.attrib['name'] for n in ET.fromstring(z.read('xl/workbook.xml')).find('s:sheets', NS)]
            for (index, name) in enumerate(names, 1):
                member = f'xl/worksheets/sheet{index}.xml'
```

```
                (root, cells, values) = contents(z, member)
                (fills, blocks, dates) = ({}, [], set())

                def assign(address, value):
                    assert address not in fills
                    assert cells.get(address) is None or cells[address].find('s:f', NS) is None
                    if isinstance(value, (int, float)):
                        assert math.isfinite(value)
                    fills[address] = value
                if name in ('计划购电量', '调整购电量'):
                    if payload['key'] == 'q1':
                        matrix = [[r[1]] for r in payload['plan']]
                        for (row, line) in enumerate(matrix, 2):
                            assign(f'B{row}', line[0])
                        blocks.append({'range': 'B2:B145', 'values': matrix})
                    else:
                        rows = payload['plan' if name == '计划购电量' else 'adjusted']
                        by_date = {int(row[0]): row[1:] for row in rows}
                        matrix = []
                        for row in range(2, 336):
                            day = int(values[f'A{row}'])
                            dates.add(day)
                            line = by_date[day]
                            matrix.append(line)
                            for (col, number) in enumerate(line, 2):
                                assign(f'{column(col)}{row}', number)
                        blocks.append({'range': 'B2:EQ335', 'values': matrix})
                elif name == '充放电量':
                    if payload['key'] == 'q1':
                        for (row, line) in enumerate(payload['battery'], 2):
                            assign(f'B{row}', line[1])
                            assign(f'C{row}', line[2])
                            if line[4] is not None:
                                assign(f'E{row}', line[4])
                    else:
                        by_date = dated_groups(payload['battery'])
                        day = None
                        for row_node in root.findall('s:sheetData/s:row', NS)[1:]:
                            row = int(row_node.get('r'))
                            label = values.get(f'A{row}')
                            if isinstance(label, (int, float)):
                                day = int(label)
                                dates.add(day)
                            elif label is not None:
                                day = None
                            if day is None:
                                continue
                            period = values.get(f'B{row}')
                            hour = int(str(period).split(':')[0])
                            assert hour in (0, 4, 8, 12, 16, 20), (filename, row, period)
                            record = by_date[day][hour // 4]
                            assign(f'C{row}', record[2])
                            assign(f'D{row}', record[3])
                            point = values.get(f'E{row}')
                            if point is not None:
                                assert point in (0, '24:00'), point
                                assign(f'F{row}', by_date[day][0 if point == 0 else 1][5])
                elif name == '紧急购电量':
                    available = defaultdict(list)
                    day = None
                    for row_node in root.findall('s:sheetData/s:row', NS)[1:]:
                        row = int(row_node.get('r'))
                        label = values.get(f'A{row}')
                        if isinstance(label, (int, float)):
                            day = int(label)
                            dates.add(day)
```

```
                        elif label is not None:
                            day = None
                        if day is not None:
                            available[day].append(row)
                    grouped = dated_groups(payload['emergency'])
                    for (day, positions) in available.items():
                        events = grouped[day]
                        assert len(events) <= len(positions), (filename, day, 'Insufficient template rows')
                        for (row, event) in zip(positions, events):
                            assign(f'B{row}', event[1])
                            assign(f'C{row}', event[2])
                else:
                    raise ValueError(name)
                if not blocks:
                    blocks = [{'range': a, 'values': [[v]]} for (a, v) in fills.items()]
                sheets.append({'name': name, 'member': member, 'fills': fills, 'blocks': blocks, 'dates': [str(date(1899, 12, 30) + timedelta(days=d)) for d in sorted(dates)], 'dimension': root.find('s:dimension', NS).get('ref'), 'preview': 'A1:F8' if name == '充放电量' else 'A1:C11' if name == '紧急购电量' else 'A1:H5' if payload['key'] != 'q1' else 'A1:B8'})
        books.append({'filename': filename, 'template': str(template), 'template_sha256': digest(template.read_bytes()), 'sheets': sheets, 'controls': payload['controls'], 'source_hashes': payload['source_hashes']})
    (WORK / 'fill_plan.json').write_text(json.dumps({'workbooks': books}, ensure_ascii=False))
    print(json.dumps({b['filename']: {s['name']: {'cells': len(s['fills']), 'dates': s['dates'] if len(s['dates']) < 10 else '334 days'} for s in b['sheets']} for b in books}, ensure_ascii=False))

def data_cell(address, value, original=None):
    if original is None:
        opening = f'<c r="{address}">'
    else:
        opening = re.match('<c\\b[^>]*>', original).group().replace('/>', '>')
    if isinstance(value, str):
        opening = re.sub('\\s+t="[^"]*"', '', opening)[:-1] + ' t="inlineStr">'
        inside = '<is><t>' + escape(value) + '</t></is>'
    else:
        opening = re.sub('\\s+t="[^"]*"', '', opening)
        inside = '<v>' + format(float(value), '.17g') + '</v>'
    return opening + inside + '</c>'
CELL = re.compile('<c\\b[^>]*?(?:/>|>.*?</c>)', re.S)
ROW = re.compile('(<row\\b[^>]*>)(.*?)(</row>)', re.S)

def patch_sheet(xml, fills):
    grouped = defaultdict(dict)
    for (address, value) in fills.items():
        grouped[int(re.search('\\d+', address).group())][address] = value
    consumed = set()

    def row_replace(match):
        row = int(re.search('\\br="(\\d+)"', match.group(1)).group(1))
        if row not in grouped:
            return match.group()
        wanted = grouped[row]
        existing = {}
        remainder = CELL.sub('', match.group(2))
        assert not remainder.strip(), remainder
        for cell in CELL.finditer(match.group(2)):
            address = re.search('\\br="([A-Z]+\\d+)"', cell.group()).group(1)
            existing[address] = cell.group()
        for (address, value) in wanted.items():
            existing[address] = data_cell(address, value, existing.get(address))
            consumed.add(address)
        ordered = ''.join((existing[a] for a in sorted(existing, key=column_number)))
        return match.group(1) + ordered + match.group(3)
    result = ROW.sub(row_replace, xml)
```

```
    assert consumed == set(fills)
    return result

def finalize(verify_artifact=False):
    plan = json.loads((WORK / 'fill_plan.json').read_text())
    reports = []
    staging = WORK / 'filled'
    staging.mkdir(exist_ok=True)
    for book in plan['workbooks']:
        template = Path(book['template'])
        assert digest(template.read_bytes()) == book['template_sha256']
        destination = staging / book['filename']
        with zipfile.ZipFile(template) as original:
            replacements = {}
            for sheet in book['sheets']:
                replacements[sheet['member']] = patch_sheet(original.read(sheet['member']).decode(), sheet['fills']).encode()
            with zipfile.ZipFile(destination, 'w') as out:
                out.comment = original.comment
                for info in original.infolist():
                    out.writestr(info, replacements.get(info.filename, original.read(info.filename)))
        with zipfile.ZipFile(template) as original, zipfile.ZipFile(destination) as result:
            assert original.namelist() == result.namelist()
            for member in original.namelist():
                if member not in replacements:
                    assert original.read(member) == result.read(member), member
            for sheet in book['sheets']:
                (old_root, old_cells, old_values) = contents(original, sheet['member'])
                (new_root, new_cells, new_values) = contents(result, sheet['member'])
                assert [(r.tag, r.attrib) for r in old_root.findall('s:sheetData/s:row', NS)] == [(r.tag, r.attrib) for r in new_root.findall('s:sheetData/s:row', NS)]
                for (a, c) in old_cells.items():
                    if a not in sheet['fills']:
                        assert ET.tostring(c) == ET.tostring(new_cells[a]), (sheet['name'], a)
                    else:
                        assert {k: v for (k, v) in c.attrib.items() if k != 't'} == {k: v for (k, v) in new_cells[a].attrib.items() if k != 't'}, (book['filename'], sheet['name'], a)
                for (a, v) in sheet['fills'].items():
                    actual = new_values[a]
                    assert actual == v or (isinstance(v, (int, float)) and math.isclose(actual, v, rel_tol=1e-13, abs_tol=1e-10))
                old_root.remove(old_root.find('s:sheetData', NS))
                new_root.remove(new_root.find('s:sheetData', NS))
                assert ET.tostring(old_root) == ET.tostring(new_root), sheet['name']
        for (relative, expected) in book['source_hashes'].items():
            assert digest((ROOT / relative).read_bytes()) == expected, relative
        reports.append({'filename': book['filename'], 'template_sha256': book['template_sha256'], 'output_sha256': digest(destination.read_bytes()), 'only_authorized_data_cells_changed': True, 'source_unchanged': True, 'independent_saved_xlsx_check': {'engine': 'OOXML readback', 'passed': True, 'checked_cells': sum((len(s['fills']) for s in book['sheets'])), 'scope': 'All filled cells and all preserved original cells and workbook parts'}, 'unchanged_row_count_and_attributes': True, 'unchanged_styles_headers_metadata_and_other_parts': True, 'filled_cells': sum((len(s['fills']) for s in book['sheets'])), 'sheet_coverage': {s['name']: s['dates'] if len(s['dates']) < 10 else '334 days' for s in book['sheets']}})
    (WORK / 'audit.json').write_text(json.dumps({'schema': 'c2026-template-data-only-v1', 'row_policy': 'Preserve all base workbook rows and existing labels, including omission marks.', 'time_policy': 'Preserve template labels; retain the accepted ordinal slot mapping.', 'workbooks': reports}, ensure_ascii=False, indent=2) + '\n')
    print(json.dumps(reports, ensure_ascii=False))

def main():
    global WORK
    parser = argparse.ArgumentParser()
    parser.add_argument('phase', choices=['verify', 'rebuild'])
    parser.add_argument('--out', type=Path)
```

```
    args = parser.parse_args()
    if args.phase == 'verify':
        report = {}
        frames = pd.read_csv(RESULTS / 'annual_slots.csv')
        for name in ('q2', 'q3', 'q4_2', 'q4_3'):
            frame = frames.loc[frames['case'] == name]
            report[name] = verify_frame(frame)
        print(json.dumps(report, indent=2))
        return
    if args.out is None:
        parser.error('--out is required for rebuild')
    if args.out.exists():
        raise FileExistsError(args.out)
    WORK = args.out.resolve()
    prepare()
    finalize(False)
if __name__ == '__main__':
    main()
```